

glucosense_ai



Ikram Ullah Afridi

FA22-CSE-040

Syed Musawer Shah

FA22-CSE-042

M. Aslam

FA22-CSE-051

Session 2022-2026

Department of Computer Systems Engineering

Faculty of Engineering & Technology

Mirpur University of Science and Technology (MUST), Mirpur-10250

glucosense_ai

By

Ikram Ullah Afridi FA22-CSE-040

Syed Musawer Shah FA22-CSE-042

M. Aslam FA22-CSE-051

A thesis submitted in partial fulfillment of the requirement for

the degree of

Bachelors in Computer Systems Engineering

In

Engineering

Session 2022-2026

Department of Computer Systems Engineering

Faculty of Engineering & Technology

Mirpur University of Science and Technology (MUST), Mirpur-10250

CERTIFICATION

This is to certify that we have read the report submitted by **Ikram Ullah Afridi (FA22-CSE-040)**, **Syed Musawer Shah (FA22-CSE-042)**, and **M. Aslam (FA22-CSE-051)** in partial fulfillment of the requirements for the degree of Bachelors in Computer Systems Engineering (BSc CSE).

Student's Name:

Registration No:

Ikram Ullah Afridi

MUST/FA22-CSE-040/AJK

Signature: _____

Syed Musawer Shah

MUST/FA22-CSE-042/AJK

Signature: _____

M. Aslam

MUST/FA22-CSE-051/AJK

Signature: _____

Date: _____

We believe that this report meets the required standards and should be accepted by Mirpur University of Science and Technology for the Bachelor's degree in Computer Systems Engineering (BSCSE) without any issues.

Supervisor: **Engr. Irfan Jamshed**

External Examiner: _____

Name: _____

University Address: _____

Chairperson: _____

DEDICATION

I dedicate this Final Year Project to my parents, whose unconditional love, prayers, and sacrifices have been the foundation of my journey.

To my family, teachers, and friends, whose constant support and encouragement inspired me to keep moving forward and achieve this milestone.

TABLE OF CONTENTS

CERTIFICATION	iii
DEDICATION	iv
TABLE OF CONTENTS.....	v
LIST OF FIGURES	xiii
LIST OF TABLES	xiv
COPY RIGHTS	xv
ABSTRACT.....	xvi
SUSTAINABLE DEVELOPMENT GOALS (SDGS)	xvii
ACKNOWLEDGMENTS	xviii
CHAPTER 01	1
INTRODUCTION	1
1.0 Introduction.....	1
1.1 Project Overview	2
Glucose Monitoring	3
Meal Analysis	3
Diabetes Risk Prediction.....	4
Trends and Reports	5
Reminders and Notifications.....	5
1.2 Problem Description	5
1.3 Project Objectives	7
1.4 Project Scope	8
1.4.1 User Authentication and Onboarding	8
1.4.2 Medical Profile Management.....	9
1.4.3 Glucose Reading Management	9
1.4.4 AI-Based Meal Analysis	9
1.4.5 Diabetes Risk Prediction.....	10
1.4.6 Trends and Reporting.....	11
1.4.7 Reminder Management.....	11
1.4.8 Platform Scope.....	11
1.4.9 Limitations within the Current Scope	11
1.5 Tools and Technologies	12
1.5.1 Flutter	12
1.5.2 Dart	12

1.5.3 BLoC and Cubit	12
1.5.4 Firebase Authentication	13
1.5.5 Cloud Firestore.....	13
1.5.6 Firebase Storage	13
1.5.7 Supabase	13
1.5.8 Google Gemini.....	14
1.5.9 Python	14
1.5.10 Scikit-learn.....	14
1.5.11 XGBoost	14
1.5.12 StandardScaler	15
1.5.13 Flask.....	15
1.5.14 SHAP	16
1.5.15 LIME.....	16
1.5.16 Dio.....	16
1.5.17 Hive.....	16
1.5.18 SharedPreferences and Secure Storage.....	16
1.5.19 Flutter Local Notifications	17
1.5.20 Development and Testing Tools	17
1.6 Chapter Summary	18
CHAPTER 02	19
LITERATURE REVIEW	19
2.0 Introduction.....	19
2.1 Mobile Technology for Diabetes Self-Management.....	20
2.2 Diabetes Prevention Through Smartphone Applications.....	20
2.3 Existing Diabetes Applications.....	21
2.3.1 mySugr	22
2.3.2 Glucose Buddy.....	22
2.4 Limitations Observed in Existing Diabetes Applications	23
2.5 Machine Learning for Diabetes Risk Prediction.....	24
2.6 Logistic Regression for Diabetes Prediction.....	25
2.7 Random Forest for Diabetes Prediction.....	25
2.8 XGBoost for Diabetes Prediction	26
2.9 Prediabetes Risk Prediction	27
2.10 Importance of Diabetes-Related Features	28

2.11 Explainable Artificial Intelligence in Diabetes Prediction	29
2.11.1 SHAP	30
2.11.2 LIME	30
2.12 Image-Based Food Recognition and Dietary Analysis	31
2.13 AI-Assisted Meal Analysis in glucosense_ai	32
2.14 Glucose Trends and Reporting	32
2.15 Reminders and User Adherence	33
2.16 Cloud-Based Data Management	34
2.17 Offline-Aware Mobile Design	35
2.18 Comparison of Existing Approaches	35
2.19 Research Gap	36
2.20 Limitations Identified from the Literature	38
2.21 Chapter Summary	39
References Used in Chapter 2	40
CHAPTER 03	42
METHODOLOGY / PROPOSED SYSTEM DESIGN	42
3.0 Proposed System	42
3.1 Functional Requirements	43
3.1.1 Authentication and Onboarding Module	43
3.1.2 Profile Management Module	44
3.1.3 Glucose Logging Module	44
3.1.4 Meal Prediction Module	44
3.1.5 Diabetes Risk Prediction Module	45
3.1.6 Trends and Reports Module	46
3.1.7 Reminder and Notification Module	46
3.1.8 Data Storage and Synchronization Module	46
3.2 Non-Functional Requirements	47
3.2.1 Performance	47
3.2.2 Security	47
3.2.3 Usability	47
3.2.4 Reliability	48
3.2.5 Maintainability	48
3.2.6 Scalability	48
3.2.7 Compatibility	49

3.3 Overall Testing Strategy	49
3.4 Chapter Summary	49
CHAPTER 04	50
SYSTEM DESIGN AND IMPLEMENTATION.....	50
4.0 Introduction.....	50
4.1 System Architecture.....	51
4.2 Components of the System Architecture	53
4.2.1 Flutter Mobile Application	53
4.2.2 Firebase Authentication	53
4.2.3 Cloud Firestore.....	54
4.2.4 Firebase Storage.....	54
4.2.5 Supabase	55
4.2.6 Google Gemini API	55
4.2.7 Flask Prediction API.....	55
4.2.8 XGBoost Prediction Model.....	56
4.2.9 Local Storage	57
4.2.10 Notification Service	57
4.3 Application Architecture.....	58
4.3.1 Presentation Layer	60
4.3.2 Domain Layer	60
4.3.3 Data Layer.....	61
4.4 Use Case Diagram.....	62
4.5 Use Case Tabular Description	64
4.5.1 Sign Up Use Case	64
4.5.2 Login Use Case.....	64
4.5.3 Complete Medical Profile Use Case	64
4.5.4 Log Glucose Reading Use Case.....	65
4.5.5 View Glucose History Use Case.....	65
4.5.6 Analyze Meal Use Case	65
4.5.7 Save Analyzed Meal Use Case	66
4.5.8 Predict Diabetes Risk Use Case.....	66
4.5.9 View Risk Result and Advice Use Case	66
4.5.10 View Trends Use Case.....	66
4.5.11 Generate Report Use Case	67

4.5.12 Manage Reminders Use Case	67
4.5.13 View Notification History Use Case.....	68
4.5.14 Edit Profile Use Case	68
4.5.15 Logout Use Case	68
4.6 Class Diagram	69
User	70
MedicalProfile.....	70
GlucoseReading	71
Meal	71
MealAnalysis	72
RiskPrediction.....	72
Reminder.....	73
NotificationLog.....	73
4.7 Sequence Diagrams.....	74
4.7.1 Sequence Diagram for Glucose Logging.....	74
4.7.2 Sequence Diagram for Meal Photo Analysis	76
4.7.3 Sequence Diagram for Diabetes Risk Prediction.....	78
4.7.4 Sequence Diagram for Reminder Scheduling and Notification Delivery.....	79
4.7.5 Sequence Diagram for Viewing Trends and Reports	81
4.8 Machine Learning Pipeline	82
4.9 Dataset and Target Preparation	84
4.10 Feature Engineering	85
4.11 Data Splitting	86
4.12 Feature Scaling.....	87
4.13 Model Training Workflow	88
Logistic Regression.....	88
Random Forest	88
XGBoost	88
4.14 Prediction API Workflow	90
4.15 Prediction API Endpoints	92
4.15.1 Health Endpoint	92
Request.....	92
4.15.2 Prediction Endpoint	92
Request.....	92

4.16 Reminder System Implementation.....	93
Notification Service	93
Reminder Scheduler.....	94
Notification History	94
4.17 Error Handling	94
4.18 Security Considerations	95
4.19 Chapter Summary	96
CHAPTER 05	97
TESTING, RESULTS, AND DISCUSSION	97
5.0 Testing Methodologies.....	97
5.0.1 Black-Box Testing	97
5.0.2 Integration Testing	97
5.0.3 API Testing	97
5.0.4 Machine Learning Model Testing.....	98
5.0.5 Notification Testing	98
5.0.6 Error Handling Testing	98
5.1 Test Cases	99
5.2 Model Selection	103
5.3 Model Evaluation.....	104
5.4 Confusion Matrix	105
5.5 Model Explainability	106
5.5.1 SHAP Analysis	106
5.5.2 LIME Analysis.....	106
CHAPTER 06	107
USER INTERFACE AND USER EXPERIENCE	107
6.0 Proposed System Interface & User Experience	107
6.1 Splash Screen	107
6.2 Onboarding Screen.....	109
6.3 Login Screen	110
6.4 Signup Screen	111
6.5 Forgot Password Screen.....	112
6.6 Medical Profile Setup Screen.....	113
6.7 Home Dashboard	115
6.8 Glucose Logging Screen	117

6.9 Glucose History Screen.....	118
6.10 Meal Predictor Screen.....	119
6.11 Meal Analysis Result Screen	121
6.12 Diabetes Risk Prediction Screen.....	122
6.13 Risk Prediction Result Screen.....	124
6.14 Trends Screen.....	125
6.15 Reports Screen	127
6.16 Reminder Settings Screen	128
6.17 Notification Delivery and Smart Reminder Interface.....	129
6.18 Profile Screen.....	130
6.19 Edit Profile Screen	131
6.20 Application Navigation Flow.....	132
6.21 Overall Application Navigation.....	133
6.22 User Experience Considerations	134
6.23 Conclusion	135
CHAPTER 07	136
CONCLUSION AND FUTURE WORK	136
7.0 Conclusion	136
7.1 Current Technical Limitations	136
7.2 Future Work.....	137
7.3 Enhanced Diabetes Risk Prediction.....	138
7.3.1 Larger and More Diverse Dataset.....	138
7.3.2 Additional Clinical Features	139
7.3.3 Advanced Machine-Learning Models.....	139
7.4 Clinical Validation of the Prediction System.....	140
7.5 Improved Model Explainability.....	140
7.6 Personalized Health Recommendations.....	141
7.7 Enhanced Meal Analysis.....	142
7.7.1 Improved Food Recognition	142
7.7.2 Meal Risk Scoring.....	143
7.7.3 Meal and Glucose Correlation	143
7.8 Integration with Continuous Glucose Monitoring Devices	143
7.9 Wearable Device Integration	144
7.10 Advanced Trends and Analytics	145

7.11 Smart Alerts and Early Warning System	146
7.12 Improved Reports and Healthcare Sharing	146
7.13 Doctor or Healthcare Professional Module.....	147
7.14 Improved API Input Validation	148
7.15 Model Versioning and Monitoring	148
7.16 Offline Functionality and Synchronization.....	149
7.17 Security and Privacy Enhancements	150
7.18 Scalability and Performance Optimization	151
7.19 Cross-Platform and Accessibility Improvements	151
7.20 Future Integration with Healthcare Systems.....	152

LIST OF FIGURES

Figure No.	Page No.
Figure 4.1 System Architecture Diagram of glucosense_ai	31
Figure 4.2 Application Architecture of glucosense_ai	36
Figure 4.3 Use Case Diagram of glucosense_ai	38
Figure 4.4 Class Diagram of glucosense_ai	44
Figure 4.5 Sequence Diagram for Glucose Logging	47
Figure 4.6 Sequence Diagram for Meal Photo Analysis	48
Figure 4.7 Sequence Diagram for Diabetes Risk Prediction	49
Figure 4.8 Sequence Diagram for Reminder Scheduling and Notification Delivery	50
Figure 4.9 Sequence Diagram for Viewing Trends and Reports	51
Figure 4.10 Machine Learning Pipeline for Diabetes Risk Prediction	51
Figure 4.11 Model Training Workflow	55
Figure 4.12 Prediction API Workflow	58
Figure 6.1 Splash Screen	91
Figure 6.2 Onboarding Screen	91
Figure 6.3 Login Screen	91
Figure 6.4 Signup Screen	92
Figure 6.5 Forgot Password Screen	92
Figure 6.6 Medical Profile Setup Screen	92
Figure 6.7 Home Dashboard	93
Figure 6.8 Glucose Logging Screen	94
Figure 6.9 Glucose Reading History	94
Figure 6.10 Meal Predictor Screen	94
Figure 6.11 AI Meal Analysis Result	95
Figure 6.12 Saved Meals Screen	95
Figure 6.13 Diabetes Risk Prediction Screen	96
Figure 6.14 Diabetes Risk Prediction Result	96
Figure 6.15 Glucose Trends Screen	97
Figure 6.16 Reports Screen	97
Figure 6.17 Reminder Settings Screen	98
Figure 6.18 Smart Reminder Notification	98
Figure 6.19 Notification History Screen	99
Figure 6.20 User Profile Screen	99
Figure 6.21 Edit Profile Screen	99

LIST OF TABLES

Table No.	Page No.
Table 2.1 Comparison of Existing Diabetes Applications.....	26
Table 4.1 Sign Up Use Case	39
Table 4.2 Login Use Case.....	40
Table 4.3 Complete Medical Profile Use Case.....	40
Table 4.4 Log Glucose Reading Use Case	40
Table 4.5 View Glucose History Use Case	41
Table 4.6 Analyze Meal Use Case.....	41
Table 4.7 Save Analyzed Meal Use Case	41
Table 4.8 Predict Diabetes Risk Use Case.....	41
Table 4.9 View Risk Result and Advice Use Case.....	42
Table 4.10 View Trends Use Case	42
Table 4.11 Generate Report Use Case	42
Table 4.12 Manage Reminders Use Case	43
Table 4.13 View Notification History Use Case	43
Table 4.14 Edit Profile Use Case.....	43
Table 4.15 Logout Use Case.....	43
Table 4.16 Diabetes Risk Category Mapping.....	52
Table 4.17 Model Feature List.....	53
Table 5-1 User Login Test Case	73
Table 5-2 Invalid Login Test Case	73
Table 5-3 Medical Profile Setup Test Case	74
Table 5-4 Log Glucose Reading Test Case	74
Table 5-5 Meal Image Analysis Test Case	74
Table 5-6 Save Analysed Meal Test Case	75
Table 5-7 Diabetes Risk Prediction Test Case	75
Table 5-8 Missing Prediction Field Test Case.....	75
Table 5-9 View Glucose Trends Test Case	75
Table 5-10 Generate Report Test Case	76
Table 5-11 Schedule Reminder Test Case	76
Table 5-12 Reminder Deeplink Test Case	76
Table 5-13 Smart Reminder Suppression Test Case	76
Table 5-14 Update User Profile Test Case	77
Table 5-15 Flask API Health Test Case	77

COPY RIGHTS

We hereby declare that this document "glucosense_ai" neither as a whole nor as a part has been copied out from any source. It is further declared that we have completed this project and the accompanying report entirely on the basis of our personal efforts, under the guidance of our supervisor, Engr. Irfan Jamshed. If any part of the system is found to be copied from any source or found to be a reproduction of any project from elsewhere, we shall stand by the consequences.

ABSTRACT

glucosense_ai is a mobile-based diabetes monitoring and risk-assessment application that brings glucose tracking, meal analysis, trend visualization, reminders, and machine-learning-based risk prediction together in a single platform. Built with Flutter and backed by Firebase and Supabase, the application allows users to maintain a medical profile, record fasting, pre-meal, and post-meal glucose readings, and review their history through trends and reports. A dedicated meal-analysis feature uses Google Gemini to interpret photographs of meals and estimate nutritional and glycaemic information. The diabetes-risk prediction component classifies a user into Normal, Prediabetic, or Diabetic categories using an XGBoost classifier, selected after comparison with Logistic Regression and Random Forest models on the basis of weighted F1-score and overall classification performance; the prediction is accessed through a Flask API and is intended to provide an understandable risk indication rather than a medical diagnosis. A reminder system with smart suppression helps users log readings at appropriate times without redundant notifications. Together, these features are intended to give users, particularly those living with or at risk of diabetes, a more organized and informative way of managing their day-to-day health information.

SUSTAINABLE DEVELOPMENT GOALS (SDGS)

The Sustainable Development Goals (SDGs) are the framework to achieve a better and more sustainable future for all. They address global challenges including poverty, inequality, climate change, environmental degradation, and peace and justice. The 17 SDGs are listed below, with the goal(s) most closely related to this project indicated.

1. No Poverty
2. Zero Hunger
- 3. Good Health and Well Being ← Primary goal addressed by glucosense_ai**
4. Quality Education
5. Gender Equality
6. Clean Water and Sanitation
7. Affordable and Clean Energy
8. Decent Work and Economic Growth
9. Industry Innovation and Infrastructure
10. Reduced Inequalities
11. Sustainable Cities and Communities
12. Responsible Consumption and Production
13. Climate Action
14. Life below Water
15. Life on Land
16. Peace and Justice and Strong Institutions
17. Partnership to Achieve the Goals

ACKNOWLEDGMENTS

First of all, we are grateful to Allah Almighty, the Merciful and the source of all knowledge, for granting us the ability and opportunity to complete this project. We are sincerely thankful to our supervisor, Engr. Irfan Jamshed, for his guidance, patience, and valuable feedback throughout the development of glucosense_ai. We would also like to thank our teachers in the Department of Computer Systems Engineering for their cooperation and support during our time at MUST, and our families and classmates for their continued encouragement.

CHAPTER 01

INTRODUCTION

1.0 Introduction

Diabetes is a long term health condition that requires regular attention to blood glucose levels, lifestyle habits, food intake, and other medical factors. For many people, keeping track of all this information can become difficult, especially when glucose readings, meal records, reminders, and health-related information are managed separately. A person may record glucose levels in one place, receive dietary information from another source, and still have no simple way to understand how these factors relate to their future diabetes risk.

The growing use of mobile applications and artificial intelligence has created opportunities to make this process easier. Instead of using a mobile application only as a digital notebook for glucose readings, modern systems can analyse collected information and provide meaningful feedback to the user. Features such as meal-image analysis, glucose trend visualization, reminders, and machine-learning-based risk prediction can help users better understand their daily health information.

glucosense_ai is a mobile-based diabetes monitoring and risk-assessment system developed to bring these functions together in one application. The application allows users to maintain their medical profile, record fasting, pre-meal, and post-meal glucose readings, analyse meals using photographs, view glucose trends and reports, configure reminders, and receive diabetes-risk predictions. The application also keeps previously analysed meals and notification history so that users can review their information when needed.

The system combines a Flutter mobile application with cloud services and artificial intelligence. Firebase is used for authentication, database management, and storage, while Supabase is also available for realtime and additional backend services. Google Gemini is

used for meal-image analysis, whereas a separately trained machine-learning model is accessed through a Flask API for diabetes-risk classification.

The diabetes-risk prediction component classifies a user's condition into three categories: **Normal, Prediabetic, and Diabetic**. The prediction is generated from demographic, clinical, behavioral, and glucose-related information. The machine-learning development process compares Logistic Regression, Random Forest, and XGBoost models, with XGBoost selected as the final classifier based on weighted F1-score and overall classification performance. The purpose of this prediction is not to replace medical diagnosis, but to provide an understandable risk indication that can encourage users to pay closer attention to their health information.

In addition to diabetes-risk prediction, glucosense_ai also provides meal analysis. A user can capture or select a photograph of a meal, after which the application uses Google Gemini to analyse nutritional information such as estimated macronutrients and glycaemic impact and to provide more suitable food suggestions.

Another important part of the application is its reminder system. Users can configure reminders for fasting readings, breakfast, lunch, dinner, and snacks. These reminders are scheduled on the device and can open the relevant glucose-logging screen directly when selected. The system also attempts to avoid unnecessary reminders when the required reading has already been recorded.

By combining glucose tracking, meal analysis, trend monitoring, reminders, and machine-learning-based risk prediction, glucosense_ai is intended to provide users with a more organized way of understanding and managing information related to diabetes.

1.1 Project Overview

glucosense_ai is a smart mobile application developed for users who are living with diabetes or may be at risk of developing it. The application focuses on regular glucose

monitoring while also providing supporting features that help users understand their eating habits, health profile, glucose trends, and predicted diabetes-risk category.

The system begins with user authentication. A new user can create an account using an email address and password, while an existing user can log in using previously registered credentials. Firebase Authentication is used to manage these authentication operations. After registration, the user completes an initial medical-profile setup containing information required by different parts of the system. The application includes a first-launch onboarding process before the user reaches the main application interface.

The main application is organized around a five-tab home interface consisting of **Home, Log, Meal, Risk, and Profile**. The Home section presents useful daily information and quick actions. The Log section allows the user to record glucose readings. The Meal section is used for AI-assisted meal analysis. The Risk section provides diabetes-risk information, while the Profile section gives access to personal and medical information along with reports, trends, reminders, and notification-related features.

Glucose Monitoring

The glucose-logging module allows users to record different types of glucose measurements, including:

- Fasting glucose
- Pre-meal glucose
- Post-meal glucose

Each record can contain the glucose value, related meal information, unit, timestamp, and other relevant details. These records are stored so that the user can later review their history and observe changes over time.

Meal Analysis

The meal-analysis feature allows users to capture a new meal photograph or select one from their device. The image is analysed using Google Gemini. The generated analysis can

provide estimated nutritional information, glycaemic-load-related insights, and alternatives that may be more suitable for a person concerned about glucose management. Analysed meals can also be saved for later viewing and reuse.

Diabetes Risk Prediction

The risk-prediction module uses a trained machine-learning model to classify users into one of three categories:

1. Normal
2. Prediabetic
3. Diabetic

Twelve input features are used by the prediction model:

1. Gender
2. Age
3. Hypertension
4. Heart disease
5. Smoking history
6. Activity level
7. Family history
8. Body Mass Index (BMI)
9. HbA1c level
10. Fasting glucose
11. Pre-meal glucose
12. Post-meal glucose

During model development, Logistic Regression, Random Forest, and XGBoost were trained and compared. XGBoost was selected as the final model. Before prediction, the input features are processed using a saved StandardScaler. The trained XGBoost model and scaler are exported separately and loaded by the Flask prediction API.

The API returns more than a single category. It provides the predicted risk category, confidence information, probabilities for individual classes, and short guidance text. This makes the result easier to present within the mobile application.

Trends and Reports

The application also provides a trends section where users can inspect their glucose information over different periods such as seven, thirty, and ninety days. Available information includes averages, ranges, time-in-range information, variation, and time-of-day patterns. Reports can also be created for a selected date range and can include glucose readings and meal-related information.

Reminders and Notifications

Users may schedule reminders for activities such as fasting glucose measurement and meal-related glucose readings. The application uses device-level notification services rather than depending only on a continuously active internet connection. Notifications can also deep-link to the glucose-entry screen so that the appropriate reading type is prepared for the user.

Overall, glucosense_ai combines monitoring and predictive functionality instead of treating them as separate applications. The aim is to provide one place where users can record health information, understand their glucose behavior, analyse meals, and receive a machine-learning-assisted indication of diabetes risk.

1.2 Problem Description

Managing diabetes-related information can become difficult because several factors influence a person's glucose condition. A single glucose reading provides useful information at a particular moment, but it does not always give a clear picture of a person's overall pattern. Users may need to consider fasting readings, glucose levels before and after meals, HbA1c, BMI, medical history, lifestyle habits, and other factors together.

One problem is that users often have to record and interpret this information manually. When readings are kept without proper organization, it becomes difficult to identify patterns over time. A user may know that a glucose reading is high on a particular day but may not easily understand whether glucose levels are consistently increasing or whether specific meals are repeatedly associated with higher readings.

Food selection introduces another challenge. Meal composition can directly affect blood glucose, but users may not always know the likely nutritional or glycaemic effect of the food they are eating. Manually searching for nutritional information for every meal is inconvenient and may discourage regular monitoring.

Regular glucose measurement also depends on consistency. Users may forget to take fasting, pre-meal, or post-meal readings at the required time. Simple reminders can help, but poorly designed notification systems may repeatedly notify users even after they have already completed the required action.

Another limitation of ordinary glucose-tracking applications is that they may focus mainly on recording historical measurements. Recording information is useful, but users can benefit more when the available information is also processed to provide a simple indication of their current risk level. A system that considers medical-profile information together with recent glucose data can offer a more informative view than isolated glucose values.

There is therefore a need for an integrated system that can:

- organize glucose measurements,
- maintain relevant personal and medical information,
- analyse meal images,
- provide glucose trends,
- remind users about routine measurements,
- and use machine learning to classify diabetes risk.

glucosense_ai addresses this problem by bringing these functions into one mobile application and connecting the application to cloud, AI, and machine-learning services.

The prediction module, however, is intended as a **risk-support feature rather than a medical diagnostic tool**. Its output should help users understand their recorded data and identify situations that may deserve further attention. A medical diagnosis should still be made by a qualified healthcare professional.

1.3 Project Objectives

The main objective of glucosense_ai is to develop a mobile-based system that supports glucose monitoring and provides intelligent diabetes-risk insights using artificial intelligence and machine learning.

The specific objectives of the project are:

- To develop a user-friendly mobile application for recording and managing glucose-related information.
- To allow users to create and maintain a personal and medical profile containing information relevant to diabetes-risk assessment.
- To provide separate logging options for fasting, pre-meal, and post-meal glucose readings.
- To maintain glucose history so users can review their previous measurements.
- To provide visual trends and summaries that make changes in glucose levels easier to understand.
- To integrate AI-based meal-image analysis so users can obtain nutritional and glycaemic-related information from a meal photograph.
- To allow users to save previously analysed meals for future reference.
- To develop a three-class diabetes-risk prediction system capable of classifying a user as Normal, Prediabetic, or Diabetic.

- To compare multiple machine-learning algorithms, including Logistic Regression, Random Forest, and XGBoost, and select a suitable model based on classification performance.
- To deploy the selected machine-learning model through a Flask-based prediction API so that it can be used by the mobile application.
- To provide prediction confidence, class probabilities, and short guidance together with the predicted risk category.
- To improve model transparency using explainability techniques such as SHAP and LIME.
- To provide customizable reminders for glucose measurements and meal-related activities.
- To maintain notification history so that users can review delivered reminders.
- To provide date-based reports containing glucose and meal-related information.
- To use secure authentication and structured cloud storage for user-related information.

These objectives are consistent with the application's implemented functionality, which covers authentication, profile setup, glucose logging, meal analysis, risk prediction, trends, reports, reminders, and profile management.

1.4 Project Scope

The scope of `glucosense_ai` covers the design and development of a mobile application for diabetes-related monitoring, meal analysis, reminders, data visualization, and machine-learning-assisted risk prediction.

The system is primarily intended for individual users or patients who want to record glucose information and better understand their health-related patterns.

1.4.1 User Authentication and Onboarding

The application provides:

- User registration
- User login
- Password reset
- Initial onboarding
- Medical-profile setup
- Session management

Firebase Authentication is used for email- and password-based account management.

1.4.2 Medical Profile Management

The system allows users to enter and update information relevant to diabetes-risk assessment. This includes information such as age, gender, BMI-related data, family history, hypertension, heart disease, smoking history, and activity level.

The medical profile is also used as one of the information sources for risk prediction.

1.4.3 Glucose Reading Management

Users can record and manage:

- Fasting readings
- Pre-meal readings
- Post-meal readings
- Meal-related information
- Reading timestamps
- Measurement units
- Additional notes where required

The recorded information can later be used for history, trends, reports, and risk assessment.

1.4.4 AI-Based Meal Analysis

The meal feature allows the user to provide a meal image. Google Gemini is used to interpret the image and return meal-related information such as estimated macronutrients, glycaemic impact, and potentially safer alternatives. Saved meal analyses can later be reopened by the user.

1.4.5 Diabetes Risk Prediction

The machine-learning component predicts one of three risk categories:

Normal → Prediabetic → Diabetic

The model uses twelve selected input variables representing demographic, health, lifestyle, and glucose information.

The machine-learning scope includes:

- Data preprocessing
- Target-class construction
- Feature engineering
- Stratified train-test splitting
- Feature scaling
- Training multiple classifiers
- Model comparison
- XGBoost model selection
- Classification evaluation
- Confusion-matrix generation
- SHAP analysis
- LIME explanation
- Model serialization
- API-based inference

The current implementation does not claim to provide a formal medical diagnosis. It provides an automated risk classification based on the data supplied to the model.

1.4.6 Trends and Reporting

The application provides glucose-related trends over different time periods, including seven-day, thirty-day, and ninety-day views. It also supports reports over a user-selected date range.

1.4.7 Reminder Management

The application allows users to manage reminders related to:

- Fasting readings
- Breakfast
- Lunch
- Dinner
- Snacks

Local notifications are scheduled through the device operating system. The implementation also contains notification-history functionality and reminder suppression where an appropriate reading has already been logged within the configured time window.

1.4.8 Platform Scope

The application is developed using Flutter, which supports mobile development for Android and iOS. The current project structure contains configuration for both platforms.

1.4.9 Limitations within the Current Scope

Some areas remain outside the completed scope or require further improvement.

First, the machine-learning repository snapshot does not currently contain the original training dataset, so exact final evaluation metrics cannot be independently regenerated from the available project files alone.

Second, the Flask prediction API checks whether required fields are present but still requires stronger validation for data types and clinically reasonable value ranges.

Third, the system provides diabetes-risk information and recommendations but should not be considered a replacement for professional medical examination, laboratory testing, or diagnosis.

Finally, the accuracy and clinical reliability of a machine-learning model depend strongly on the dataset used to train it. Broader clinical validation would therefore be required before such a system could be considered for real clinical decision-making.

1.5 Tools and Technologies

A combination of mobile-development, cloud, artificial-intelligence, machine-learning, and local-device technologies is used to develop glucosense_ai.

1.5.1 Flutter

Flutter is used to develop the mobile application. It provides a single codebase that can be used to build applications for Android and iOS.

The application interface, navigation, screens, forms, charts, and user interactions are implemented in Flutter. The project uses a feature-first structure so that major features remain separated and easier to manage.

1.5.2 Dart

Dart is the main programming language used for the Flutter application. It is used to implement application logic, models, repositories, state management, and user-interface behavior.

1.5.3 BLoC and Cubit

The flutter_bloc package is used for state management. Depending on feature complexity, the application uses either BLoC or Cubit.

This separates the user interface from application logic. Instead of screens directly communicating with Firebase or AI services, the application follows a flow similar to:

Widget → Cubit/BLoC → Repository → Data Source → External Service

This structure improves organization and testability.

1.5.4 Firebase Authentication

Firebase Authentication is used for:

- User registration
- Login
- Authentication-state management
- Password-related authentication operations

It provides the main authentication mechanism for application users.

1.5.5 Cloud Firestore

Cloud Firestore is used as a cloud database for storing application information such as user profiles and health-related records.

The application architecture keeps database communication within data sources and repositories rather than directly accessing Firestore from user-interface widgets.

1.5.6 Firebase Storage

Firebase Storage is used for file and image storage where required by the application, particularly for image-related functionality.

1.5.7 Supabase

Supabase is included as an additional backend and realtime service. The application's clean architecture allows data sources to be changed or replaced without requiring major changes to the presentation layer.

1.5.8 Google Gemini

Google Gemini is integrated into the meal-analysis feature. It processes meal-related image information and provides nutritional and glucose-related insights.

The application accesses Gemini through the `google_generative_ai` integration.

1.5.9 Python

Python is used for developing the machine-learning portion of the system. It is used for:

- Data preprocessing
- Feature engineering
- Model training
- Model evaluation
- Explainability analysis
- Model export
- Prediction API development

1.5.10 Scikit-learn

Scikit-learn is used for preprocessing and machine-learning-related operations such as train-test splitting, StandardScaler preprocessing, Logistic Regression, Random Forest, and evaluation metrics.

1.5.11 XGBoost

XGBoost is used for the final diabetes-risk classification model.

After comparing Logistic Regression, Random Forest, and XGBoost, XGBoost was selected as the final model based on weighted F1-score and overall classification performance.

The trained model is exported as:

XGBoost_model.pkl

1.5.12 StandardScaler

A StandardScaler is fitted using training data and then applied consistently to testing data and API prediction inputs.

The fitted scaler is exported as:

scaler.pkl

This ensures that new prediction inputs are processed in the same way as the data used during model development.

1.5.13 Flask

Flask is used to expose the trained machine-learning model through an API.

The API provides two main endpoints:

- GET /health
- POST /predict

The /predict endpoint receives the twelve required input features and returns the predicted risk category, confidence, class probabilities, and advice.

The mobile application therefore does not need to execute the XGBoost model directly. Instead, it sends data to the Flask service and receives the prediction response.

1.5.14 SHAP

SHAP is included to provide global model explainability. It can be used to determine which input features have the strongest overall influence on model predictions.

This is particularly useful because XGBoost is more difficult to interpret directly than simpler linear models.

1.5.15 LIME

LIME is used for local explainability.

Where SHAP can help explain general model behavior, LIME is useful for explaining a particular prediction by showing how individual features influenced the output for one patient.

1.5.16 Dio

Dio is used for HTTP networking in the Flutter application. It supports communication between the mobile application and remote services, including API-based functionality.

1.5.17 Hive

Hive is used for lightweight local storage. In the current application, it is particularly associated with notification history and offline-aware functionality.

1.5.18 SharedPreferences and Secure Storage

SharedPreferences is used for lightweight local preferences and session-related information, while secure storage is used where sensitive local values such as tokens need stronger protection.

1.5.19 Flutter Local Notifications

The `flutter_local_notifications` package is used to provide scheduled device notifications.

Together with timezone-related packages, the application can schedule reminders for specific activities and times. The reminder implementation also handles notification permissions, scheduling, cancellation, history, and deep-link behavior.

1.5.20 Development and Testing Tools

The project also uses supporting tools and packages for development and testing, including:

- Android Studio
- Flutter development tools
- Jupyter Notebook for machine-learning experimentation
- `flutter_test`
- `bloc_test`
- `mocktail`

The Flutter testing packages are included for user-interface, state-management, and application-logic testing.

1.6 Chapter Summary

This chapter introduced glucosense_ai and explained the main problem addressed by the project. The system is designed to combine glucose monitoring, AI-assisted meal analysis, diabetes-risk prediction, trends, reports, and reminders in a single mobile application.

The chapter also described the main objectives and scope of the project. The application uses Flutter for mobile development, Firebase and Supabase for backend-related services, Google Gemini for meal analysis, and an XGBoost machine-learning model served through Flask for diabetes-risk prediction.

The next chapter will review **existing diabetes-monitoring applications, diabetes-risk prediction research, machine-learning approaches, AI-based dietary analysis, and related systems**, and will identify the gap that glucosense_ai is intended to address.

CHAPTER 02

LITERATURE REVIEW

2.0 Introduction

Diabetes management has changed considerably with the growth of mobile health applications, cloud-based systems, artificial intelligence, and machine learning. In the past, most diabetes-related systems mainly focused on storing blood glucose readings or maintaining patient records. More recent systems try to go beyond simple record keeping by providing trends, reminders, nutritional support, personalized feedback, and prediction of future health risks.

The need for such systems is significant. According to the World Health Organization, the number of people living with diabetes increased from around 200 million in 1990 to approximately 830 million in 2022. Diabetes can lead to serious complications involving the heart, kidneys, eyes, nerves, and blood vessels when it is not properly managed. [1] ([World Health Organization](#))

Early identification of people who may be at risk is therefore important. The American Diabetes Association identifies HbA1c, fasting plasma glucose, and glucose tolerance measurements as important indicators for diabetes and prediabetes. For example, an HbA1c value between 5.7% and 6.4% is considered within the prediabetes range, while an HbA1c of 6.5% or above is one diagnostic criterion for diabetes. [2] ([Diabetes Journals](#))

This chapter reviews existing diabetes-management applications and research related to mobile self-management, glucose monitoring, diabetes-risk prediction, XGBoost, explainable artificial intelligence, image-based meal analysis, and reminder systems. The purpose of the review is not only to examine what existing solutions provide but also to identify the limitations that motivated the development of **glucosense_ai**.

2.1 Mobile Technology for Diabetes Self-Management

Managing diabetes is not limited to checking glucose once in a while. People may need to monitor blood glucose regularly, maintain a suitable diet, stay physically active, follow prescribed medication, and observe changes in their health over time. Carrying out all these activities consistently can be difficult.

Mobile applications have therefore become useful tools for diabetes self-management. They allow users to keep health-related information on a device that is already part of their daily routine. Common features include glucose logging, medication records, dietary tracking, physical-activity monitoring, reminders, charts, and reports.

Corsica et al. reviewed diabetes self-management applications and found that mobile applications have the potential to improve adherence to diabetes-related behaviors. Their review also reported evidence that a number of diabetes applications can contribute to improvements in HbA1c, although the authors noted that stronger and more consistent research is still required to determine which application features provide the greatest benefit. [3] ([PubMed](#))

A separate review by Lee and Kim examined diabetes-management applications from the perspective of clinical effectiveness and usability. The authors found that mobile applications could provide convenient access to diabetes-management features and that several reviewed interventions were associated with improved blood glucose control. They also emphasized that usability needs to be evaluated carefully when developing diabetes-related applications. [4] ([PubMed](#))

These findings are relevant to glucosense_ai because the proposed application does not depend on a single feature. Instead, it attempts to combine glucose records, medical-profile information, food-related information, reminders, trends, and predictive support in a single system.

2.2 Diabetes Prevention Through Smartphone Applications

Mobile applications are increasingly being studied not only for people already diagnosed with diabetes but also for individuals considered at risk.

Jahan et al. conducted a systematic review and meta-analysis of smartphone applications intended to support prevention of type 2 diabetes. Twenty-four studies involving 2,378 participants were included. Their analysis showed promising improvements in factors such as body weight and BMI, although evidence for direct effects on diabetes incidence remained limited. The authors also highlighted the importance of personalization in future diabetes-prevention applications. [5] ([PubMed](#))

This is particularly important for systems such as glucosense_ai. A person does not necessarily need only a label stating whether they currently have diabetes. It may also be helpful to identify whether their current health indicators place them in an intermediate or higher-risk group. This idea supports the use of the three categories adopted in this project:

- Normal
- Prediabetic
- Diabetic

However, prediction through a mobile application should be considered supportive information rather than a replacement for laboratory testing and professional medical diagnosis.

2.3 Existing Diabetes Applications

Several mobile applications already provide diabetes-management functions. Studying these applications helps identify both useful features and areas where further integration may be possible.

2.3.1 mySugr

mySugr is a widely used diabetes-management application that works primarily as a digital diabetes logbook. It allows users to record information including blood glucose, meals, insulin, activity, and other therapy-related information.

The application also supports connections with compatible glucose meters, allowing readings from supported devices to be transferred automatically. Users can view statistics over different periods and generate reports that can be shared with healthcare professionals. Meal photographs and blood-glucose reminders are also supported in some configurations. [6] ([mySugr](#))

One useful feature of mySugr is its ability to provide a clear historical view of glucose information. Users can review their data over periods such as 7, 14, 30, and 90 days. Reports can also be produced in different formats depending on the available service level. ([mySugr](#))

The reminder function can be used, for example, to remind a user to perform a post-meal blood glucose measurement. ([mySugr](#))

Although mySugr provides strong glucose-management and reporting features, the focus is mainly on diabetes logging and management. `glucosense_ai` extends this general concept by combining glucose management with a separate machine-learning diabetes-risk classifier and AI-based interpretation of meal photographs.

2.3.2 Glucose Buddy

Glucose Buddy is another diabetes-management application that brings several kinds of health information into one interface. It allows users to record blood glucose, food, medication, blood pressure, and other health-related data. [7] ([Glucose Buddy](#))

The application provides charts for understanding changes in glucose over time, estimated HbA1c information, reminders, meal-related features, and integrations with supported

health devices and services. Its Meal IQ feature is intended to help users understand how food and activities relate to changes in blood glucose. ([Glucose Buddy](#))

Glucose Buddy demonstrates the value of combining multiple aspects of diabetes management rather than maintaining glucose records alone. However, glucosense_ai has a somewhat different goal. In addition to glucose logging and trends, it incorporates a custom Flask-based machine-learning API that returns one of three diabetes-risk categories together with prediction confidence and probabilities.

2.4 Limitations Observed in Existing Diabetes Applications

Existing diabetes applications provide many useful functions, but previous research has identified several recurring limitations.

A review of diabetes self-management applications reported issues such as limited personalized feedback, difficulties with data entry, and weak integration between users, healthcare providers, and other information systems. [8] ([PubMed](#))

Modern reviews also suggest that although digital interventions can support diabetes management, not every application has the same clinical evidence, and the effectiveness of specific features remains difficult to compare. ([PubMed](#))

For the present project, four gaps are particularly relevant:

First, many diabetes applications are strong at recording data but do not use a dedicated machine-learning model to classify the user's diabetes-risk category.

Second, meal logging often requires users to enter food information manually. Photograph-based analysis can reduce some of this effort.

Third, prediction systems developed in research environments are not always integrated into complete mobile applications that include monitoring, reminders, trends, meals, and user profiles.

Fourth, highly accurate machine-learning models can be difficult for users to understand when no explanation of their predictions is provided.

These limitations provided important direction for glucosense_ai.

2.5 Machine Learning for Diabetes Risk Prediction

Machine learning has increasingly been applied to health datasets for identifying patterns associated with diabetes.

Unlike a traditional fixed decision rule, a machine-learning classifier learns relationships from training examples. Once trained, it can process new feature values and assign a predicted class or probability.

Common machine-learning approaches applied to diabetes prediction include:

- Logistic Regression
- Decision Trees
- Random Forest
- Support Vector Machines
- Artificial Neural Networks
- Gradient Boosting
- XGBoost

Different models have different advantages. Logistic Regression is relatively straightforward and provides a useful baseline. Random Forest combines multiple decision trees and can capture nonlinear relationships. Boosting methods such as XGBoost construct models sequentially, allowing later trees to focus on errors made by previous ones.

The glucosense_ai model-development process follows a similar comparison strategy by evaluating **Logistic Regression, Random Forest, and XGBoost** rather than selecting an algorithm without comparison.

2.6 Logistic Regression for Diabetes Prediction

Logistic Regression is one of the commonly used baseline methods for health-related classification problems. It estimates the probability of an outcome using relationships between input variables and a target class.

Its main advantage is simplicity. Compared with complex ensemble models, Logistic Regression is easier to interpret and generally requires less computational power.

However, relationships between diabetes-related characteristics are not always purely linear. Interaction between factors such as age, BMI, glucose measurements, hypertension, family history, and lifestyle may be complex. For this reason, Logistic Regression can provide a useful baseline but may not always produce the strongest classification performance.

Within the glucosense_ai training process, Logistic Regression serves as one of the benchmark models against which more advanced models can be evaluated.

2.7 Random Forest for Diabetes Prediction

Random Forest is an ensemble-learning method that builds several decision trees and combines their predictions.

Instead of relying on one decision tree, the algorithm uses multiple trees trained on different samples and feature combinations. This generally makes it more stable than a single decision tree and allows nonlinear relationships between variables to be captured.

Random Forest has been widely used in medical classification because it can work with a variety of feature types and can provide measures of feature importance.

The glucosense_ai model training therefore includes Random Forest as another benchmark before selection of the final model.

2.8 XGBoost for Diabetes Prediction

Extreme Gradient Boosting, commonly known as **XGBoost**, has become a popular machine-learning algorithm for structured or tabular datasets.

Unlike Random Forest, where trees are largely trained independently, boosting creates trees sequentially. Each new learner attempts to improve errors remaining from earlier learners. XGBoost also includes regularization and optimization mechanisms intended to improve generalization and computational efficiency.

Recent diabetes-prediction research demonstrates why XGBoost is a reasonable candidate for such systems.

Li, Peng, and Peng developed a diabetes-prediction approach based on an optimized XGBoost model and stacking ensemble. Their work used behavioral risk data and compared machine-learning techniques for early diabetes-risk identification. SHAP analysis was also used to understand the influence of individual predictors. [9] ([PubMed](#))

Another large-scale study using UK Biobank data used XGBoost to predict the development of type 2 diabetes over a ten-year period. HbA1c was identified as the strongest predictor, followed by factors including BMI, waist circumference, blood glucose, family history, and age. The XGBoost model achieved an AUC of approximately 0.90 in that study. [10] ([PubMed Central \(PMC\)](#))

A more recent study using Canadian primary-care data compared seven machine-learning classification models and found that XGBoost provided the strongest performance, reporting an AUC of 92%. HbA1c was among the important predictive variables. [11] ([PubMed](#))

These studies do not mean that XGBoost will always be the best model for every diabetes dataset. Performance depends on data quality, selected variables, class balance, preprocessing, and evaluation procedure. They do, however, support the decision to include XGBoost as a serious candidate and compare it with other classifiers.

For glucosense_ai, XGBoost was ultimately selected after comparison with Logistic Regression and Random Forest using weighted F1-score and overall classification behavior.

2.9 Prediabetes Risk Prediction

Prediabetes is particularly important because it represents an intermediate state in which glucose levels are above normal but do not yet meet the diagnostic criteria for diabetes.

The ADA identifies an HbA1c range of **5.7–6.4%** as one criterion for prediabetes and fasting plasma glucose between **100 and 125 mg/dL** as another. [2] ([Diabetes Journals](#))

Machine learning has also been studied specifically for identifying prediabetes risk.

A recent study using China Health and Nutrition Survey data compared Logistic Regression, Random Forest, Support Vector Machine, Decision Tree, Artificial Neural Network, Naive Bayes, and XGBoost. XGBoost achieved the strongest reported discrimination, with an AUC of 0.939. The researchers also used SHAP to identify important factors associated with prediabetes risk. [12] ([PubMed Central \(PMC\)](#))

This is relevant to glucosense_ai because the project does not use only the two categories of diabetic and non-diabetic. Instead, it introduces **Prediabetic** as an intermediate risk category.

One important distinction must still be maintained. In glucosense_ai, the original dataset contains a binary diabetes label, while the three-class target is derived during preprocessing. Therefore, the prediabetes class in this project should be described as a **derived risk category** rather than as an independently diagnosed clinical label.

2.10 Importance of Diabetes-Related Features

Diabetes prediction does not depend on only one characteristic. Several demographic, medical, behavioral, and biochemical variables can contribute to risk.

For example, ADA screening guidance considers factors such as family history, cardiovascular disease, hypertension, obesity, and physical inactivity when determining whether an individual should be assessed for prediabetes or diabetes. ([Diabetes Journals](#))

Studies using machine learning have similarly identified variables such as:

- Age
- BMI
- HbA1c
- Blood glucose
- Hypertension
- Family history
- Physical activity
- Other metabolic measurements

as useful predictive factors. ([PubMed Central \(PMC\)](#))

The glucosense_ai model combines twelve features:

1. Gender
2. Age
3. Hypertension
4. Heart disease
5. Smoking history
6. Activity level
7. Family history
8. BMI
9. HbA1c level
10. Fasting glucose

11. Pre-meal glucose

12. Post-meal glucose

Using a mixture of clinical and behavioral information allows the model to consider several aspects of the user's profile instead of relying on a single measurement.

2.11 Explainable Artificial Intelligence in Diabetes Prediction

Prediction accuracy is important, but in healthcare-related applications another question becomes equally important:

Why did the model make that prediction?

Models such as XGBoost may produce strong predictions but are harder to interpret directly than simple statistical models.

Explainable Artificial Intelligence, commonly called XAI, attempts to make machine-learning decisions more understandable.

Two widely used techniques are:

- SHAP
- LIME

A recent diabetes-risk study comparing several machine-learning models used both SHAP and LIME to interpret predictions. The authors found that explainability methods could highlight how variables such as age, BMI, blood pressure, and other health factors influenced model outputs. [13] ([PubMed Central \(PMC\)](#))

For glucosense_ai, explainability is included for the same reason. A prediction such as **Prediabetic** is more useful academically and potentially more understandable when the system can also examine which variables had the greatest influence on the model.

2.11.1 SHAP

SHAP stands for **SHapley Additive exPlanations**.

It is based on concepts from cooperative game theory and assigns each feature a contribution toward a model's prediction.

SHAP can be used at two useful levels.

Global explanation shows which variables are generally most important across many predictions.

Local explanation shows how particular features contributed to an individual prediction.

Studies using XGBoost for diabetes and prediabetes have used SHAP successfully to rank risk factors and understand their contribution to predictions. ([PubMed](#))

The glucosense_ai notebook includes SHAP summary analysis to examine overall feature influence.

2.11.2 LIME

LIME stands for **Local Interpretable Model-Agnostic Explanations**.

Instead of trying to explain the entire machine-learning model, LIME concentrates on a particular prediction. It creates an interpretable approximation around the selected case and estimates which variables contributed positively or negatively to the prediction.

This makes LIME useful when the purpose is to answer a question such as:

Why was this particular patient classified as Prediabetic?

The glucosense_ai model-development notebook includes LIME for this form of individual-level interpretation.

Using both SHAP and LIME strengthens the explainability aspect of the proposed system because they provide complementary ways of understanding model behavior.

2.12 Image-Based Food Recognition and Dietary Analysis

Diet plays an important role in diabetes management, but maintaining detailed food records manually can become tiring.

Traditional food diaries generally require a person to enter the food name, quantity, calories, or other nutritional information. This process takes time and also depends heavily on the user's knowledge and estimation ability.

Image-based food analysis provides an alternative. A user takes a photograph of a meal, and an artificial intelligence system attempts to identify the food and estimate nutritional information.

A systematic review of image-based food-recognition systems examined 78 relevant studies and found that deep-learning methods, particularly convolutional neural networks, had become common in food-recognition research. The review described several typical stages including food segmentation, classification, and estimation of calories or nutrients. [14] ([PubMed Central \(PMC\)](#))

Another systematic review comparing AI dietary-analysis methods with human or ground-truth estimates found considerable progress in food-image analysis. At the same time, results varied between datasets and food types, and the authors concluded that further development is still required before such methods can be relied upon as stand-alone clinical dietary-assessment tools. [15] ([PubMed Central \(PMC\)](#))

These findings support the use of image analysis as a convenient **assistance feature**, while also showing why generated nutritional information should be presented as an estimate rather than an exact clinical measurement.

2.13 AI-Assisted Meal Analysis in glucosense_ai

The meal-analysis module of glucosense_ai follows the general direction identified in image-based dietary research but uses a multimodal generative AI service rather than training a separate food-classification model.

Users can capture or select a meal photograph and submit it for analysis. Google Gemini is used to interpret the image and return information such as estimated nutritional composition, glycaemic-related insights, and suggestions for alternatives.

The application also allows analysed meals to be stored and reopened later.

This feature provides an important connection between diet and glucose management. Instead of requiring the user to maintain an entirely separate food diary, meal analysis becomes part of the same application in which glucose readings and diabetes risk are managed.

However, because image-based nutrition estimation itself has known uncertainty, these results should be treated as guidance rather than laboratory-quality dietary measurements. This is consistent with findings from current dietary-image analysis research. ([PubMed Central \(PMC\)](#))

2.14 Glucose Trends and Reporting

Individual glucose measurements are useful, but trends across several days or weeks can provide a clearer picture of how glucose levels are changing.

Diabetes-management applications therefore commonly provide charts, summaries, and reports.

For example, mySugr allows users to view diabetes statistics across different time ranges and create reports for sharing. ([mySugr](#)) Glucose Buddy similarly provides blood-glucose graphs and trend-related tools. ([Glucose Buddy](#))

glucosense_ai follows the same useful principle by allowing users to review trends over:

- 7 days
- 30 days
- 90 days

The application includes measures such as average glucose, range, variation, time-in-range-related information, and time-of-day patterns.

Reports can also summarize readings and meals over a selected date period.

The purpose of this feature is to convert a collection of individual readings into information that is easier for the user to interpret.

2.15 Reminders and User Adherence

A diabetes-management system is useful only when the user continues entering relevant information.

Users may forget glucose measurements, especially those that need to be performed around meal times. Reminder systems can therefore support regular self-monitoring.

mySugr, for example, provides reminders that can be configured after an entry to remind a user of a later glucose measurement. ([mySugr](#)) Glucose Buddy also includes reminders for glucose checks and medication. ([Glucose Buddy](#))

Research into diabetes self-management applications has also shown that digital behavior-change interventions can support self-management, although the effectiveness varies according to the features and intervention design. ([PubMed](#))

The reminder system in glucosense_ai includes:

- Fasting reminders
- Breakfast reminders

- Lunch reminders
- Dinner reminders
- Snack reminders
- Notification history
- Deep linking to the relevant screen
- Smart suppression

Smart suppression is particularly useful. If the user has already recorded the relevant reading around the reminder period, the system can avoid sending an unnecessary reminder during the next scheduling process.

This makes the reminder module more context-aware than a basic fixed alarm.

2.16 Cloud-Based Data Management

Modern mobile applications often separate the user interface from persistent data storage.

glucosense_ai uses Firebase services for authentication, Firestore database operations, and storage. Supabase is also available for realtime or additional backend functionality.

Using cloud-based storage provides several practical advantages. Information can remain available even when the user changes devices, different features can access a consistent data source, and application logic does not have to rely entirely on local device memory.

At the same time, health-related applications need to treat security carefully because medical and personal data are sensitive. Authentication, access controls, secure communication, and careful storage design are therefore important parts of the overall architecture.

2.17 Offline-Aware Mobile Design

A mobile healthcare application should not assume that the internet connection is always reliable.

glucosense_ai includes local technologies such as Hive and SharedPreferences alongside cloud services. The feature-first clean architecture also separates data sources from the rest of the application.

This arrangement allows the application to support local information and offline-aware behavior where appropriate instead of completely failing when a remote service is temporarily unavailable.

This is especially useful for functions such as notification history and local reminders, which should not depend entirely on a constant server connection.

2.18 Comparison of Existing Approaches

The major differences between existing approaches and the proposed glucosense_ai system can be summarized as follows.

Feature	mySugr	Glucose Buddy	Research ML Systems	glucosense_ai
User authentication	Yes	Yes	Usually No	Yes
Glucose logging	Yes	Yes	Usually No	Yes
Fasting/pre/post-meal logging	Supported through logging	Supported	Dataset dependent	Yes
Glucose trends	Yes	Yes	Usually No	Yes
Reports	Yes	Yes	Usually No	Yes
Reminders	Yes	Yes	Usually No	Yes
Meal photo support	Yes	Yes/food recognition features	Usually No	Yes
AI meal interpretation	Limited/different purpose	Meal-related intelligence	Usually No	Yes

Feature	mySugr	Glucose Buddy	Research ML Systems	glucosense_ai
Diabetes-risk ML model	Not main function	Not main function	Yes	Yes
Three risk categories	No	No	Study dependent	Yes
XGBoost model	No	No	Frequently studied	Yes
Prediction confidence/probabilities	No	No	Often research only	Yes
SHAP explainability	No	No	Some studies	Yes
LIME explainability	No	No	Some studies	Yes
Flask prediction API	No	No	Sometimes	Yes
Cloud synchronization	Yes	Yes	Usually not app-focused	Yes
Local reminder system	Yes	Yes	Usually No	Yes
Notification history	Not central	Not central	No	Yes
Integrated monitoring + ML + meal AI	Partial	Partial	No	Yes

The comparison shows that no single reviewed approach exactly matches the combination used in glucosense_ai.

Commercial applications are generally stronger in everyday diabetes tracking and user experience, while academic machine-learning systems are stronger in predictive modeling and evaluation. The proposed project attempts to connect these two areas.

2.19 Research Gap

After reviewing diabetes-management applications and recent machine-learning studies, a clear gap can be identified.

Many mobile diabetes applications already provide good **monitoring functionality**. Users can record glucose values, view graphs, configure reminders, and generate reports. However, these applications do not necessarily include a custom, transparent three-class machine-learning pipeline integrated with the application.

On the other side, many research studies build highly capable **diabetes-prediction models** using algorithms such as Random Forest, Gradient Boosting, and XGBoost. These studies may achieve strong evaluation results and may even include explainability methods such as SHAP. However, the research commonly ends at model evaluation and does not provide a complete daily-use mobile system containing glucose logging, meal analysis, trends, reminders, notification history, and user-profile management.

Image-based dietary research also shows that AI can reduce the effort required for traditional manual dietary recording, but dietary-image analysis and diabetes-risk prediction are frequently studied as separate problems. ([PubMed Central \(PMC\)](#))

Therefore, the primary gap addressed by glucosense_ai is **integration**.

The proposed system brings together:

Glucose Monitoring + Medical Profile + Meal Image Analysis + Trends + Reports + Smart Reminders + Machine-Learning Risk Prediction + Explainability

within one mobile application.

A second gap is interpretability. The project does not rely only on a prediction output. SHAP and LIME are included in the machine-learning work so that model decisions can be investigated rather than treating XGBoost as a completely unexplained black box.

A third gap is deployment. Instead of keeping the trained model only inside a Jupyter Notebook, the model is exported and exposed through a Flask API, allowing the mobile application to communicate with the prediction model.

The overall system therefore aims to connect research-oriented machine learning with practical mobile diabetes-management functionality.

2.20 Limitations Identified from the Literature

The literature also highlights several issues that should be kept in mind while developing the proposed system.

First, machine-learning results depend heavily on the training population. A model that performs strongly on one dataset may not achieve the same performance on another population.

Second, prediction accuracy alone is insufficient for healthcare-related applications. Class-specific recall, precision, F1-score, calibration, and external validation are also important.

Third, AI-based food analysis can provide useful estimates, but food quantity, ingredients, cooking method, and image quality can make accurate nutritional estimation difficult. ([PubMed Central \(PMC\)](#))

Fourth, mobile applications depend on consistent user engagement. A technically strong system will provide little value if recording information becomes too complicated or users stop using it. Previous reviews have therefore emphasized usability and personalization. ([PubMed](#))

Finally, diabetes-risk predictions generated by software should not be presented as confirmed medical diagnoses. Current ADA guidance relies on recognized clinical testing and, in many situations, confirmatory testing for diagnosis. ([Diabetes Journals](#))

These limitations will be considered when describing the design, implementation, testing, and future improvements of glucosense_ai.

2.21 Chapter Summary

This chapter reviewed existing work related to diabetes management and the technologies used in `glucosense_ai`.

Mobile diabetes applications such as `mySugr` and `Glucose Buddy` demonstrate that glucose logging, reports, reminders, dietary tracking, and trend analysis can make diabetes-related information easier to manage. Research studies also show growing interest in mobile applications for diabetes self-management and prevention.

The literature on machine learning shows that algorithms such as Logistic Regression, Random Forest, and XGBoost can be used to estimate diabetes risk from patient information. XGBoost has demonstrated strong performance in several recent diabetes- and prediabetes-related studies, making it a reasonable model to evaluate for the proposed project. ([PubMed](#))

Explainability methods such as SHAP and LIME can improve understanding of model predictions, while advances in image-based dietary assessment demonstrate the potential of AI-assisted food analysis. At the same time, the literature shows that prediction systems, food-analysis systems, and everyday diabetes-management applications are commonly treated separately.

`glucosense_ai` attempts to address this gap through an integrated mobile system containing glucose tracking, meal analysis, risk prediction, explainability, trends, reporting, and smart reminders.

The next chapter presents the **System Design and Implementation of `glucosense_ai`**, including the system architecture, application architecture, use case diagram, class diagram, sequence diagrams, machine-learning pipeline, model-training workflow, Flask prediction API, and implementation of the main application modules.

References Used in Chapter 2

- [1] World Health Organization, *Diabetes Fact Sheet*, 2024. ([World Health Organization](#))
- [2] American Diabetes Association, *Diagnosis and Classification of Diabetes: Standards of Care in Diabetes—2026*, Diabetes Care, 2026. ([Diabetes Journals](#))
- [3] J. A. Corsica et al., “Mobile apps for diabetes self-management: An updated review of app features and effectiveness,” *Journal of Behavioral Medicine*, 2025. ([PubMed](#))
- [4] J. L. Lee and Y. Kim, “Evaluation of Mobile Applications for Patients with Diabetes Mellitus: A Scoping Review,” *Healthcare*, 2024. ([PubMed](#))
- [5] E. Jahan et al., “Smartphone Applications to Prevent Type 2 Diabetes: A Systematic Review and Meta-Analysis,” *American Journal of Preventive Medicine*, 2024. ([PubMed](#))
- [6] mySugr, *Diabetes Management Application – Features and Functionality*. ([mySugr](#))
- [7] Glucose Buddy, *Glucose Buddy Tracking and Application Features*. ([Glucose Buddy](#))
- [8] Research review, “Mobile applications for diabetes self-management: status and potential.” ([PubMed](#))
- [9] W. Li, Y. Peng, and K. Peng, “Diabetes prediction model based on GA-XGBoost and stacking ensemble algorithm,” *PLOS ONE*, 2024. ([PubMed](#))
- [10] “Identifying top ten predictors of type 2 diabetes through machine learning analysis of UK Biobank data,” 2024. ([PubMed Central \(PMC\)](#))
- [11] “Predicting Diabetes in Canadian Adults Using Machine Learning,” 2025. ([PubMed](#))
- [12] “Interpretable machine learning method to predict the risk of pre-diabetes using a national-wide cross-sectional data,” 2025. ([PubMed Central \(PMC\)](#))

[13] “Is explainability the missing link in artificial intelligence-based diabetes prediction?” 2025. ([PubMed Central \(PMC\)](#))

[14] “Applying Image-Based Food-Recognition Systems on Dietary Assessment: A Systematic Review,” 2022. ([PubMed Central \(PMC\)](#))

[15] “AI-based digital image dietary assessment methods compared to humans and ground truth: a systematic review,” 2024. ([PubMed Central \(PMC\)](#))

CHAPTER 03

METHODOLOGY / PROPOSED SYSTEM DESIGN

3.0 Proposed System

The proposed system, **glucosense_ai**, is a smart mobile-based diabetes management and risk prediction system developed to assist users in monitoring their blood glucose levels, analysing meals, identifying diabetes risk, observing glucose trends, and maintaining regular monitoring routines. The application combines mobile technology, cloud services, artificial intelligence, and machine learning to provide an integrated platform for diabetes-related monitoring and decision support.

The mobile application is developed using **Flutter and Dart** and follows a feature-first clean architecture. The application uses BLoC/Cubit for state management and separates the data, domain, and presentation layers to improve maintainability and testing. According to the implemented architecture, the user interface does not communicate directly with external services; instead, requests move through the application's state-management, repository, and data-source layers.

The system provides authentication using Firebase Authentication and maintains user-related information through cloud services. The user can create a personal and medical profile containing information required by the diabetes-risk prediction module. The application also enables users to record fasting, pre-meal, and post-meal blood-glucose readings together with relevant timestamps and meal information.

One of the major features of the proposed system is the **Meal Predictor**. A user can capture or select an image of a meal and submit it for analysis. The application integrates the **Google Gemini API** to analyse the meal and provide information such as estimated nutritional composition, glycaemic impact, and more suitable food alternatives. Analysed meals can also be stored and accessed again when required.

Another important component is the **Diabetes Risk Prediction module**. The Flutter application communicates with a Flask-based prediction API that serves the trained machine-learning model. The model uses twelve input features consisting of demographic, clinical, behavioural, and glucose-related information. The input features are preprocessed using a saved **StandardScaler**, after which the final **XGBoost classifier** predicts one of three diabetes-risk categories:

Normal, Prediabetic, or Diabetic.

The API returns the predicted risk category together with prediction confidence, class probabilities, and short guidance text. The machine-learning development process also incorporates **SHAP** for global feature interpretation and **LIME** for individual prediction explanation.

The proposed system additionally provides glucose trends, date-based reports, saved meal information, reminders, notifications, and profile management. The application supports glucose analysis over periods such as 7, 30, and 90 days and provides measurements including averages, ranges, and time-of-day patterns.

The reminder system operates through local device services and allows users to configure reminders for fasting measurements and meal-related glucose checks. It uses scheduled operating-system notifications and includes smart suppression so that a reminder can be skipped when the required reading has already been recorded near the scheduled time.

Overall, the proposed system integrates glucose monitoring, artificial-intelligence-based meal analysis, machine-learning-based diabetes-risk classification, reporting, and reminders into a single mobile platform.

3.1 Functional Requirements

Functional requirements describe the functions and operations that the proposed system must provide to its users.

3.1.1 Authentication and Onboarding Module

The authentication and onboarding module manages initial user access to the application.

The system shall allow a new user to create an account using an email address and password. Existing users shall be able to log in using registered credentials. The system shall authenticate the credentials through Firebase Authentication before allowing access to protected application features.

The system shall provide a password-reset mechanism for users who forget their passwords. New users shall also be provided with introductory onboarding screens explaining the major functionality of the application.

After successful authentication, the system shall determine whether the user's medical profile has been completed. Users with an incomplete profile shall be directed to the profile setup process before using risk-prediction functionality.

The implemented application contains login, signup, forgot-password, onboarding, and profile-setup flows backed by Firebase Authentication.

3.1.2 Profile Management Module

The profile module shall allow users to enter, view, and update their personal and medical information.

The profile shall contain information required for diabetes-risk prediction, including demographic and health-related characteristics. Relevant fields used by the prediction model include age, gender, hypertension status, heart-disease status, smoking history, activity level, family history, BMI, and HbA1c level.

The system shall store profile information so that it can be reused when risk predictions are generated. Users shall be able to modify their profile when their health or personal information changes.

The application's profile functionality is designed to synchronize personal and medical preferences through its backend services.

3.1.3 Glucose Logging Module

The glucose logging module shall allow users to record their blood-glucose measurements.

The system shall support different glucose-reading types, including:

Fasting glucose, pre-meal glucose, and post-meal glucose.

Each reading shall contain the glucose value, relevant reading type, meal information where applicable, measurement unit, and timestamp.

The system shall save recorded glucose readings and make them available for history, trends, reports, and diabetes-risk prediction. The application shall also allow a reading screen to be pre-filled when it is opened through a reminder notification.

3.1.4 Meal Prediction Module

The Meal Predictor shall allow users to capture a new meal image or select an existing image from the device.

The selected meal image shall be processed through the integrated AI service. The system shall send the image and appropriate prompt information to the Google Gemini API and receive an analysis of the meal.

The returned information may include nutritional information, estimated macronutrients, glycaemic impact, and suggestions for more diabetes-friendly alternatives.

The user shall be able to save analysed meals for later access and reuse.

3.1.5 Diabetes Risk Prediction Module

The Risk Prediction module shall classify a user into one of three risk categories:

Normal, Prediabetic, or Diabetic.

The prediction shall be generated using the following twelve input features:

No.	Prediction Feature
1	Gender
2	Age
3	Hypertension
4	Heart Disease
5	Smoking History
6	Activity Level
7	Family History
8	BMI
9	HbA1c Level
10	Fasting Glucose
11	Pre-Meal Glucose
12	Post-Meal Glucose

The application shall collect the required information from the user's medical profile and glucose information and submit the values to the Flask Prediction API.

The API shall preprocess the input using the saved StandardScaler and pass the transformed values to the trained XGBoost model.

The prediction response shall contain the risk category, prediction confidence, individual class probabilities, and appropriate guidance text.

The application shall display the prediction result in a form that can be easily understood by the user.

3.1.6 Trends and Reports Module

The system shall allow users to analyse their historical glucose information.

The Trends module shall provide charts and statistical summaries for selected periods, including **7 days, 30 days, and 90 days**. The available analysis can include average glucose levels, glucose ranges, Time in Range (TIR), variance, and time-of-day patterns.

The Reports module shall enable users to select a date range and generate a summary containing glucose readings and meal-related information.

The generated report shall be designed so that it can be shared when required.

3.1.7 Reminder and Notification Module

The system shall allow users to configure reminders related to their glucose-monitoring routine.

Users shall be able to configure reminders for fasting readings, breakfast, lunch, dinner, snacks, and other supported reading periods.

The reminder scheduler shall calculate the next appropriate notification time and schedule the notification through the mobile operating system.

When the user taps an applicable reminder, the system shall open the glucose logging screen and pre-fill the appropriate reading type and meal information.

The system shall also maintain a local notification history.

Smart suppression shall prevent unnecessary reminders when the user has already completed the associated glucose-reading activity within the configured time window.

3.1.8 Data Storage and Synchronization Module

The system shall store user and application information using the configured cloud and local storage services.

Firebase services are used for authentication, database operations, and storage, while Supabase is also present as an additional backend/realtime service. Local information such as notification history and session-related information can be maintained using Hive, SharedPreferences, and secure storage.

The system shall maintain synchronization between locally required application information and cloud-based information whenever network connectivity is available.

3.2 Non-Functional Requirements

Non-functional requirements define the quality, reliability, security, usability, and technical characteristics expected from glucosense_ai.

3.2.1 Performance

The application should provide responsive interaction during normal usage.

Operations such as navigation, loading profile information, saving glucose readings, and displaying stored records should be performed without unnecessary delays.

Requests sent to external services such as the Flask Prediction API and Google Gemini API should provide appropriate loading states while the result is being processed.

The application should prevent the user interface from becoming unresponsive during asynchronous operations.

3.2.2 Security

Security is particularly important because the application handles personal and health-related information.

User authentication shall be handled through Firebase Authentication. Unauthorized users shall not be able to access another user's information.

Sensitive configuration information such as API keys shall not be stored directly in publicly accessible source code. The application documentation already supports environment-based configuration for the Gemini API key.

Communication with cloud services and prediction APIs should use secure network communication when deployed.

Appropriate authorization and database-security rules should be applied to cloud-stored data.

3.2.3 Usability

The mobile application shall provide simple and understandable navigation.

The user should be able to access the major functions, including Home, Glucose Logging, Meal Prediction, Risk Prediction, and Profile, through the application's main navigation structure. The implemented application uses a five-tab home shell covering these major functions.

Medical and prediction information should be presented in clear language so that users can understand the system output without requiring technical knowledge of machine learning.

3.2.4 Reliability

The application should maintain reliable data storage and retrieval.

A failure in one external service should be handled gracefully rather than causing the complete application to terminate.

API failures, invalid responses, network failures, and data-loading errors should produce an appropriate application state or error message.

Reminder scheduling should remain functional where supported by the operating system, and pending notifications should be restored after device restart where applicable. The application includes a boot receiver for re-scheduling pending notifications on Android.

3.2.5 Maintainability

The application should be structured so that future developers can modify, test, or extend individual features without significantly affecting unrelated components.

glucosense_ai addresses this requirement through feature-first clean architecture, where each feature is separated into data, domain, and presentation layers.

Repository interfaces and data sources should remain sufficiently independent so that an implementation can be replaced without requiring major changes in the presentation layer.

3.2.6 Scalability

The architecture should support future expansion of the application.

Additional prediction models, health indicators, glucose-analysis features, reports, cloud services, or AI functions should be capable of being incorporated without redesigning the complete application.

The separation between the Flutter frontend, cloud backend, Gemini service, and Flask-based machine-learning API helps provide this modularity.

3.2.7 Compatibility

The application is designed using Flutter, allowing a common codebase to support Android and iOS platforms.

Device-specific features such as notification permissions, scheduled alarms, and battery optimization shall be handled according to the requirements of the corresponding operating system.

3.3 Overall Testing Strategy

Testing is an essential part of `glucosense_ai` because the application combines mobile functionality, cloud services, external AI services, scheduled notifications, and machine-learning prediction.

The application's documented testing coverage includes authentication and profile setup, glucose logging and retrieval, meal analysis, risk prediction, and reminder scheduling and notification delivery.

3.4 Chapter Summary

This chapter presented the methodology and requirements of the proposed **`glucosense_ai`** system. The system integrates a Flutter mobile application with cloud services, Google Gemini-based meal analysis, local reminder services, and a Flask-based machine-learning prediction API. The functional requirements defined the major operations provided by authentication, profile management, glucose logging, meal analysis, diabetes-risk prediction, trends, reports, and reminders.

The chapter also defined important non-functional requirements, including performance, security, usability, reliability, maintainability, scalability, and compatibility. Finally, different testing approaches and formal test cases were presented to verify the functionality and integration of the complete system.

One point should remain unchanged when we finalize this chapter: **we should not insert invented XGBoost accuracy/F1 values into Section 4.3.4**. Once the actual training run is available, we can replace that paragraph with the real classification results.

CHAPTER 04

SYSTEM DESIGN AND IMPLEMENTATION

4.0 Introduction

This chapter explains the design and implementation of **glucosense_ai**. While the previous chapters introduced the problem and reviewed related work, this chapter focuses on how the actual system is organized and how its different components communicate with one another.

glucosense_ai is not built as a single large application where every feature directly accesses the database or external services. The project follows a **feature-first clean architecture** in which the application is divided into separate features and each feature is further organized into presentation, domain, and data layers. This separation makes the code easier to understand, test, maintain, and extend.

The system consists of a Flutter mobile application, Firebase services, Supabase support, local storage, Google Gemini for meal analysis, and a Flask API that connects the mobile application with the trained XGBoost diabetes-risk prediction model. The application uses BLoC/Cubit for state management and Dio for network communication.

This chapter covers the system architecture, application architecture, use cases, class structure, sequence diagrams, machine-learning pipeline, model-training workflow, prediction API, and the implementation of the major application modules.

4.1 System Architecture

The overall architecture of glucosense_ai has been designed so that the mobile application acts as the main interface between the user and the services used by the system.

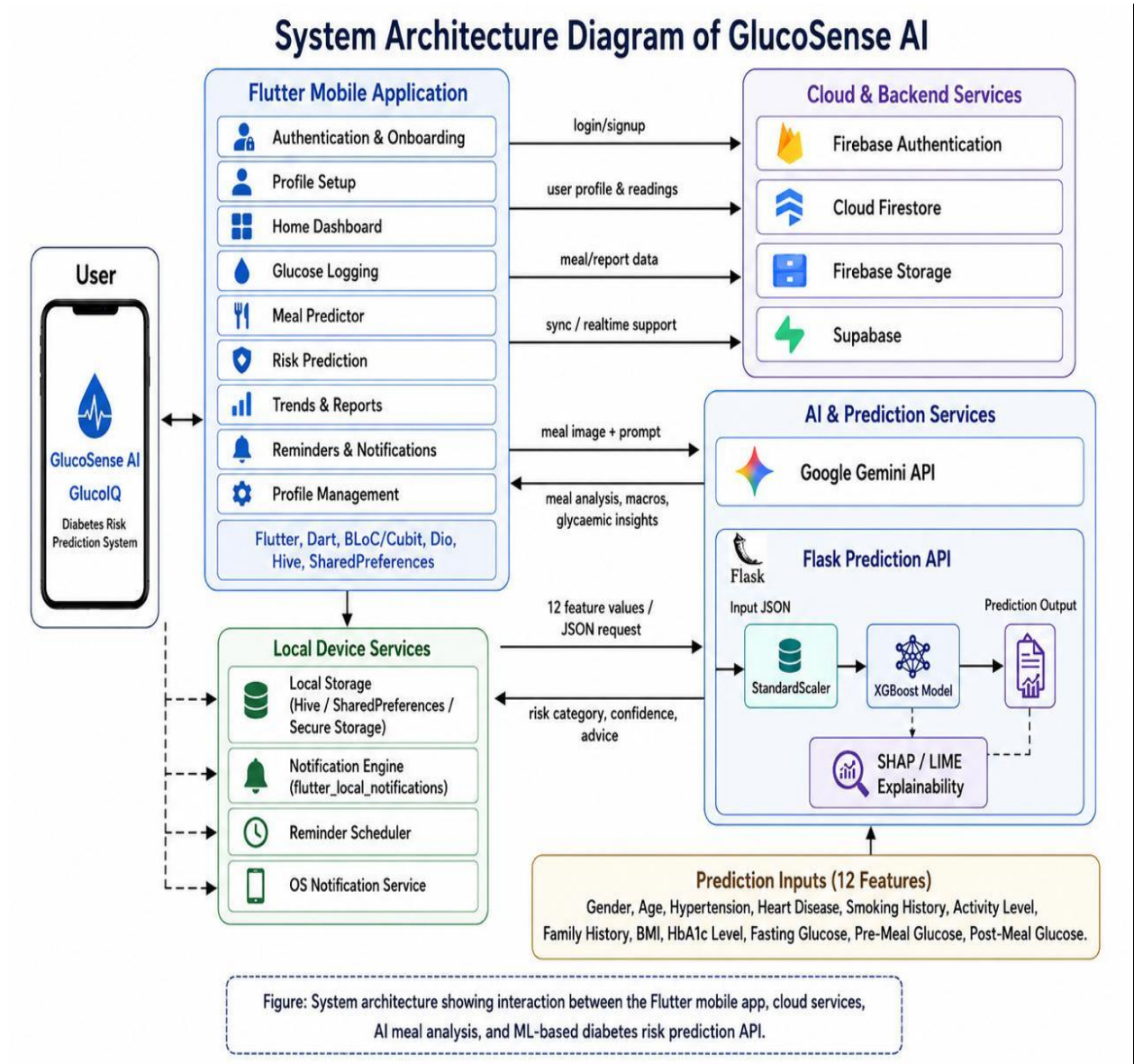


Figure 4.1: System Architecture Diagram of glucosense_ai

The user interacts with the system through the Flutter mobile application. From the user's point of view, all major operations such as registration, glucose logging, meal analysis, risk

prediction, viewing trends, generating reports, and managing reminders are available from the same application.

Behind the mobile interface, however, different services are responsible for different tasks.

The main architectural components are:

- Flutter Mobile Application
- Firebase Authentication
- Cloud Firestore
- Firebase Storage
- Supabase
- Local Device Storage
- Google Gemini API
- Flask Prediction API
- StandardScaler
- XGBoost Model
- Device Notification Services

The architecture intentionally separates normal application data from machine-learning processing. The mobile application does not directly load or execute the .pkl machine-learning model. Instead, it prepares the required patient information and sends it to the Flask API. The Flask service loads the saved scaler and XGBoost model, performs the prediction, and returns the result to the application.

Similarly, meal-image analysis is handled separately using Google Gemini. This keeps the meal-understanding function independent of the diabetes-risk classification model.

4.2 Components of the System Architecture

4.2.1 Flutter Mobile Application

Flutter is used to build the main user-facing application. Dart is used as the programming language, while BLoC and Cubit manage application state.

The mobile application contains features for:

- Authentication
- Onboarding
- Medical-profile setup
- Home dashboard
- Glucose logging
- Meal analysis
- Diabetes-risk prediction
- Trends
- Reports
- Reminders
- Notification history
- Profile management

These features are maintained as separate modules within the project structure rather than being placed together in one large codebase.

4.2.2 Firebase Authentication

Firebase Authentication is responsible for managing user accounts.

It supports:

- Registration
- Login
- Password reset

- Authentication-state checking
- Session-related access control

When the application starts, the system checks whether a user is already authenticated. Depending on the authentication and profile state, the user may be sent to onboarding, profile setup, or the main home screen.

4.2.3 Cloud Firestore

Cloud Firestore stores structured application data.

Examples include:

- User profile information
- Medical-profile information
- Glucose readings
- Saved meal information
- Meal-analysis results
- Risk-related records
- Reminder configuration
- Other user-specific application data

Firestore provides cloud-based persistence so information does not have to remain only on the user's phone.

4.2.4 Firebase Storage

Firebase Storage is used where image or file storage is required.

One important use of storage is meal-related image handling. A meal photograph can be associated with the corresponding meal record and analysis.

4.2.5 Supabase

Supabase is included as an additional realtime/backend service in the project.

One advantage of the clean architecture used in `glucosense_ai` is that repositories are separated from concrete data sources. This means a backend data source can be replaced or supplemented without requiring major changes to the user-interface layer.

4.2.6 Google Gemini API

Google Gemini is used for the **meal-analysis module**.

When a user captures or selects a photograph of a meal, the image is analysed to obtain information such as:

- Meal understanding
- Estimated calories
- Carbohydrate information
- Protein
- Fat
- Estimated glycaemic impact
- General food-related insights
- Potentially healthier alternatives

The meal-analysis results are then displayed in the mobile application and can be saved for future reference. The application specifically provides a meal-predictor screen that accepts an image and returns nutritional and diabetic-friendly guidance.

4.2.7 Flask Prediction API

The Flask API acts as a bridge between the Flutter application and the trained machine-learning model.

The mobile application sends twelve required input values in JSON format to the API. The API preprocesses them using the same scaler used during model training and then forwards the scaled feature vector to the XGBoost classifier.

The two primary endpoints are:

- GET /health
- POST /predict

The /health endpoint is used to verify that the prediction service is available.

The /predict endpoint is responsible for generating diabetes-risk predictions.

4.2.8 XGBoost Prediction Model

The final trained model is stored as:

XGBoost_model.pkl

It predicts three possible classes:

- Normal
- Prediabetic
- Diabetic

The model receives twelve features describing demographic, clinical, behavioral, and glucose-related information.

These are:

1. Gender
2. Age
3. Hypertension
4. Heart disease

5. Smoking history
6. Activity level
7. Family history
8. BMI
9. HbA1c level
10. Fasting glucose
11. Pre-meal glucose
12. Post-meal glucose

The prediction response includes the predicted category, confidence score, probability values for the available classes, and short guidance.

4.2.9 Local Storage

glucosense_ai also uses local device storage.

Hive, SharedPreferences, and secure storage are used for different purposes. The application infrastructure contains a Hive-backed notification history as well as session preferences and local storage utilities.

The use of local storage is useful because not every feature should depend on a continuous internet connection.

4.2.10 Notification Service

The notification system is responsible for generating scheduled reminders on the user's device.

The implementation contains a low-level notification engine and a separate reminder scheduler. The notification service manages permissions, channels, immediate notifications, scheduled notifications, cancellation, history, and diagnostics, while the

reminder scheduler calculates upcoming reminder times and handles higher-level reminder logic.

4.3 Application Architecture

glucosense_ai follows a **feature-first clean architecture**.

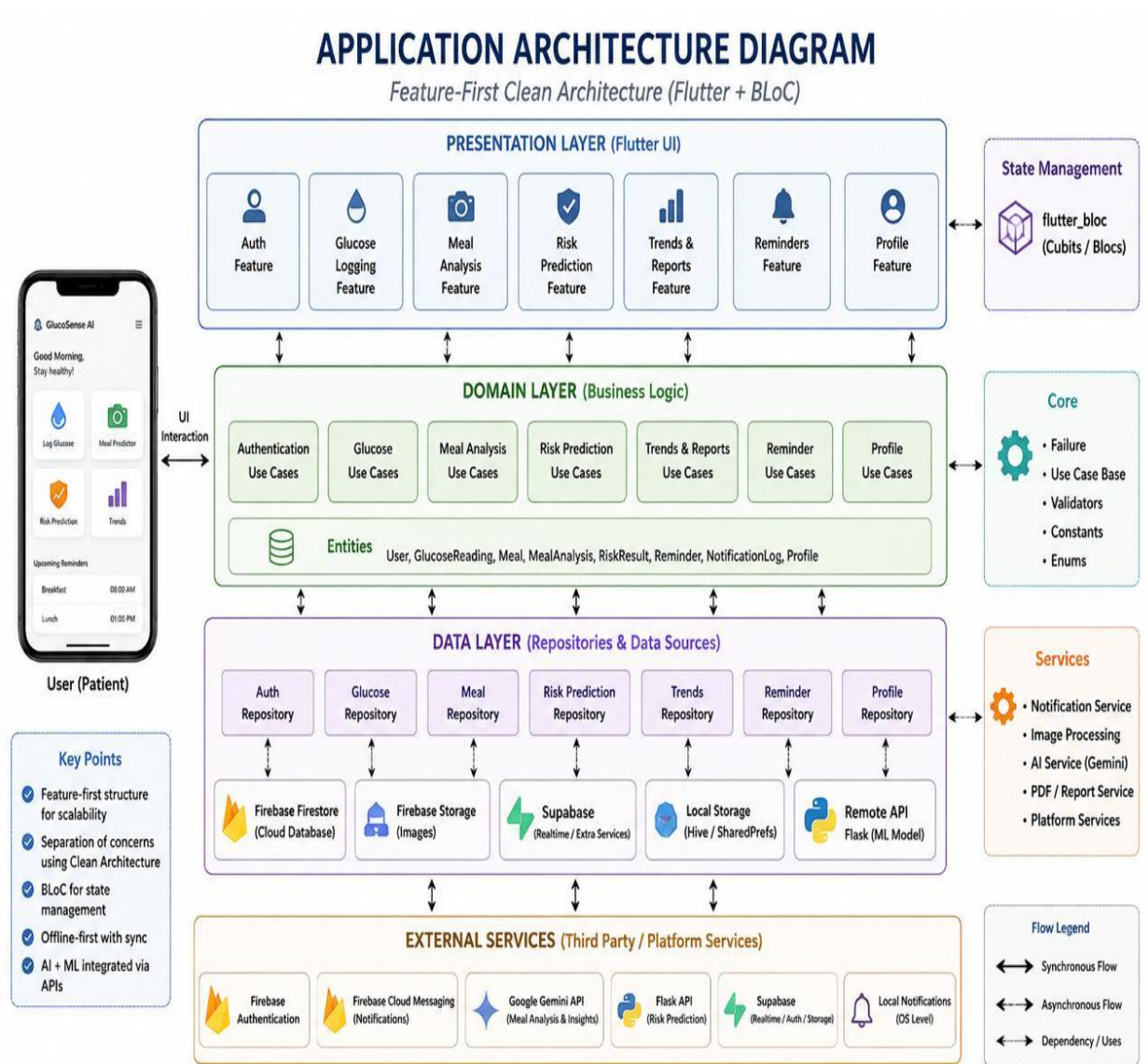


Figure 4.2: Application Architecture of glucosense_ai

Instead of grouping the complete project only by technical type, the application is divided into features. Each feature contains the code required for that particular function.

For example:

features/

auth/

log_reading/

meal/

risk_prediction/

trends/

reports/

reminders/

profile/

Each feature is then divided into three main layers:

Presentation Layer



Domain Layer



Data Layer

The project source describes this directly as a three-layer feature architecture containing data, domain, and presentation.

4.3.1 Presentation Layer

The presentation layer contains everything directly related to the user interface.

It includes:

- Screens
- Widgets
- Forms
- BLoCs
- Cubits
- UI states

For example, when a user enters a glucose value, the screen itself is not responsible for writing directly to Firestore. It passes the required action to a BLoC or Cubit.

This keeps the user interface focused on presentation.

4.3.2 Domain Layer

The domain layer contains the central business rules of the application.

It includes:

- Entities
- Repository contracts
- Business operations
- Feature-specific rules

The domain layer does not depend directly on Flutter or Firebase. This keeps it relatively independent from frameworks and external services.

Important domain entities include:

- User
 - MedicalProfile
 - GlucoseReading
 - Meal
 - MealAnalysis
 - RiskPrediction
 - Reminder
 - NotificationLog
-

4.3.3 Data Layer

The data layer handles interaction with actual data sources.

It includes:

- Data-source implementations
- JSON models
- Repositories
- Firebase access
- Supabase access
- API clients
- Gemini communication

The preferred communication flow is:

Widget → Cubit/BLoC → Repository → DataSource → External Service

The project specifically avoids having UI components directly communicate with Firebase or Gemini.

This separation reduces coupling and makes individual components easier to replace or test.

4.4 Use Case Diagram

The use case diagram provides a high-level view of the functions available in glucosense_ai.

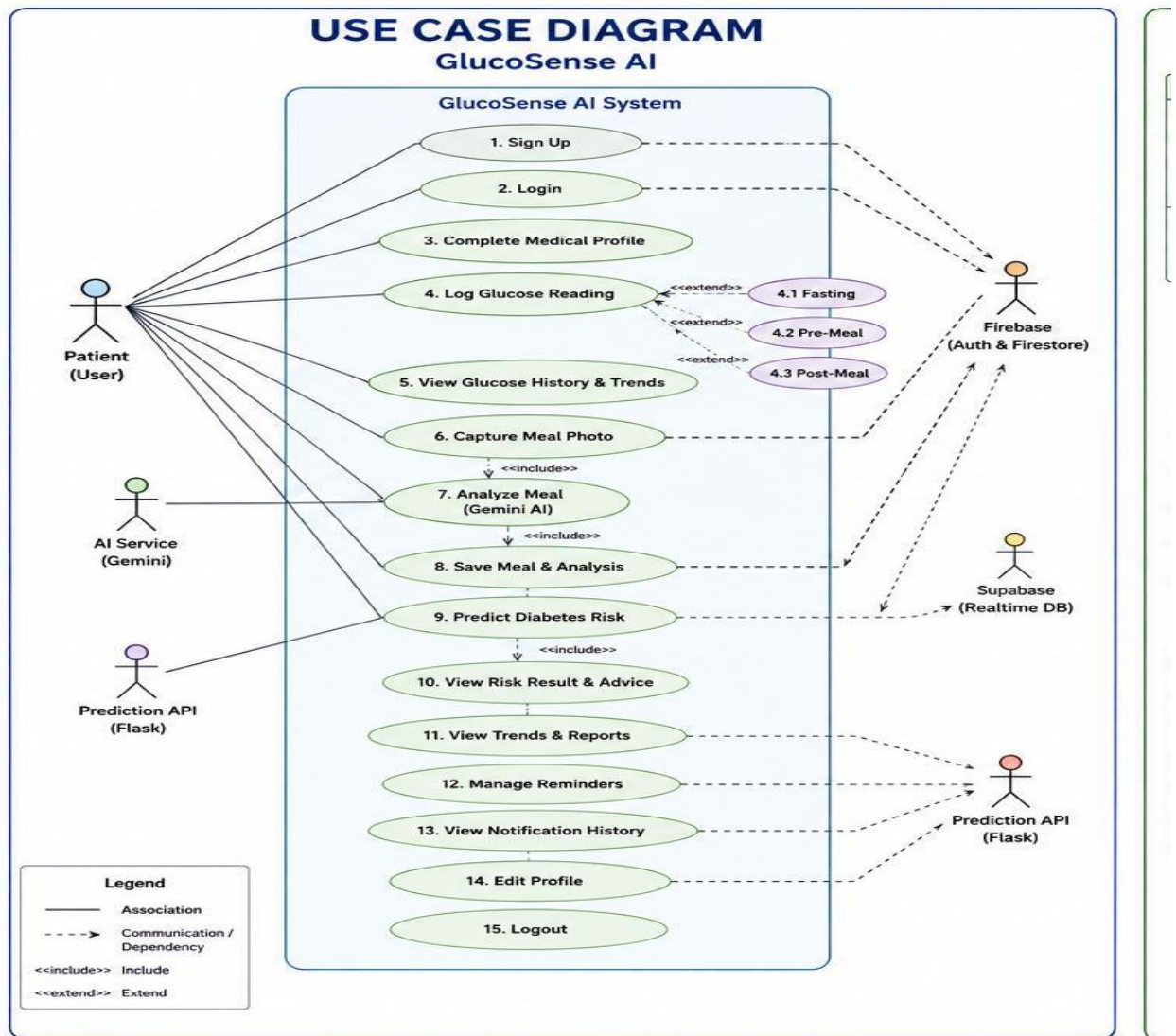


Figure 4.3: Use Case Diagram of glucosense_ai

The primary actor is the **Patient/User**.

Supporting systems include:

- Firebase
- Google Gemini
- Flask Prediction API
- Supabase
- Device notification services

The major use cases are:

- Sign Up
- Login
- Reset Password
- Complete Medical Profile
- Edit Profile
- Log Glucose Reading
- View Glucose History
- Analyze Meal
- Save Meal Analysis
- Predict Diabetes Risk
- View Prediction and Advice
- View Trends
- Generate Reports
- Manage Reminders
- View Notification History
- Logout

The application's actual screen organization follows these same functional areas, including separate screens for authentication, medical-profile setup, glucose logging, meal prediction, risk prediction, trends, reports, reminders, notifications, and profile management.

4.5 Use Case Tabular Description

4.5.1 Sign Up Use Case

Field	Description
Use Case ID	UC-01
Use Case Name	Sign Up
Description	Creates a new user account.
Actor	Patient/User
Pre-Condition	User does not already have an account.
Main Flow	User enters required account details and submits registration.
Post-Condition	User account is created successfully.

4.5.2 Login Use Case

Field	Description
Use Case ID	UC-02
Use Case Name	Login
Description	Authenticates a registered user.
Actor	Patient/User
Pre-Condition	User has a valid account.
Main Flow	User enters email and password; Firebase verifies credentials.
Post-Condition	User enters the application if authentication succeeds.

4.5.3 Complete Medical Profile Use Case

Field	Description
Use Case ID	UC-03
Use Case Name	Complete Medical Profile
Description	Collects personal and medical information required by the system.
Actor	Patient/User
Pre-Condition	User is registered and authenticated.
Main Flow	User enters medical/profile details and saves them.
Post-Condition	Medical profile becomes available to relevant features.

4.5.4 Log Glucose Reading Use Case

Field	Description
Use Case ID	UC-04
Use Case Name	Log Glucose Reading
Description	Records a fasting, pre-meal, or post-meal glucose reading.
Actor	Patient/User
Pre-Condition	User is logged in.
Main Flow	User selects reading type, enters glucose value and related information, then saves it.
Post-Condition	Glucose reading is stored and becomes available in history and trends.

4.5.5 View Glucose History Use Case

Field	Description
Use Case ID	UC-05
Use Case Name	View Glucose History
Description	Displays previously recorded glucose measurements.
Actor	Patient/User
Pre-Condition	At least one glucose record is available.
Post-Condition	Historical readings are displayed.

4.5.6 Analyze Meal Use Case

Field	Description
Use Case ID	UC-06
Use Case Name	Analyze Meal
Description	Analyzes a captured or selected meal photograph using AI.
Actor	Patient/User
Supporting Service	Google Gemini
Pre-Condition	User provides a valid meal image.
Post-Condition	Meal analysis and dietary insights are displayed.

4.5.7 Save Analyzed Meal Use Case

Field	Description
Use Case ID	UC-07
Use Case Name	Save Analyzed Meal
Description	Saves generated meal information for later viewing.
Actor	Patient/User
Pre-Condition	Meal analysis has been completed.
Post-Condition	Meal is stored and becomes available in saved meals.

4.5.8 Predict Diabetes Risk Use Case

Field	Description
Use Case ID	UC-08
Use Case Name	Predict Diabetes Risk
Description	Generates a three-class diabetes-risk prediction.
Actor	Patient/User
Supporting Service	Flask Prediction API
Pre-Condition	Required profile and prediction information is available.
Post-Condition	Risk category and probability information are returned.

4.5.9 View Risk Result and Advice Use Case

Field	Description
Use Case ID	UC-09
Use Case Name	View Risk Result and Advice
Description	Displays the predicted risk category, confidence, probabilities, and guidance.
Actor	Patient/User
Pre-Condition	Prediction has been completed.
Post-Condition	User can review the generated result.

4.5.10 View Trends Use Case

Field	Description
Use Case ID	UC-10
Use Case Name	View Trends
Description	Displays summarized glucose information over time.

Field	Description
Actor	Patient/User
Pre-Condition	Historical readings exist.
Post-Condition	Charts and statistics are displayed.

The implemented trends screen supports 7-day, 30-day, and 90-day views and includes time-in-range, mean, range, and time-of-day information.

4.5.11 Generate Report Use Case

Field	Description
Use Case ID	UC-11
Use Case Name	Generate Report
Description	Creates a summary over a selected date range.
Actor	Patient/User
Pre-Condition	Relevant readings or meal records exist.
Post-Condition	Report is generated and can be shared.

4.5.12 Manage Reminders Use Case

Field	Description
Use Case ID	UC-12
Use Case Name	Manage Reminders
Description	Allows users to configure glucose- and meal-related reminders.
Actor	Patient/User
Pre-Condition	User has access to reminder settings.
Post-Condition	Selected reminders are scheduled or updated.

4.5.13 View Notification History Use Case

Field	Description
Use Case ID	UC-13
Use Case Name	View Notification History
Description	Shows previously delivered notifications.
Actor	Patient/User
Pre-Condition	Notification records exist.
Post-Condition	Notification history is displayed.

4.5.14 Edit Profile Use Case

Field	Description
Use Case ID	UC-14
Use Case Name	Edit Profile
Description	Allows the user to modify personal or medical-profile information.
Actor	Patient/User
Pre-Condition	User is authenticated.
Post-Condition	Updated information is saved.

4.5.15 Logout Use Case

Field	Description
Use Case ID	UC-15
Use Case Name	Logout
Description	Ends the active user session.
Actor	Patient/User
Pre-Condition	User is logged in.
Post-Condition	User returns to the authentication flow.

4.6 Class Diagram

The class diagram describes the main objects used by the application and the relationships between them.

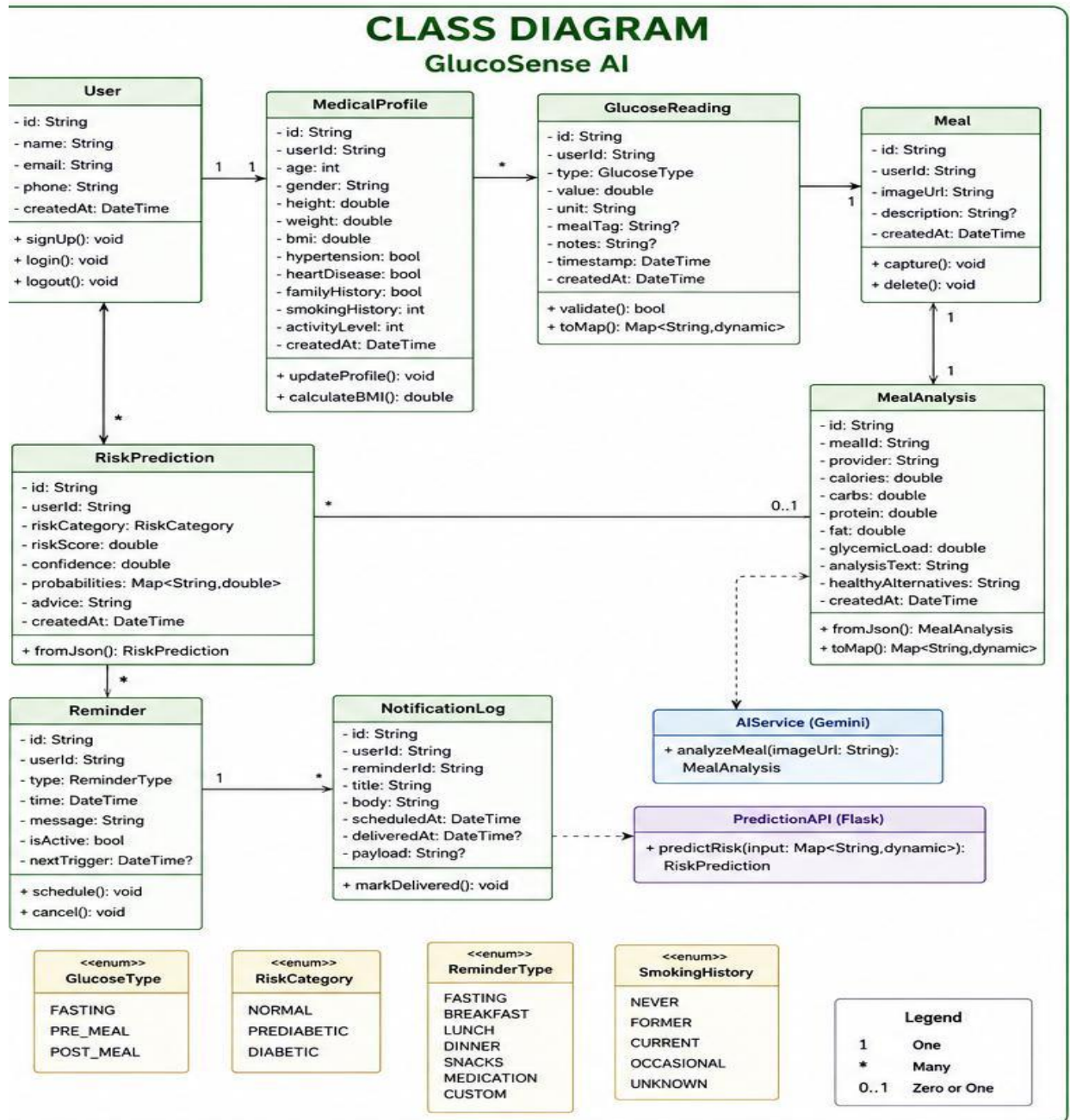


Figure 4.4: Class Diagram of glucosense_ai

The major classes are described below.

User

The User class represents the application account.

Typical information includes:

- ID
- Name
- Email
- Phone
- Account creation time

A user is associated with a medical profile and with the health information created through the application.

MedicalProfile

The MedicalProfile class stores information required for health-related functionality.

Important attributes include:

- Age
- Gender
- Height
- Weight
- BMI
- Hypertension
- Heart disease
- Family history
- Smoking history
- Activity level

The medical profile is important for the diabetes-risk prediction process.

GlucoseReading

The GlucoseReading class represents a single recorded measurement.

It contains information such as:

- Reading ID
- User ID
- Reading type
- Glucose value
- Measurement unit
- Meal tag
- Notes
- Timestamp

Reading types include:

- Fasting
- Pre-meal
- Post-meal

Meal

The Meal class represents a meal stored by the user.

It may contain:

- Meal ID
- User ID
- Image URL

- Meal name
- Description
- Creation time

A meal can be linked with its analysis.

MealAnalysis

MealAnalysis contains information returned after the meal image has been interpreted.

Possible fields include:

- Calories
- Carbohydrates
- Protein
- Fat
- Glycaemic load
- AI-generated notes
- Alternative recommendations

RiskPrediction

The RiskPrediction class represents a diabetes-risk result.

It can contain:

- Predicted category
- Risk score
- Confidence
- Per-class probabilities
- Advice

- Creation time

Possible risk categories are:

- Normal
- Prediabetic
- Diabetic

Reminder

The Reminder class represents a scheduled user reminder.

Possible types include:

- Fasting
- Breakfast
- Lunch
- Dinner
- Snack
- Custom

The reminder stores its time, activity status, repeat behavior, and next scheduled occurrence.

NotificationLog

The NotificationLog class records delivered or scheduled notifications.

This supports the notification-history feature and allows the user to review previous alerts.

4.7 Sequence Diagrams

Sequence diagrams describe how application components communicate during a particular process.

Instead of showing only what functionality exists, these diagrams show **the order in which messages are exchanged** between the user, application, backend, local storage, AI services, and prediction services.

4.7.1 Sequence Diagram for Glucose Logging

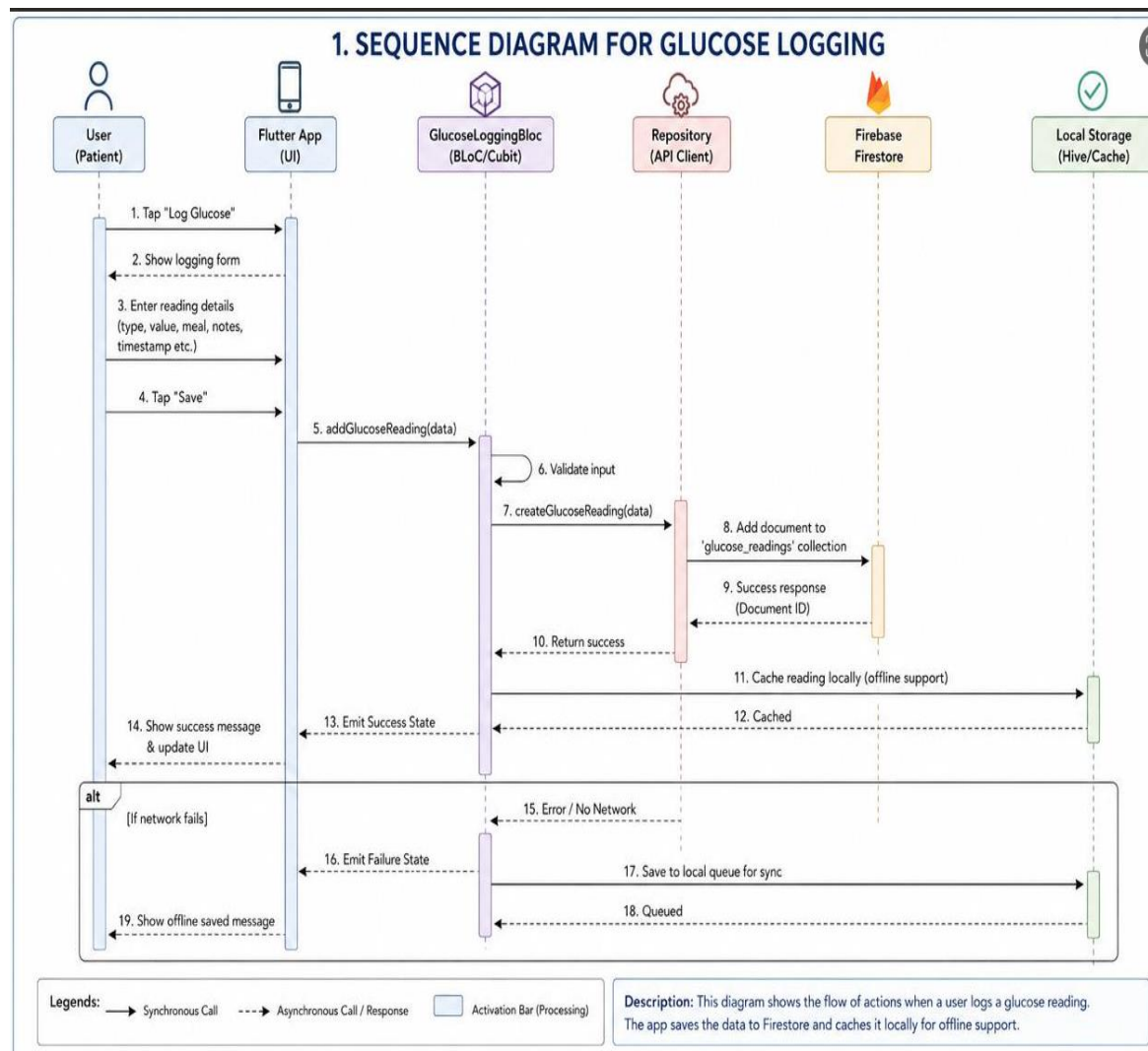


Figure 4.5: Sequence Diagram for Glucose Logging

The process begins when the user selects the glucose-logging feature.

The user provides information such as:

- Reading type
- Glucose value
- Meal information
- Notes
- Timestamp

After the user taps the save button, the interface passes the entered information to the Glucose Logging BLoC or Cubit.

The data is first validated. If the input is valid, the relevant repository sends the record to the selected persistent data source. The saved record then becomes available for history, trends, and reports.

The application also contains local-storage infrastructure, which supports offline-aware behavior and local application state. This fits the project's separation between UI, repositories, data sources, and local/cloud services.

If the operation fails, the interface receives a failure state instead of silently ignoring the error.

This is consistent with the application's architecture, where asynchronous operations are expected to return explicit loading, success, empty, or error states.

4.7.2 Sequence Diagram for Meal Photo Analysis

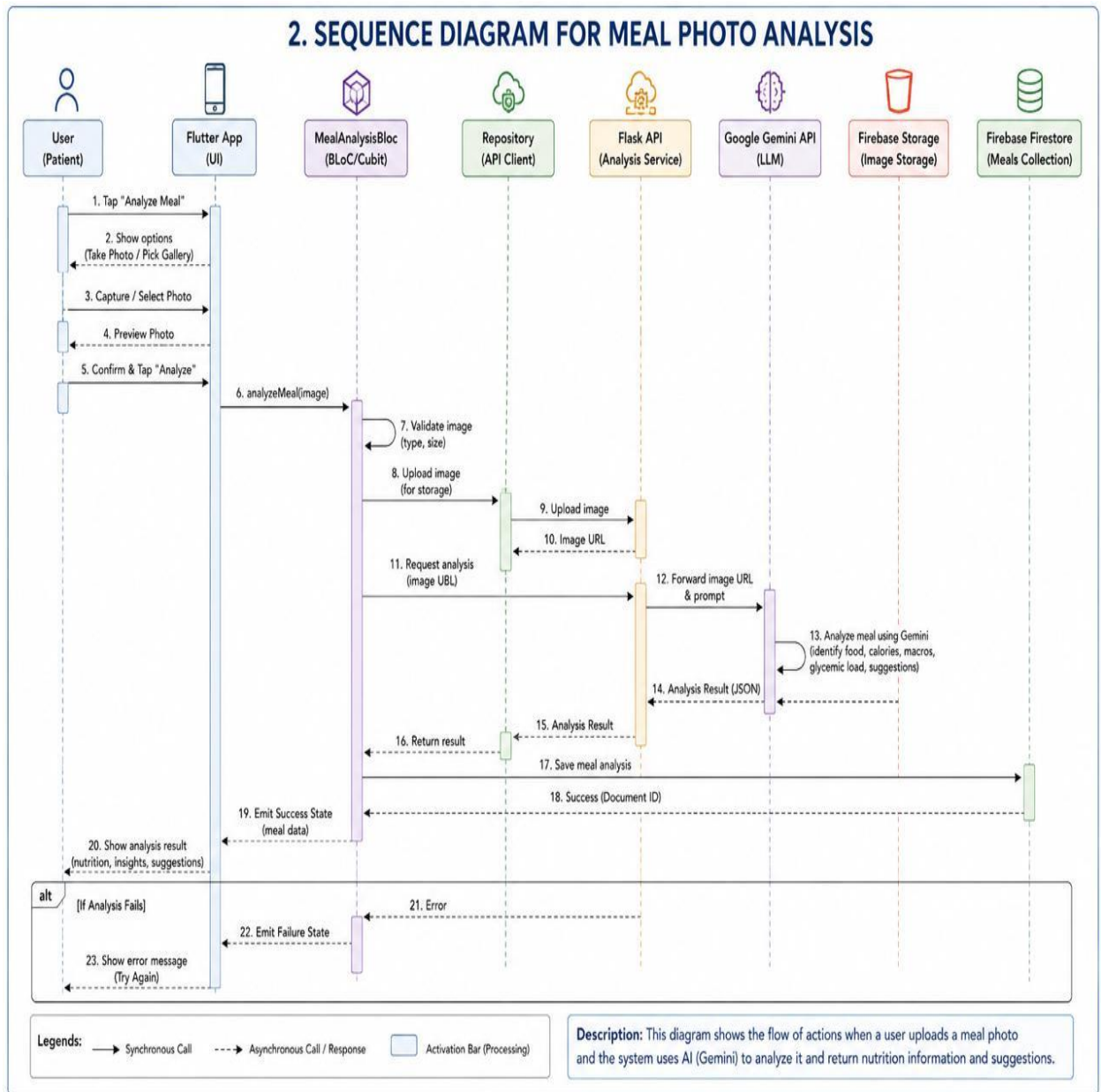


Figure 4.6: Sequence Diagram for Meal Photo Analysis

Meal analysis begins when the user opens the meal-prediction feature and either captures a photograph or selects an existing image.

The general sequence is:

1. User selects or captures the meal image.

2. Application displays the selected image.
3. User confirms the analysis request.
4. Meal-analysis state-management component processes the request.
5. Image input is validated.
6. Required image/storage operations are performed.
7. The image and analysis instructions are sent through the meal-analysis data layer.
8. Google Gemini analyses the meal.
9. Nutritional and glycaemic-related information is returned.
10. The application converts the result into usable meal-analysis data.
11. The analysis can be saved.
12. The result is displayed to the user.

The actual application design includes a meal-predictor screen that sends a selected image for analysis and returns macros, glycaemic estimates, and diabetic-friendly alternatives.

If the AI request fails, the application returns a failure state and informs the user instead of storing an incomplete result.

4.7.3 Sequence Diagram for Diabetes Risk Prediction

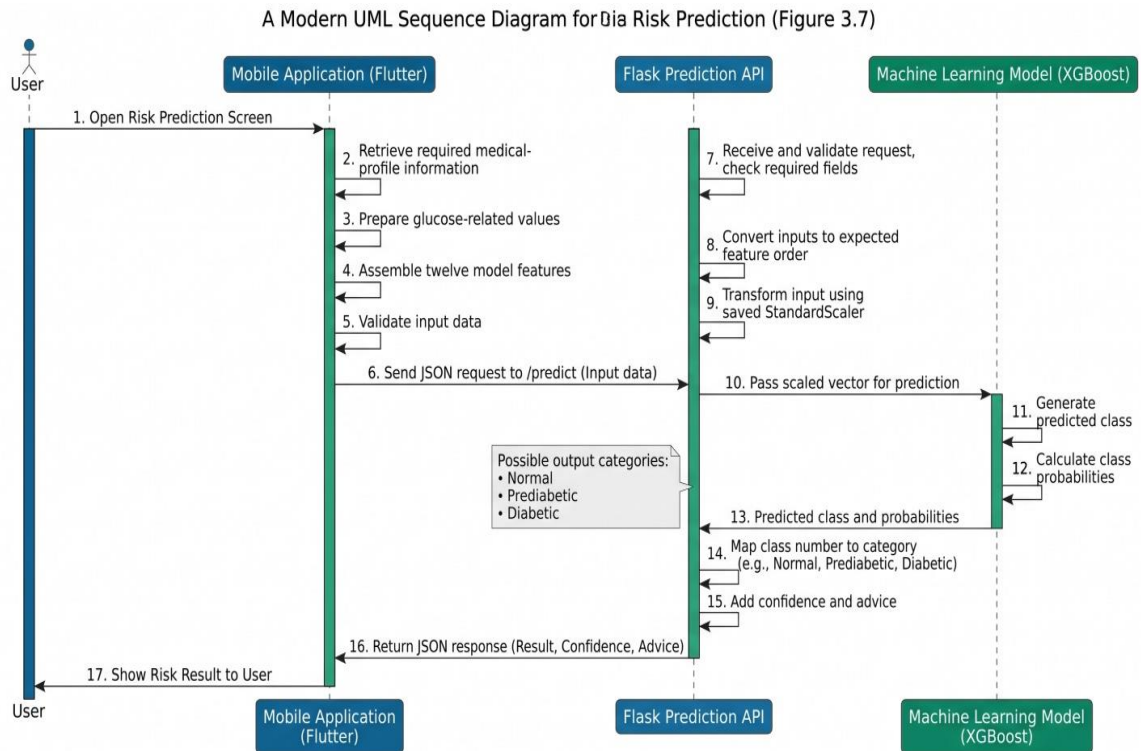


Figure 4.7: Sequence Diagram for Diabetes Risk Prediction

The diabetes-risk prediction sequence connects the mobile application with the Flask prediction API.

The flow is:

1. User opens the Risk Prediction screen.
2. The application retrieves required medical-profile information.
3. Required glucose-related values are prepared.
4. Twelve model features are assembled.
5. Input data is validated.
6. Flutter sends a JSON request to /predict.
7. Flask receives the request.
8. Required fields are checked.
9. Input values are converted into the expected feature order.
10. The saved StandardScaler transforms the input.

11. The scaled vector is passed to the XGBoost model.
12. The model generates the predicted class.
13. Class probabilities are calculated.
14. The API converts the class number into the corresponding category.
15. Confidence and advice are added.
16. JSON response is returned to the application.
17. Risk result is shown to the user.

The possible output categories are:

Normal, Prediabetic, and Diabetic.

4.7.4 Sequence Diagram for Reminder Scheduling and Notification Delivery

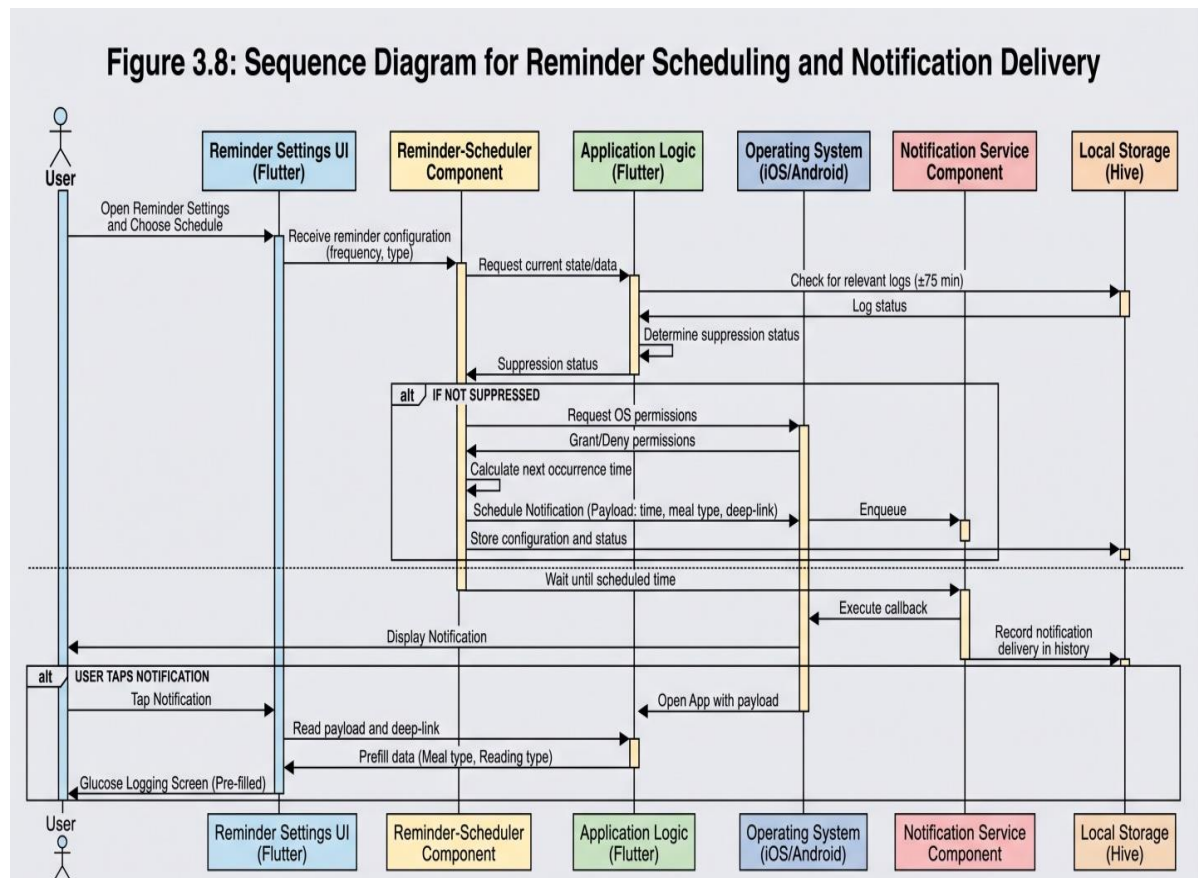


Figure 4.8: Sequence Diagram for Reminder Scheduling and Notification Delivery

The reminder feature has more device-level interaction than most other application features.

The process begins when the user opens the reminder settings and chooses the required schedule.

The system:

1. Receives the reminder configuration.
2. Calculates the next reminder occurrence.
3. Checks whether the reminder should be suppressed.
4. Requests any required operating-system permissions.
5. Schedules the notification.
6. Stores the configuration.
7. Waits until the scheduled time.
8. Device notification service delivers the notification.
9. Notification delivery can be recorded in Hive.
10. If the user taps the notification, its payload is read.
11. The relevant glucose-logging screen is opened.
12. The meal tag or reading type can be prefilled automatically.

The actual implementation uses separate notification-service and reminder-scheduler components. It also keeps a Hive-backed notification history.

A useful feature is **smart suppression**. If the user has already logged the reading associated with a reminder within approximately ± 75 minutes, the reminder can be skipped during the next scheduling process.

Notification taps can also deep-link directly to the glucose logging screen with appropriate reading information.

4.7.5 Sequence Diagram for Viewing Trends and Reports

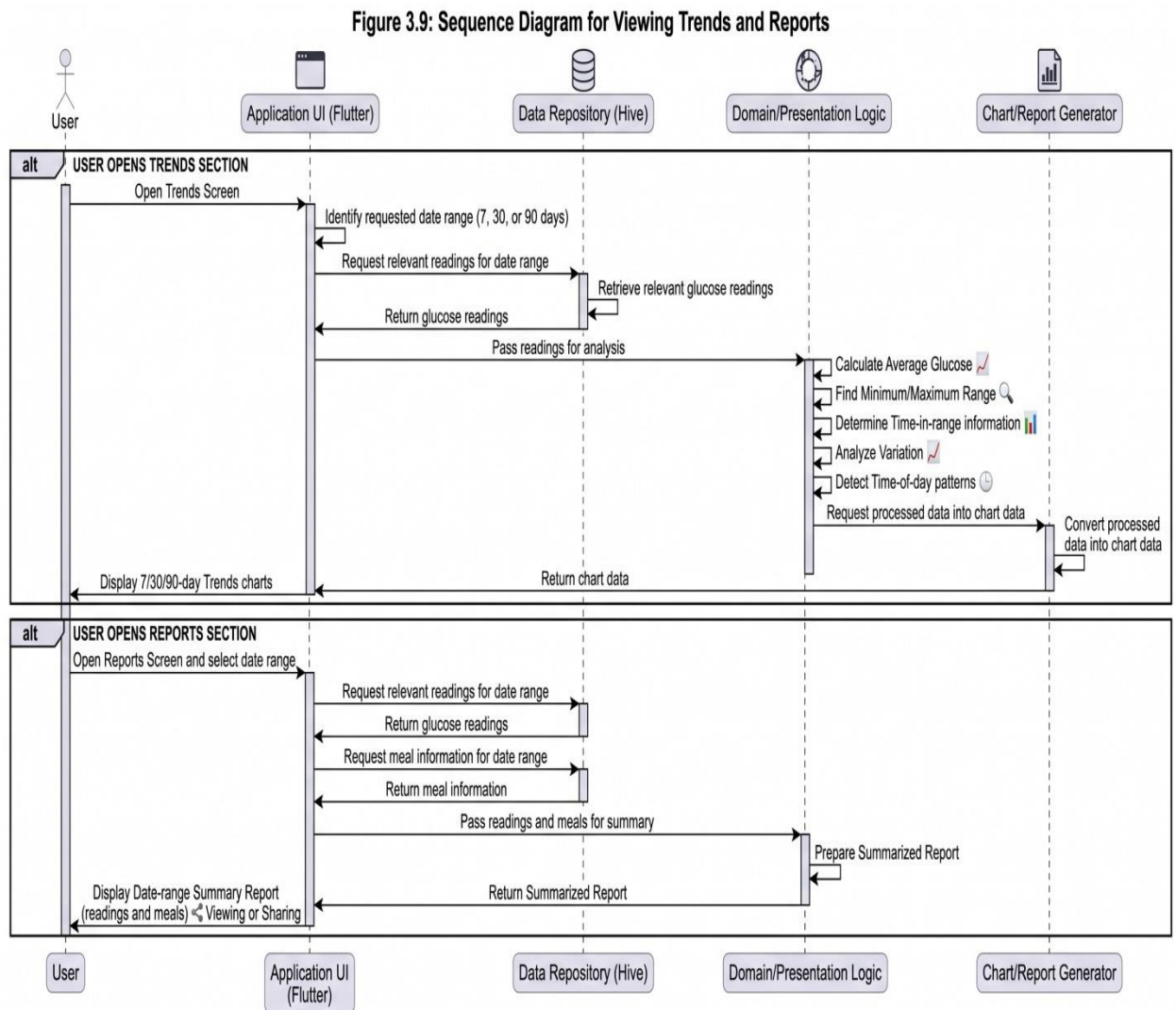


Figure 4.9: Sequence Diagram for Viewing Trends and Reports

When a user opens the trends or reports section, the application first identifies the requested date range.

For trends, commonly available ranges are:

- 7 days
- 30 days
- 90 days

The repository retrieves relevant readings and passes them to the domain/presentation logic.

The system then calculates or prepares values such as:

- Average glucose
- Minimum and maximum range
- Time-in-range information
- Variation
- Time-of-day patterns

The processed information is converted into chart data and displayed to the user.

For reports, the user selects a date range and the application retrieves both reading and meal information for that period. A summarized report is then prepared for viewing or sharing. The application documentation confirms that the trends screen supports 7-, 30-, and 90-day charts, while the reports screen provides date-range summaries of readings and meals.

4.8 Machine Learning Pipeline

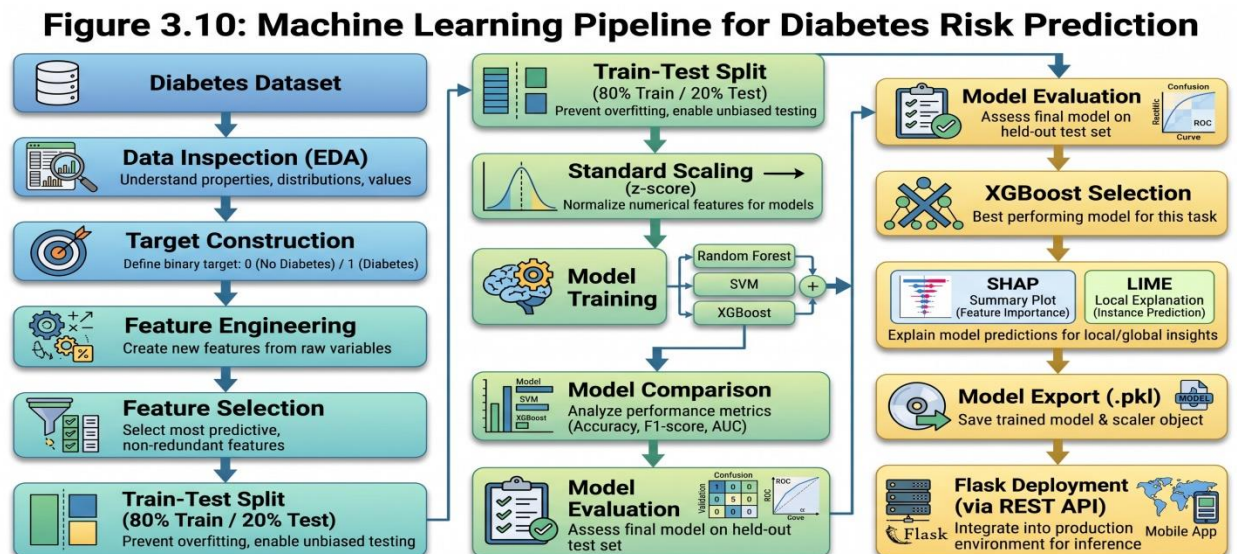


Figure 4.10: Machine Learning Pipeline for Diabetes Risk Prediction

The machine-learning pipeline begins with the diabetes dataset and ends with an exported model that can be accessed through the Flask API.

The overall pipeline is:

Dataset



Data Inspection



Target Construction



Feature Engineering



Feature Selection



Train-Test Split



Standard Scaling



Model Training



Model Comparison



Model Evaluation



XGBoost Selection



SHAP / LIME Analysis



Model Export



Flask Deployment

4.9 Dataset and Target Preparation

The training notebook expects the dataset file:

diabetes_prediction_dataset.csv

The original dataset contains a binary field:

diabetes

with values:

- 0 = not labelled diabetic
- 1 = diabetic

For this project, a new three-class target named risk_category is created.

The target mapping is:

Class	Risk Category
0	Normal
1	Prediabetic
2	Diabetic

The rule used is:

IF diabetes == 1

→ Diabetic

ELSE IF HbA1c_level >= 5.7

OR blood_glucose_level >= 100

→ Prediabetic

ELSE

→ Normal

An important point is that **Prediabetic is a derived class** rather than an original diagnosis label in the dataset.

This distinction should be kept clear when interpreting the results.

4.10 Feature Engineering

Several additional features are created to align the machine-learning dataset with the information collected by the mobile application.

The engineered features include:

- Fasting glucose
- Pre-meal glucose
- Post-meal glucose
- Family history
- Activity level

The final model uses twelve features.

No.	Feature
1	Gender
2	Age
3	Hypertension
4	Heart disease
5	Smoking history
6	Activity level
7	Family history
8	BMI
9	HbA1c level
10	Fasting glucose
11	Pre-meal glucose
12	Post-meal glucose

The same feature order must be maintained during both training and API inference. Changing the order at prediction time could result in incorrect outputs even if all twelve values are provided.

4.11 Data Splitting

The processed data is divided into:

- **80% training data**
- **20% testing data**

A stratified split is used.

Stratification helps preserve approximately the same class distribution in both the training and test sets, which is particularly useful when working with multiclass classification.

The test set is not used for fitting the model.

4.12 Feature Scaling

StandardScaler is applied during preprocessing.

Importantly, the scaler is fitted only on the training data.

The process is:

Training Data



Fit StandardScaler



Transform Training Data

Test Data



Use Same Fitted Scaler



Transform Test Data

After training, the fitted scaler is exported as:

scaler.pkl

The API later uses this same scaler for new patient input.

Using the same preprocessing procedure during training and deployment prevents differences between how training samples and real application requests are interpreted.

4.13 Model Training Workflow

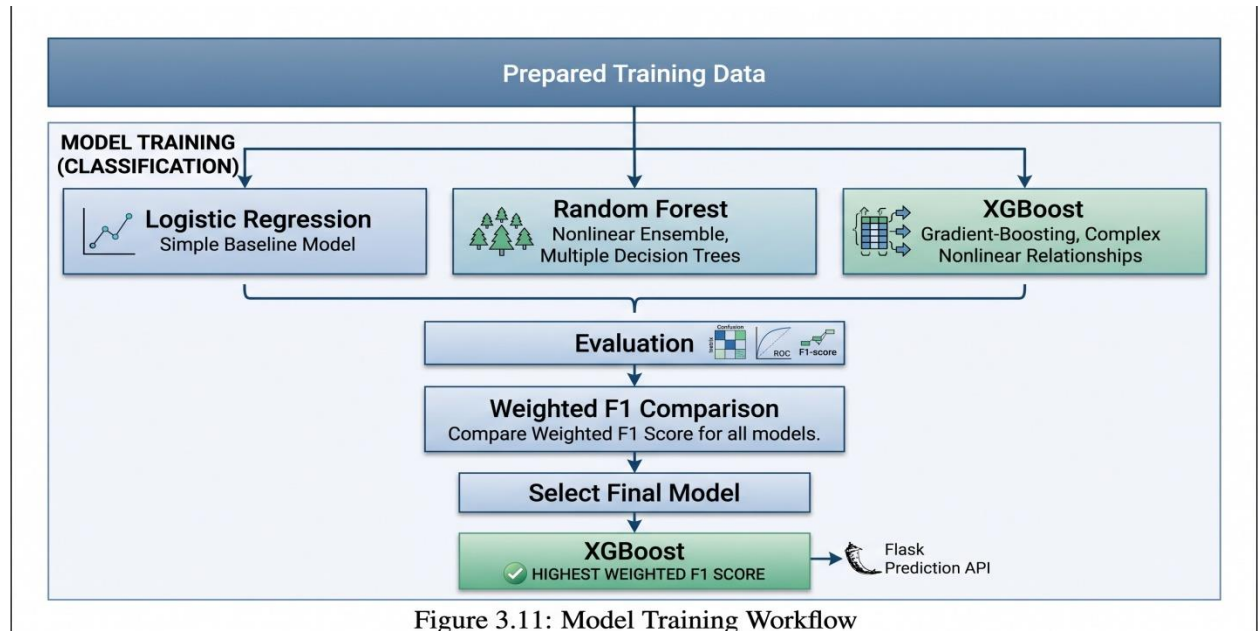


Figure 3.11: Model Training Workflow

Figure 4.11: Model Training Workflow

Three classification algorithms are trained and compared:

Logistic Regression

Logistic Regression provides a relatively simple baseline model.

Random Forest

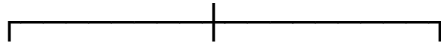
Random Forest provides a nonlinear ensemble model made from multiple decision trees.

XGBoost

XGBoost is a gradient-boosting model capable of learning complex nonlinear relationships from structured data.

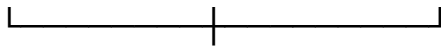
The training workflow can be summarized as:

Prepared Training Data



Logistic Random XGBoost

Regression Forest



Evaluation



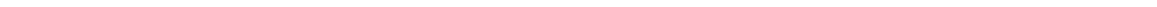
Weighted F1 Comparison



Select Final Model



XGBoost



4.14 Prediction API Workflow

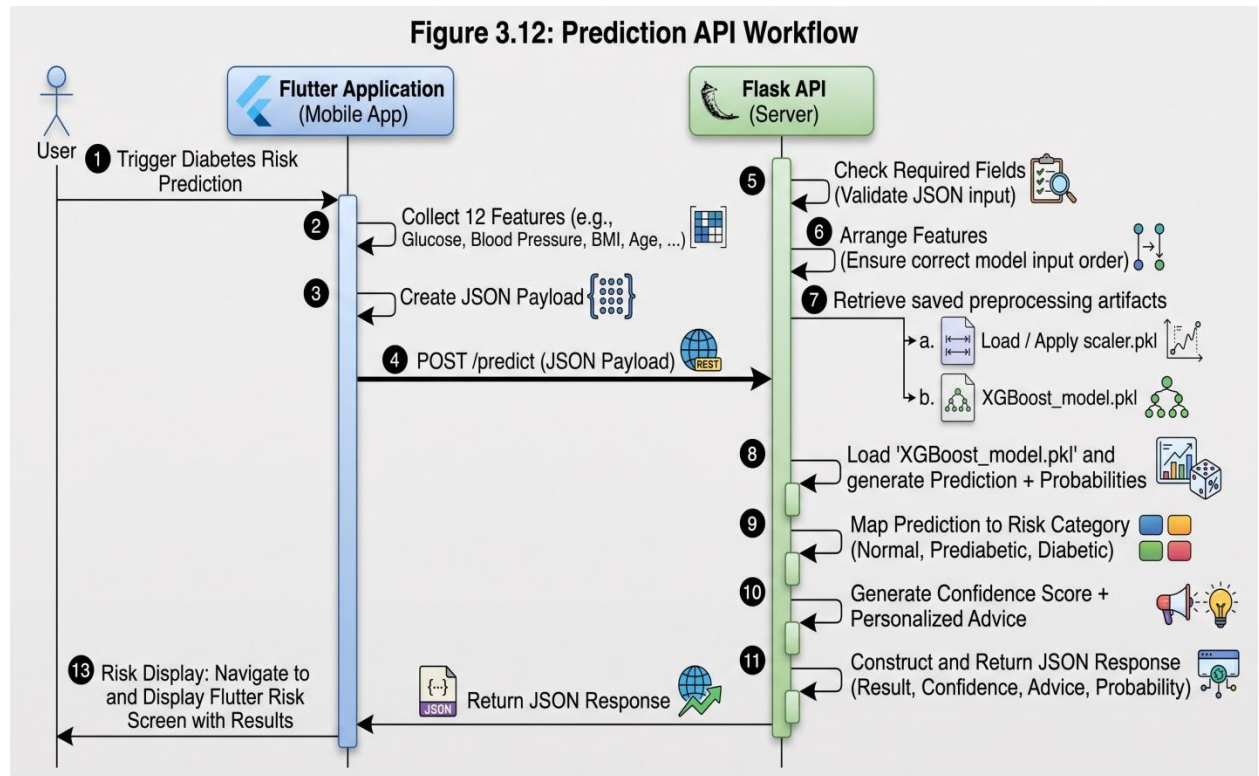


Figure 4.12: Prediction API Workflow

The prediction API allows the trained model to be used by the mobile application without embedding the Python model directly inside Flutter.

The API workflow is:

Flutter Application



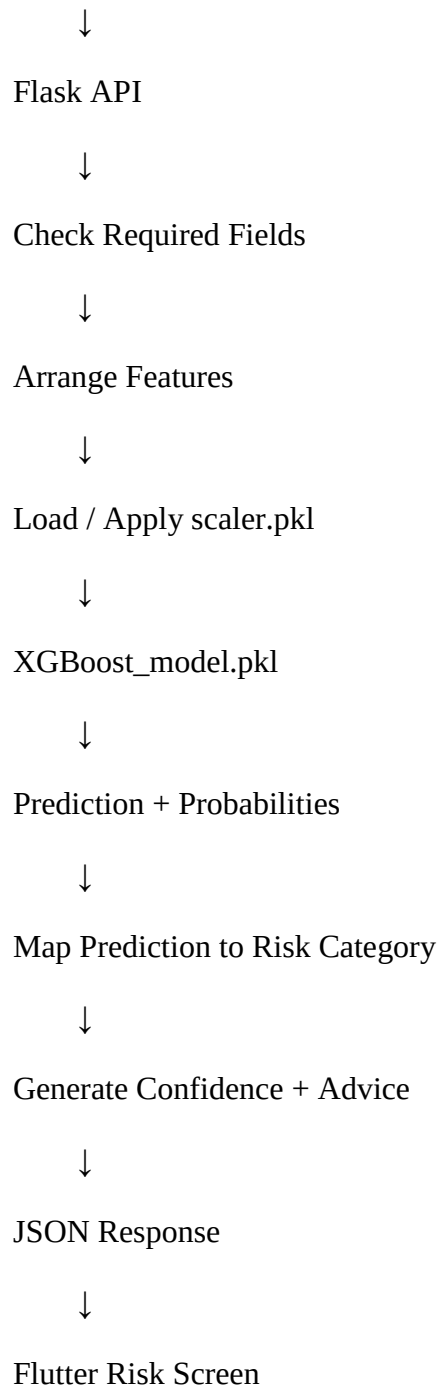
Collect 12 Features



Create JSON Payload



POST /predict



4.15 Prediction API Endpoints

4.15.1 Health Endpoint

Request

GET /health

The health endpoint can return information such as:

- API availability
- Model status
- Expected feature list
- Available output classes

This allows the system or developer to confirm whether the machine-learning service is running correctly.

4.15.2 Prediction Endpoint

Request

POST /predict

The request contains the twelve required fields in JSON format.

An example structure is:

```
{  
  "gender": 1,  
  "age": 52,  
  "hypertension": 1,  
  "heart_disease": 0,
```

```
"smoking_history": 3,  
"activity_level": 1,  
"family_history": 1,  
"bmi": 30.2,  
"HbA1c_level": 6.4,  
"fasting_glucose": 128,  
"pre_meal_glucose": 121,  
"post_meal_glucose": 186  
}
```

The response contains information similar to:

Risk Category

Confidence

Probabilities

Advice

The exact returned values depend on the model prediction.

4.16 Reminder System Implementation

The reminder system is one of the more detailed local-device components of `glucosense_ai`.

Its main components include:

Notification Service

Responsible for:

- Notification initialization

- Permissions
- Notification channels
- Immediate notification display
- Scheduling
- Cancellation
- Diagnostic checks

Reminder Scheduler

Responsible for:

- Calculating next reminder time
- Managing reminder slots
- Smart suppression
- Notification payloads
- Canceling completed reminders

Notification History

Delivered notifications are recorded using Hive and can later be viewed inside the application.

The system also handles required Android notification and scheduling permissions and contains receivers for scheduled notifications and reboot rescheduling.

4.17 Error Handling

glucosense_ai uses explicit state handling instead of assuming that every network or storage operation will succeed.

The architecture expects BLoCs and Cubits to represent states such as:

- Loading

- Success
- Empty
- Error

and asynchronous production operations are expected to include error handling.

This is particularly important for:

- Failed Firebase requests
- Missing network connection
- Gemini API failures
- Flask API failures
- Invalid prediction input
- Notification permission problems
- Storage failures

A clear failure state allows the user interface to display useful feedback instead of freezing or silently losing information.

4.18 Security Considerations

The system contains medical-profile and glucose-related data and therefore requires careful handling.

Important security-related elements include:

- Firebase Authentication for account access
- Secure handling of user sessions
- Secure storage for sensitive local values
- API communication over network connections
- Separation between application layers
- Restricted access to user-specific cloud data

- Protected API keys in production environments

The project keeps configuration and secret-related functionality inside the core configuration infrastructure rather than scattering secret values throughout feature code.

The Gemini API key should also not be hard-coded into a public repository.

4.19 Chapter Summary

This chapter presented the complete design and implementation structure of **glucosense_ai**.

The system uses a Flutter mobile application with a feature-first clean architecture. Each feature is divided into presentation, domain, and data layers, while BLoC/Cubit is used to manage state and repositories separate the user interface from Firebase, Supabase, APIs, and other external services.

The system architecture connects the Flutter application with Firebase for authentication and persistent data, local device services for reminders and notification history, Google Gemini for meal-image analysis, and a Flask API for machine-learning inference.

The machine-learning component uses a three-class prediction approach with **Normal, Prediabetic, and Diabetic** as its output categories. Logistic Regression, Random Forest, and XGBoost are compared, with XGBoost selected as the final model. The trained model and StandardScaler are exported separately and used through the Flask prediction API. SHAP and LIME are also included to support model explainability.

The chapter also described the system's use cases, main classes, glucose-logging sequence, meal-analysis sequence, diabetes-risk prediction sequence, reminder flow, trends/report flow, machine-learning pipeline, training workflow, API endpoints, navigation, error handling, and security considerations. The next chapter will present the **testing, results, and discussion ,testing strategy, and detailed test cases** for glucosense_ai.

CHAPTER 05

TESTING, RESULTS, AND DISCUSSION

5.0 Testing Methodologies

5.0.1 Black-Box Testing

Black-box testing is used to evaluate application functionality from the user's perspective without considering the internal source-code implementation.

Inputs are provided through the application's screens and outputs are compared with expected behavior.

This type of testing is suitable for verifying operations such as signup, login, profile setup, glucose logging, meal analysis, risk prediction, viewing trends, configuring reminders, and updating profile information.

5.0.2 Integration Testing

Integration testing is performed to confirm that the different components of `glucosense_ai` communicate correctly.

Important integrations include:

Flutter application with Firebase Authentication, Flutter application with cloud data storage, Flutter application with Google Gemini API, Flutter application with Flask Prediction API, Flask API with StandardScaler, Flask API with XGBoost model, and the application with operating-system notification services.

The purpose of integration testing is to ensure that information passes correctly between all connected components.

5.0.3 API Testing

The Flask Prediction API shall be tested independently from the Flutter application.

The `/health` endpoint shall be tested to verify that the prediction service is running and that model information can be returned.

The `/predict` endpoint shall be tested using complete and incomplete JSON requests.

For a valid request containing all twelve required features, the API should return a prediction response containing the risk category, confidence, probability information, and advice.

For a request with missing required information, the API should return an appropriate validation error instead of attempting prediction.

5.0.4 Machine Learning Model Testing

The machine-learning pipeline compares **Logistic Regression, Random Forest, and XGBoost**.

Model evaluation is based on classification metrics including accuracy, weighted F1-score, per-class precision, per-class recall, classification report, and confusion matrix.

The final XGBoost model was selected according to weighted F1-score and overall classification performance.

However, the exact numerical model metrics are not available from the current repository snapshot. Therefore, numerical accuracy, precision, recall, or F1 values should **not be fabricated in this thesis**. These values should be inserted after a reproducible execution of the final notebook with the original dataset.

5.0.5 Notification Testing

The notification system shall be tested for both immediate and scheduled delivery.

The application contains diagnostic mechanisms for differentiating between general notification permission problems and scheduled-alarm problems. The implementation provides an immediate notification test and a scheduled notification test.

Testing should also verify reminder deeplinking, configured reminder times, smart suppression, and notification restoration following supported device restarts.

5.0.6 Error Handling Testing

The application shall be tested using invalid inputs, unavailable network connections, failed external service requests, missing API information, and incomplete prediction requests.

The application architecture defines explicit loading, success, empty, and error states rather than relying on uncontrolled null states.

This ensures that failures can be handled appropriately without unexpected application termination.

5.1 Test Cases

The following test cases represent the principal functional tests for the proposed glucosense_ai system.

Table 5-1: User Login Test Case

Field	Description
Test Case ID	TC-001
Description	Verify that a registered user can log into the system using valid credentials.
Pre-Conditions	User account exists and Firebase Authentication is available.
Steps	Open Login Screen → Enter valid email and password → Press Login button.
Expected Result	User is successfully authenticated and redirected to the appropriate application screen.
Actual Result	To be recorded from final system test.

Table 5-2: Invalid Login Test Case

Field	Description
Test Case ID	TC-002
Description	Verify system behavior when invalid login credentials are entered.
Pre-Conditions	Login screen is available.
Steps	Enter invalid email/password → Press Login.
Expected Result	Authentication is rejected and an appropriate error message is displayed.
Actual Result	To be recorded from final system test.

Table 5-3: Medical Profile Setup Test Case

Field	Description
Test Case ID	TC-003
Description	Verify that the user can complete the medical profile.
Pre-Conditions	User is authenticated and profile setup is incomplete.
Steps	Open Profile Setup → Enter required personal and medical information → Save profile.
Expected Result	Profile information is successfully stored and the user is allowed to continue to the application.

Field	Description
Actual Result	To be recorded from final system test.

Table 5-4: Log Glucose Reading Test Case

Field	Description
Test Case ID	TC-004
Description	Verify that a user can record a blood-glucose reading.
Pre-Conditions	User is logged in.
Steps	Open Log Reading → Select reading type → Enter glucose value → Select required meal information → Save.
Expected Result	The glucose reading is successfully stored and becomes available in history and analysis.
Actual Result	To be recorded from final system test.

Table 5-5: Meal Image Analysis Test Case

Field	Description
Test Case ID	TC-005
Description	Verify AI-based analysis of a meal image.
Pre-Conditions	User is logged in, internet connection is available, and Gemini service is configured.
Steps	Open Meal Predictor → Capture/select meal image → Submit image for analysis.
Expected Result	Meal analysis is returned and nutritional/glycaemic information is displayed to the user.
Actual Result	To be recorded from final system test.

Table 5-6: Save Analysed Meal Test Case

Field	Description
Test Case ID	TC-006
Description	Verify that an analysed meal can be saved.
Pre-Conditions	A meal has been successfully analysed.
Steps	Complete meal analysis → Select Save option → Open saved meals.
Expected Result	Analysed meal is stored and can subsequently be reopened.
Actual Result	To be recorded from final system test.

Table 5-7: Diabetes Risk Prediction Test Case

Field	Description
Test Case ID	TC-007
Description	Verify that the system can generate a diabetes-risk prediction.
Pre-Conditions	Required profile and glucose information is available and Flask Prediction API is running.
Steps	Open Risk Prediction → Prepare required feature information → Submit prediction request.
Expected Result	The system returns Normal, Prediabetic, or Diabetic together with confidence, class probabilities, and advice.
Actual Result	To be recorded from final system test.

Table 5-8: Missing Prediction Field Test Case

Field	Description
Test Case ID	TC-008
Description	Verify API behavior when a required prediction feature is missing.
Pre-Conditions	Flask Prediction API is running.
Steps	Send /predict request without one or more required fields.
Expected Result	Prediction is rejected and an appropriate missing-field error is returned.
Actual Result	To be recorded from final API test.

Table 5-9: View Glucose Trends Test Case

Field	Description
Test Case ID	TC-009
Description	Verify that historical glucose trends can be displayed.
Pre-Conditions	User has stored glucose readings.
Steps	Open Trends → Select 7-day, 30-day, or 90-day period.
Expected Result	Relevant charts and glucose statistics are displayed for the selected period.
Actual Result	To be recorded from final system test.

Table 5-10: Generate Report Test Case

Field	Description
Test Case ID	TC-010
Description	Verify generation of a date-range report.
Pre-Conditions	User has stored reading and/or meal information.
Steps	Open Reports → Select date range → Generate report.
Expected Result	A report containing the relevant information for the selected date range is generated.
Actual Result	To be recorded from final system test.

Table 5-11: Schedule Reminder Test Case

Field	Description
Test Case ID	TC-011
Description	Verify that the user can configure and schedule a glucose reminder.
Pre-Conditions	User is logged in and notification permission is available.
Steps	Open Reminder Settings → Enable reminder → Select type, time and active days → Save.
Expected Result	Reminder is successfully scheduled for the configured time.
Actual Result	To be recorded from final system test.

Table 5-12: Reminder Deeplink Test Case

Field	Description
Test Case ID	TC-012
Description	Verify that tapping a reminder opens the correct logging flow.
Pre-Conditions	A glucose reminder has been delivered.
Steps	Tap delivered reminder notification.
Expected Result	Log Reading screen opens with relevant reading type and meal information pre-filled.
Actual Result	To be recorded from final system test.

Table 5-13: Smart Reminder Suppression Test Case

Field	Description
Test Case ID	TC-013
Description	Verify that unnecessary reminders are suppressed after the associated reading has already been recorded.
Pre-Conditions	Reminder is configured and matching reading exists within the suppression interval.
Steps	Record matching glucose reading → Allow scheduler to calculate upcoming reminder.
Expected Result	Matching reminder is skipped rather than unnecessarily notifying the user.
Actual Result	To be recorded from final system test.

Table 5-14: Update User Profile Test Case

Field	Description
Test Case ID	TC-014
Description	Verify that users can modify their personal or medical information.
Pre-Conditions	User is authenticated and profile exists.
Steps	Open Profile → Edit information → Save changes.
Expected Result	Updated profile information is successfully saved and displayed.
Actual Result	To be recorded from final system test.

Table 5-15: Flask API Health Test Case

Field	Description
Test Case ID	TC-015
Description	Verify availability of the prediction service.
Pre-Conditions	Flask application is running.
Steps	Send a GET request to /health.
Expected Result	API returns service status together with model/feature information.
Actual Result	To be recorded from final API test.

5.2 Model Selection

The main model-selection criterion used in the notebook is **weighted F1-score**.

Weighted F1 is useful in multiclass classification because it considers both precision and recall while weighting each class according to the number of samples belonging to it.

After comparing Logistic Regression, Random Forest, and XGBoost, **XGBoost was selected as the final model** based on weighted F1-score and overall classification performance.

The final model is exported as:

XGBoost_model.pkl

At present, the committed project snapshot does not contain the reproducible numeric output from the final training run. Therefore, exact accuracy or F1 values should not be invented or reported until the dataset and training environment are available again.

5.3 Model Evaluation

The notebook is designed to evaluate models using several measures.

These include:

- Accuracy
- Weighted F1-score
- Precision
- Recall
- Classification report
- Confusion matrix

Accuracy represents the proportion of all predictions that were correctly classified.

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Precision represents the proportion of predictions for a class that are actually correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall represents the proportion of actual samples belonging to a class that are correctly identified.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1-score combines precision and recall.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

For this project, class-specific results are especially useful because the model predicts three categories rather than a simple yes/no outcome.

5.4 Confusion Matrix

A confusion matrix provides a direct comparison between actual and predicted classes.

For the three-class model, the matrix distinguishes predictions involving:

- Normal
- Prediabetic
- Diabetic

The diagonal cells represent correct predictions, while non-diagonal cells represent misclassification.

A confusion matrix is particularly helpful because overall accuracy alone does not reveal which risk category is being confused with another.

5.5 Model Explainability

The project includes two explainability methods:

- SHAP
- LIME

These tools help provide some understanding of how the XGBoost model reaches its predictions.

5.5.1 SHAP Analysis

SHAP is used mainly for overall feature-impact analysis.

A SHAP summary plot can indicate which model inputs influence predictions most strongly across the dataset.

For example, the analysis may show whether variables such as HbA1c, glucose values, BMI, age, or family history have greater influence on the model than others.

The actual ranking should only be reported from the generated SHAP results and should not be assumed in advance.

5.5.2 LIME Analysis

LIME is used to explain an individual prediction.

Instead of explaining the model globally, LIME helps show which features contributed to the prediction for one particular patient.

For example, if a patient receives a Prediabetic result, LIME can be used during model analysis to investigate which input values moved the prediction toward that category.

CHAPTER 06

USER INTERFACE AND USER EXPERIENCE

6.0 Proposed System Interface & User Experience

The user interface of **glucosense_ai** is designed to provide a simple and structured experience for users who want to monitor glucose readings, analyse meals, review diabetes-risk information, manage reminders, and maintain their medical profile. The application is developed using **Flutter and Dart** with Material-based user-interface components and follows a feature-first architecture in which each major application function is implemented as an independent feature.

The application provides a five-tab main navigation structure containing **Home, Log Reading, Meal Predictor, Risk Prediction, and Profile**. Additional functions such as trends, reports, reminders, saved meals, and notification history are accessed through their corresponding screens.

The main interfaces of the system are described below.

6.1 Splash Screen

The Splash Screen is the first interface displayed when the application is launched. It presents the identity of **glucosense_ai** while the application performs its initial startup operations.

During this stage, the system determines whether the user already has an authenticated session. Based on the authentication status, the user is redirected either to the onboarding/login process or toward the profile setup and main application interface.

The documented application flow starts from the Splash Screen and then determines whether the user is already signed in.

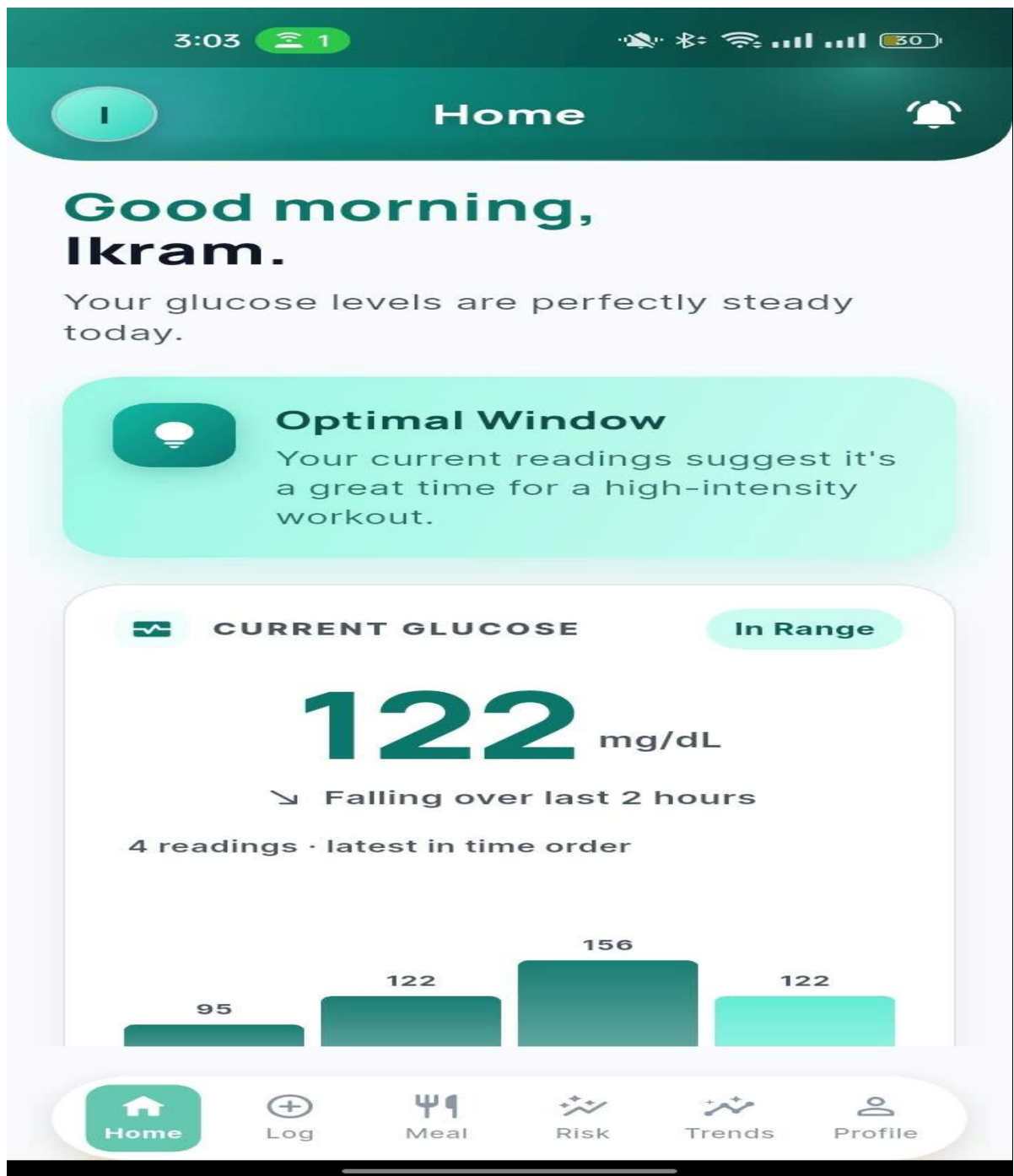


Figure 6.1: Splash Screen

6.2 Onboarding Screen

The onboarding interface introduces the user to the main purpose and functionality of glucosense_ai.

The application contains two onboarding screens that provide a first-launch introduction before the user proceeds to account authentication.

The onboarding process is intended to help new users understand that the application can be used for glucose monitoring, meal analysis, diabetes-risk prediction, reminders, and health-related tracking.

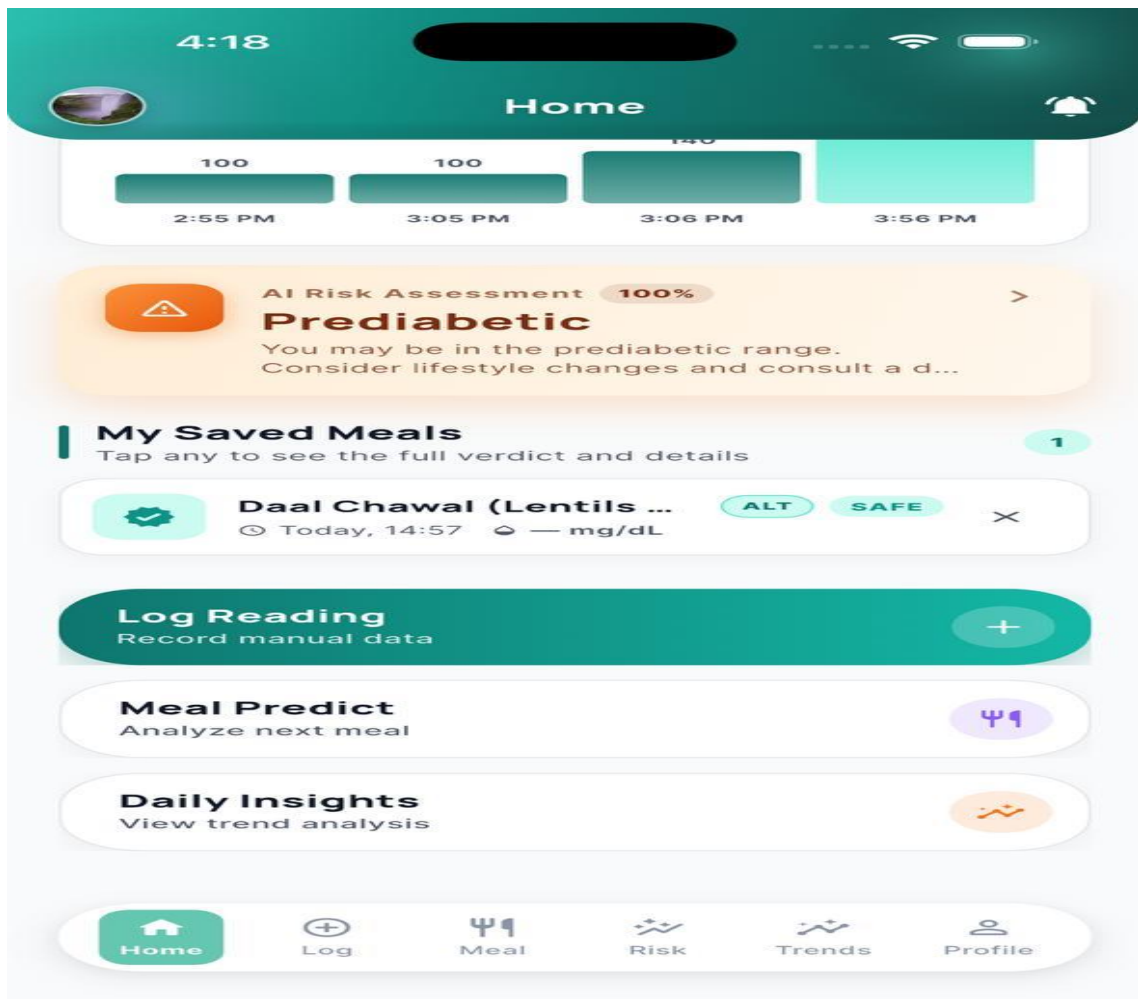


Figure 6.2: Onboarding Screen

6.3 Login Screen

The Login Screen allows registered users to access the application through their email address and password.

After the user enters valid credentials, Firebase Authentication verifies the account information. Once authentication is successful, the user is redirected to the appropriate application screen.

The application supports email/password login, account registration, and password-reset functionality through Firebase Authentication.

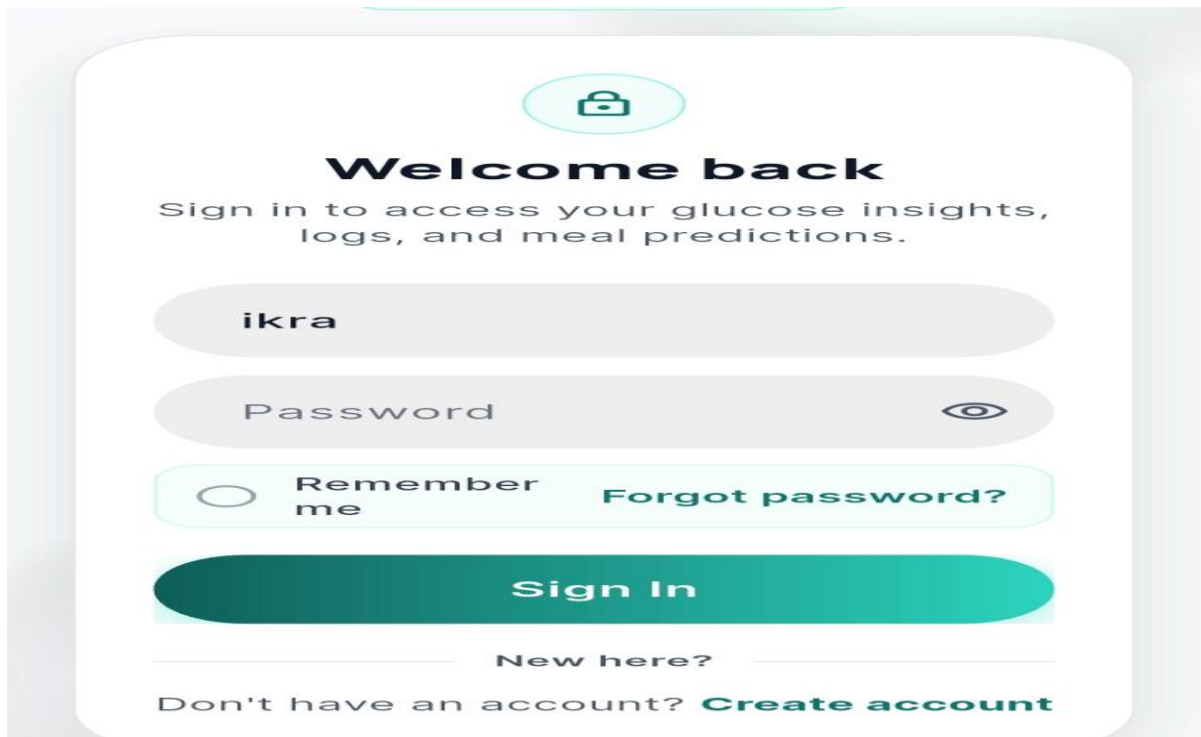


Figure 6.3: Login Screen

6.4 Signup Screen

The Signup Screen allows new users to create an account.

The user provides the required account information, including an email address and password. The account is then registered through Firebase Authentication.

After successful account creation, the user can proceed to the medical-profile setup process before using the risk-prediction features of the application.

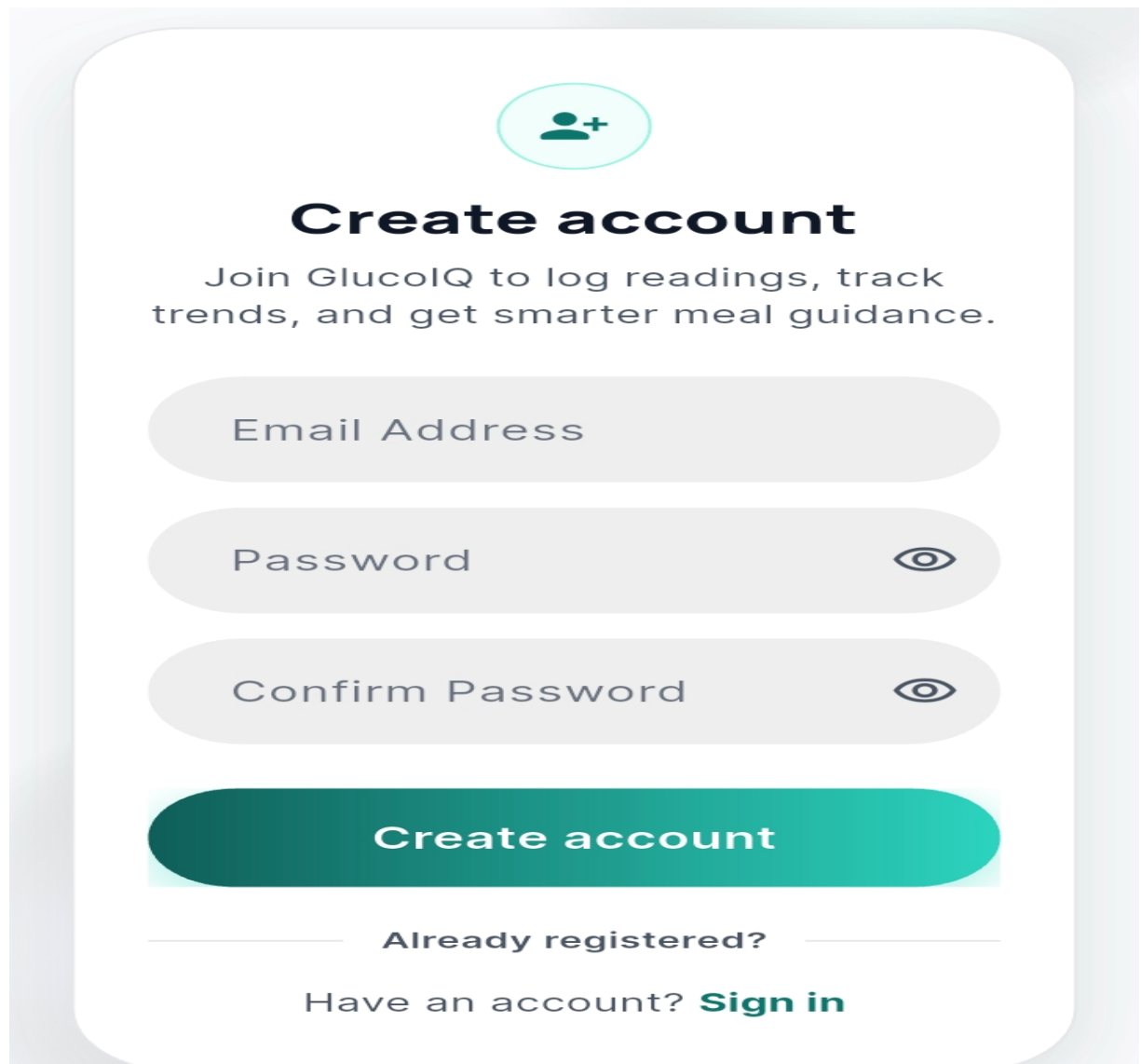
The image shows a mobile app signup screen. At the top, there is a teal circular icon with a white person silhouette and a plus sign. Below this is the title "Create account" in bold black text. Under the title is a subtitle: "Join GlucoIQ to log readings, track trends, and get smarter meal guidance." There are three input fields: "Email Address", "Password", and "Confirm Password". Each field is a light gray rounded rectangle. To the right of the "Password" and "Confirm Password" fields is a teal eye icon. Below the input fields is a large teal button with the text "Create account" in white. At the bottom, there is a link "Already registered?" with a teal underline. Below that is the text "Have an account? Sign in" where "Sign in" is in teal.

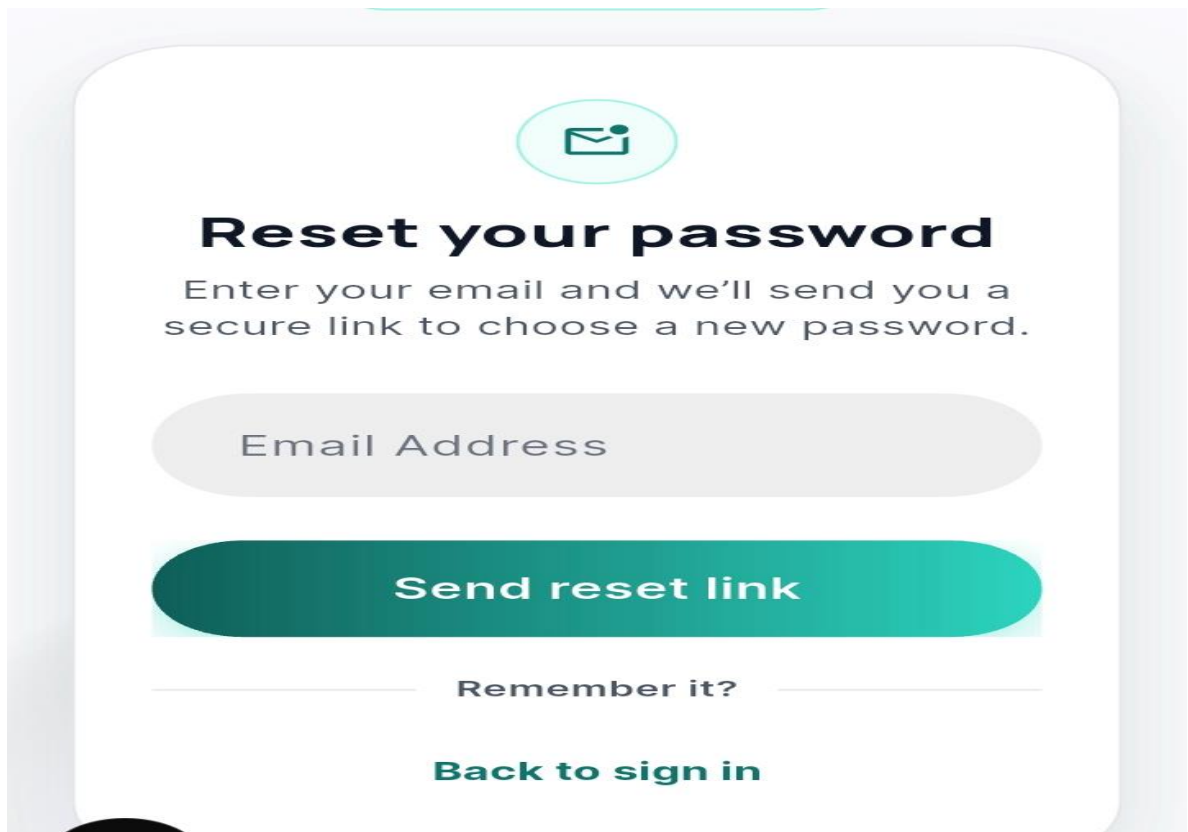
Figure 6.4: Signup Screen

6.5 Forgot Password Screen

The Forgot Password Screen provides account-recovery functionality.

A registered user can request a password-reset procedure using the email address associated with the account. This functionality reduces the possibility of users losing permanent access to their application profile because of forgotten credentials.

The authentication module explicitly includes login, signup, and forgot-password screens.

The image shows a mobile application screen for resetting a password. At the top, there is a teal circular icon containing a white envelope symbol. Below this icon, the text "Reset your password" is displayed in a bold, black font. Underneath, a subtitle reads "Enter your email and we'll send you a secure link to choose a new password." in a smaller, gray font. A light gray rounded rectangular input field is labeled "Email Address" in a gray font. Below the input field is a prominent teal rounded rectangular button with the text "Send reset link" in white. At the bottom, there is a link that says "Remember it?" with a horizontal line on either side, and a teal link that says "Back to sign in".



Reset your password

Enter your email and we'll send you a secure link to choose a new password.

Email Address

Send reset link

Remember it?

Back to sign in

Figure 6.5: Forgot Password Screen

6.6 Medical Profile Setup Screen

The Medical Profile Setup Screen collects information required for user profiling and diabetes-risk prediction.

The profile contains medical and personal information such as:

- Age
- Gender
- BMI
- Family history
- Hypertension
- Heart disease
- Smoking history
- Activity level
- HbA1c information, where available

The profile information is subsequently used together with glucose information by the diabetes-risk prediction component.

The application documentation confirms that the Profile Setup Screen collects medical information such as age, sex, BMI, and family history for use in risk prediction.

 Target Range

Current Goal

85 - 160 mg/dL

Edit →



Your details



Age

22



Biological sex

Male



Height

179 cm



Weight

79.0 kg



BMI

24.7



Home



Log



Meal



Risk



Trends



Profile

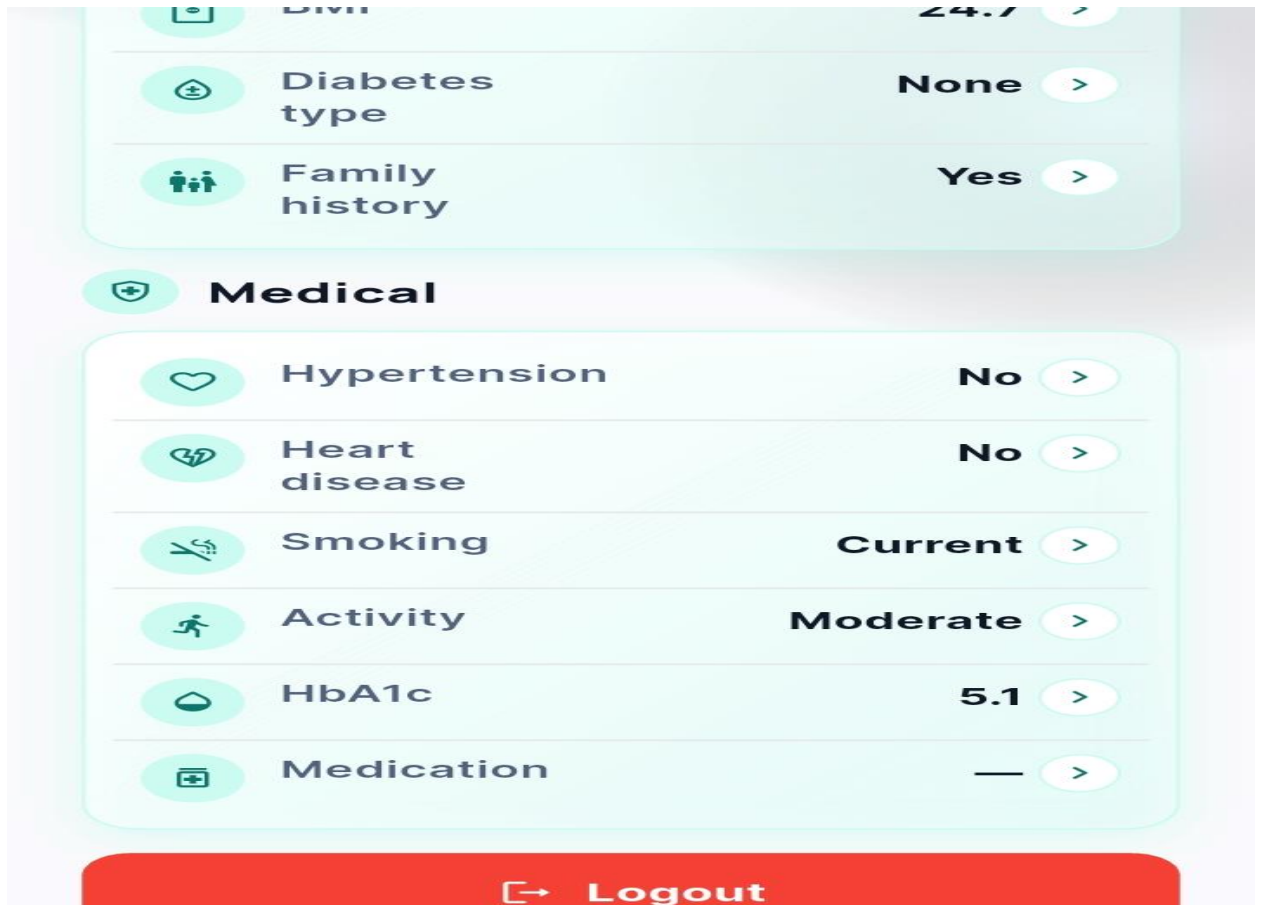


Figure 6.6: Medical Profile Setup Screen

6.7 Home Dashboard

The Home Dashboard acts as the main entry point after the user successfully accesses the application.

It provides a summary of important information and quick access to frequently used functions. The Home screen includes information such as the latest glucose reading, daily summary information, quick actions, and recent meal-analysis information.

The dashboard is intended to reduce the number of steps required to reach common actions such as recording a glucose reading or reviewing recent health information.

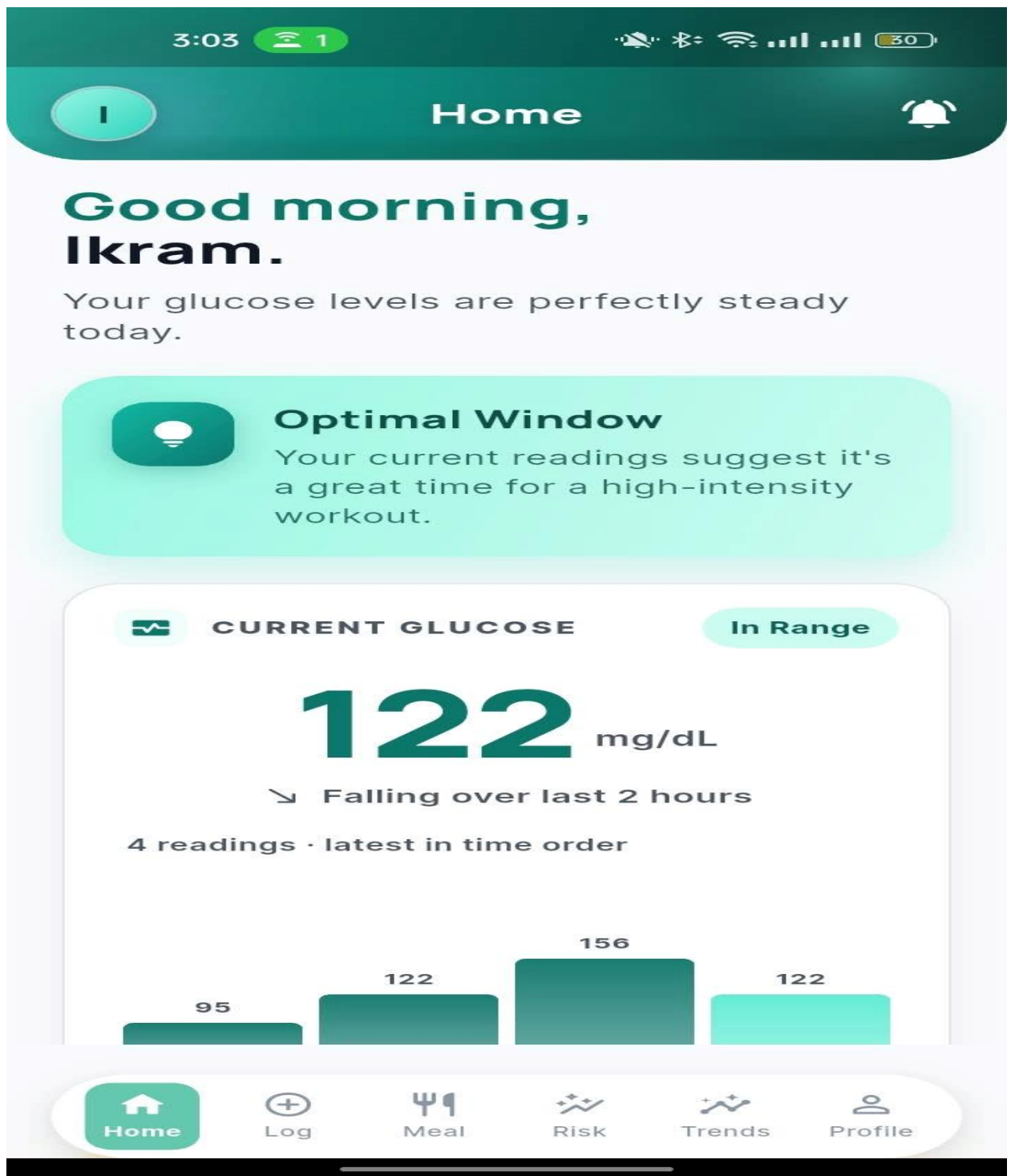


Figure 6.7: Home Dashboard

6.8 Glucose Logging Screen

The Glucose Logging Screen is one of the primary monitoring interfaces of the application.

Users can record different types of glucose measurements, including:

- Fasting glucose
- Pre-meal glucose
- Post-meal glucose

The user can associate the reading with the appropriate meal context, unit, and timestamp before saving the information.

The system maintains glucose-reading history so that previous readings can later be used in trends, reports, and diabetes-risk analysis. The application also supports opening the Log Reading Screen directly from reminder notifications with relevant information pre-filled.

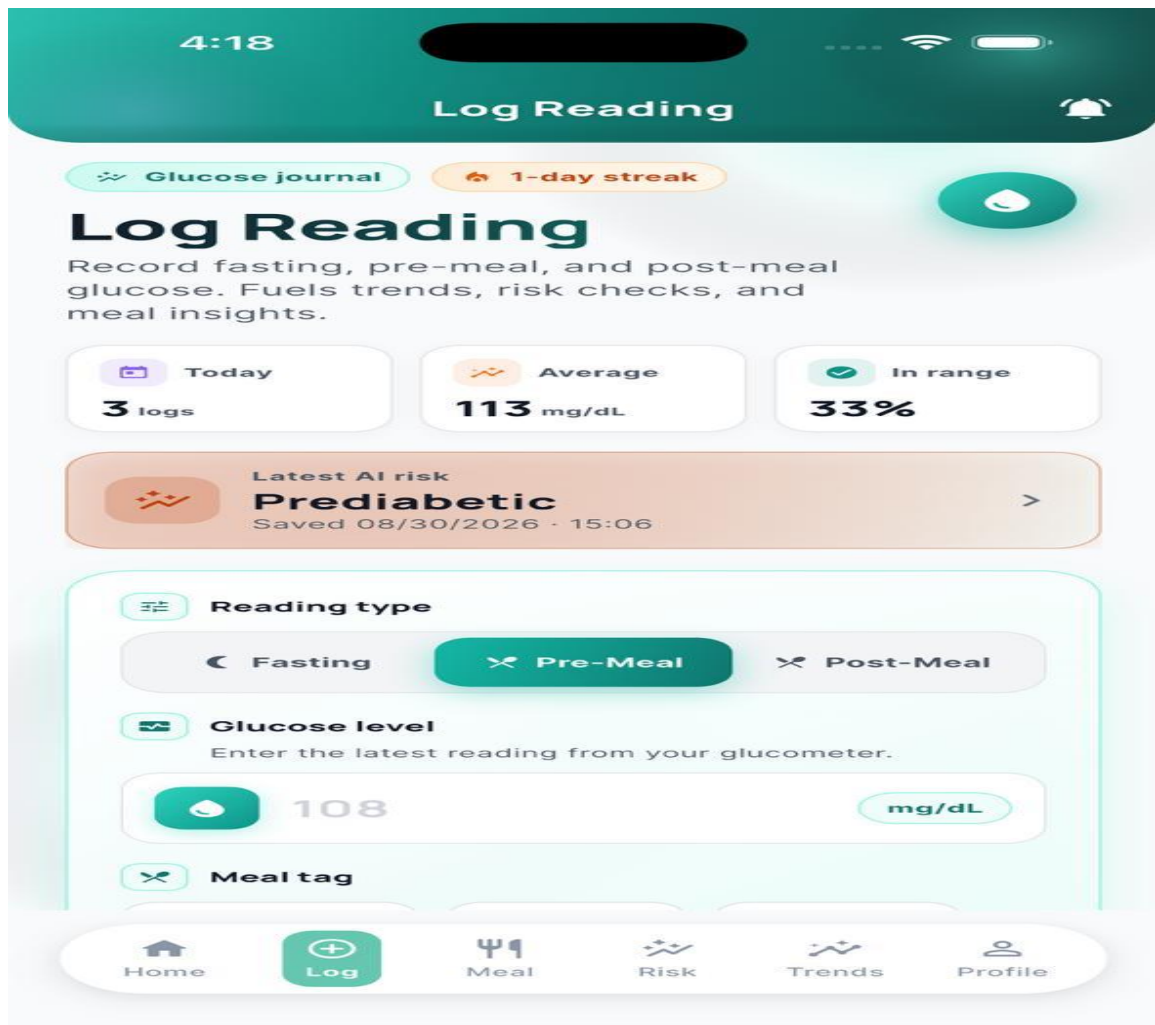


Figure 6.8: Glucose Logging Screen

6.9 Glucose History Screen

The glucose-history interface allows users to review previously stored blood-glucose measurements.

Historical records help users observe how their glucose values change over time and provide the data required by trend analysis and report-generation features.

Each stored reading can be associated with its measurement type and timestamp, helping the user distinguish between fasting, pre-meal, and post-meal measurements.

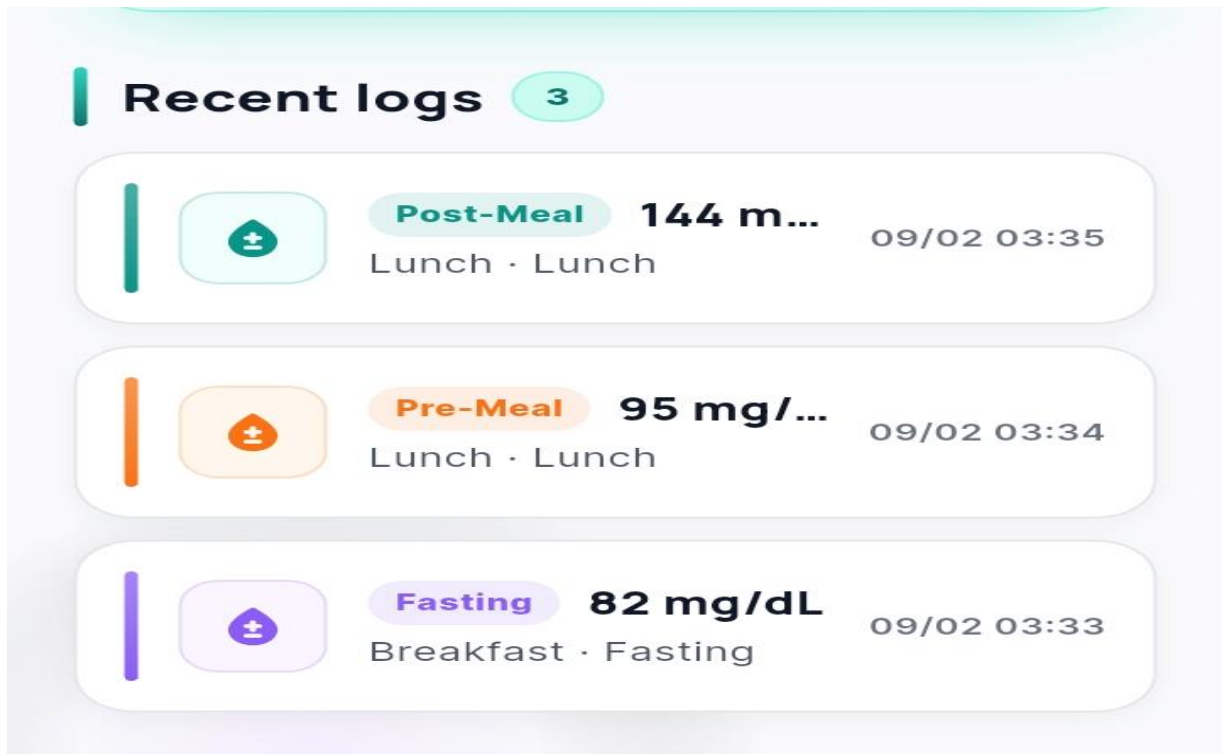


Figure 6.9: Glucose Reading History

6.10 Meal Predictor Screen

The Meal Predictor Screen provides the AI-assisted meal-analysis functionality of glucosense_ai.

The user can capture a photograph of a meal or select an existing image. The selected image is sent for analysis through the integrated Google Gemini service.

The system then provides information related to the meal, which may include:

- Estimated macronutrients
- Glycaemic impact
- Meal-related nutritional information

- More suitable diabetes-friendly alternatives

The application documentation specifically describes the meal-prediction feature as analysing a meal image using Gemini to provide macros, glycaemic load information, and safer alternatives.

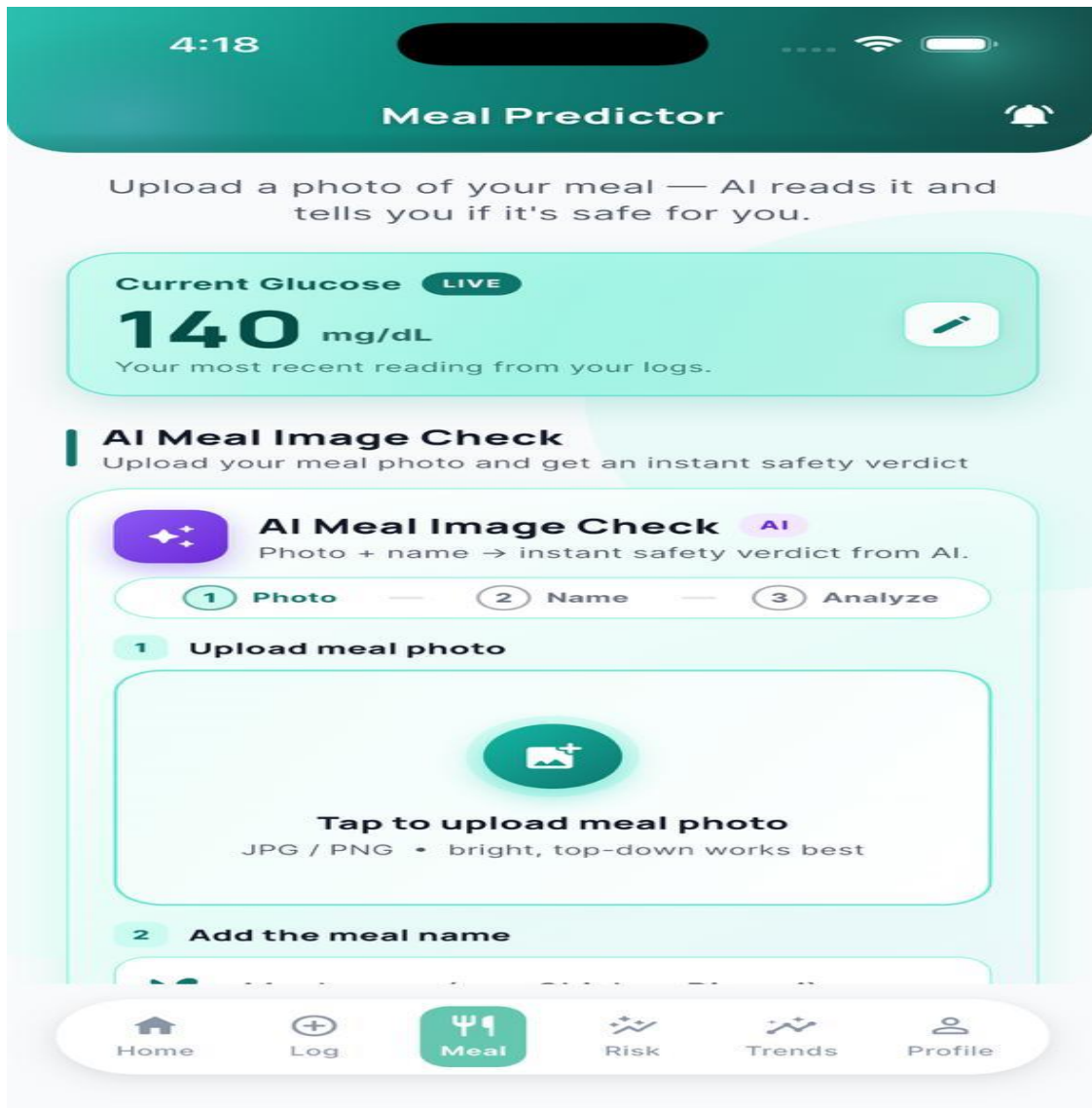


Figure 6.10: Meal Predictor Screen

6.11 Meal Analysis Result Screen

After the selected meal image has been processed, the analysis result is presented to the user.

This interface provides the AI-generated interpretation of the meal and is intended to help the user better understand the potential nutritional and glycaemic characteristics of the selected food.

The analysed meal can also be stored so that the user can access it later without repeating the complete image-analysis process.

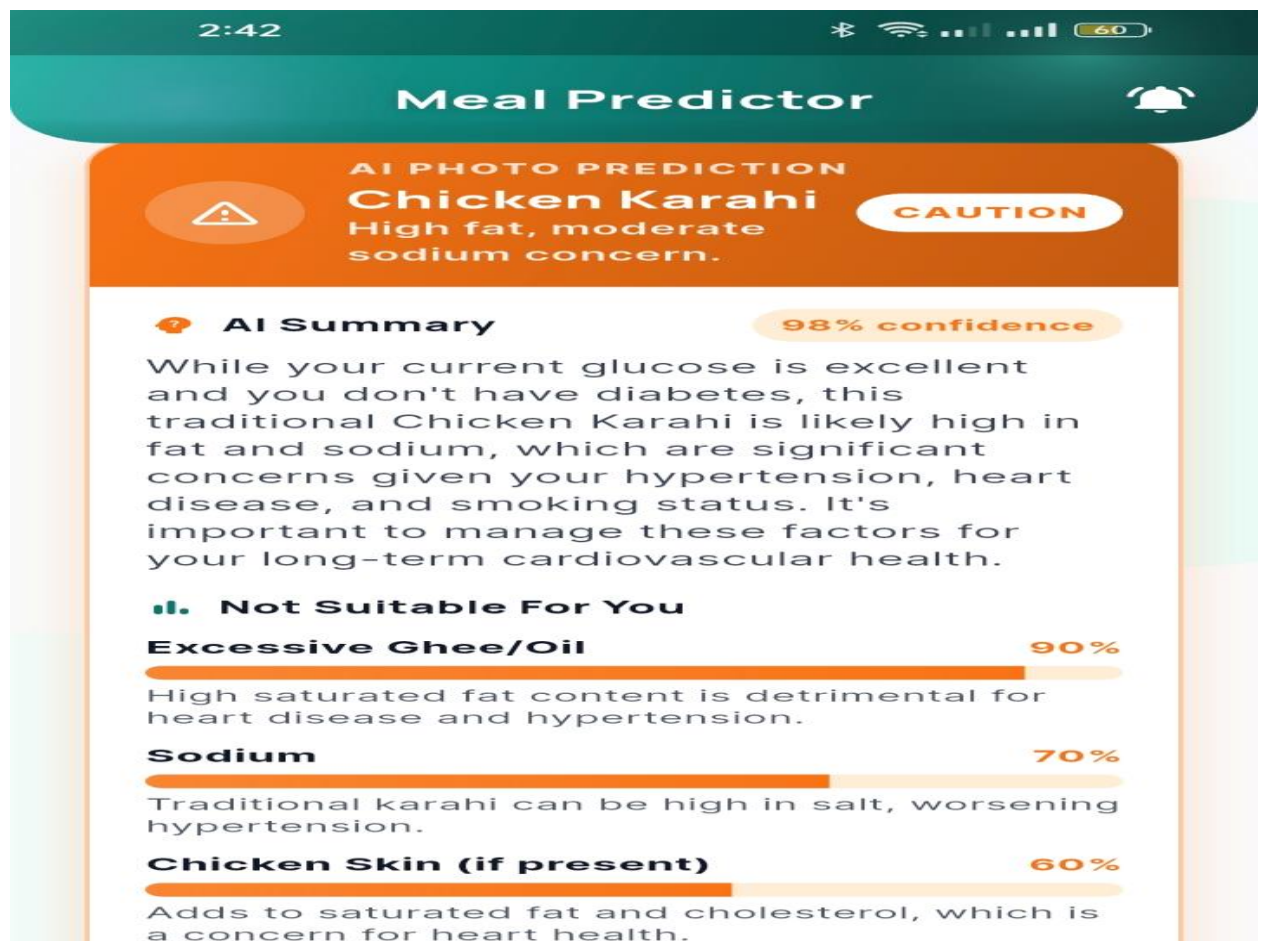




Figure 6.11: AI Meal Analysis Result

6.12 Diabetes Risk Prediction Screen

The Risk Prediction Screen provides one of the major intelligent features of glucosense_ai.

The prediction system uses information from the user's medical profile and relevant glucose readings. The data is sent to the Flask-based prediction API, where the saved preprocessing scaler and XGBoost model are used to generate the final classification.

The user can receive one of three risk categories:

- **Normal**
- **Prediabetic**
- **Diabetic**

The prediction response can include:

- Risk category
- Confidence
- Individual class probabilities
- Guidance text

The application documentation identifies risk prediction as a major system function based on medical profile information and glucose history.

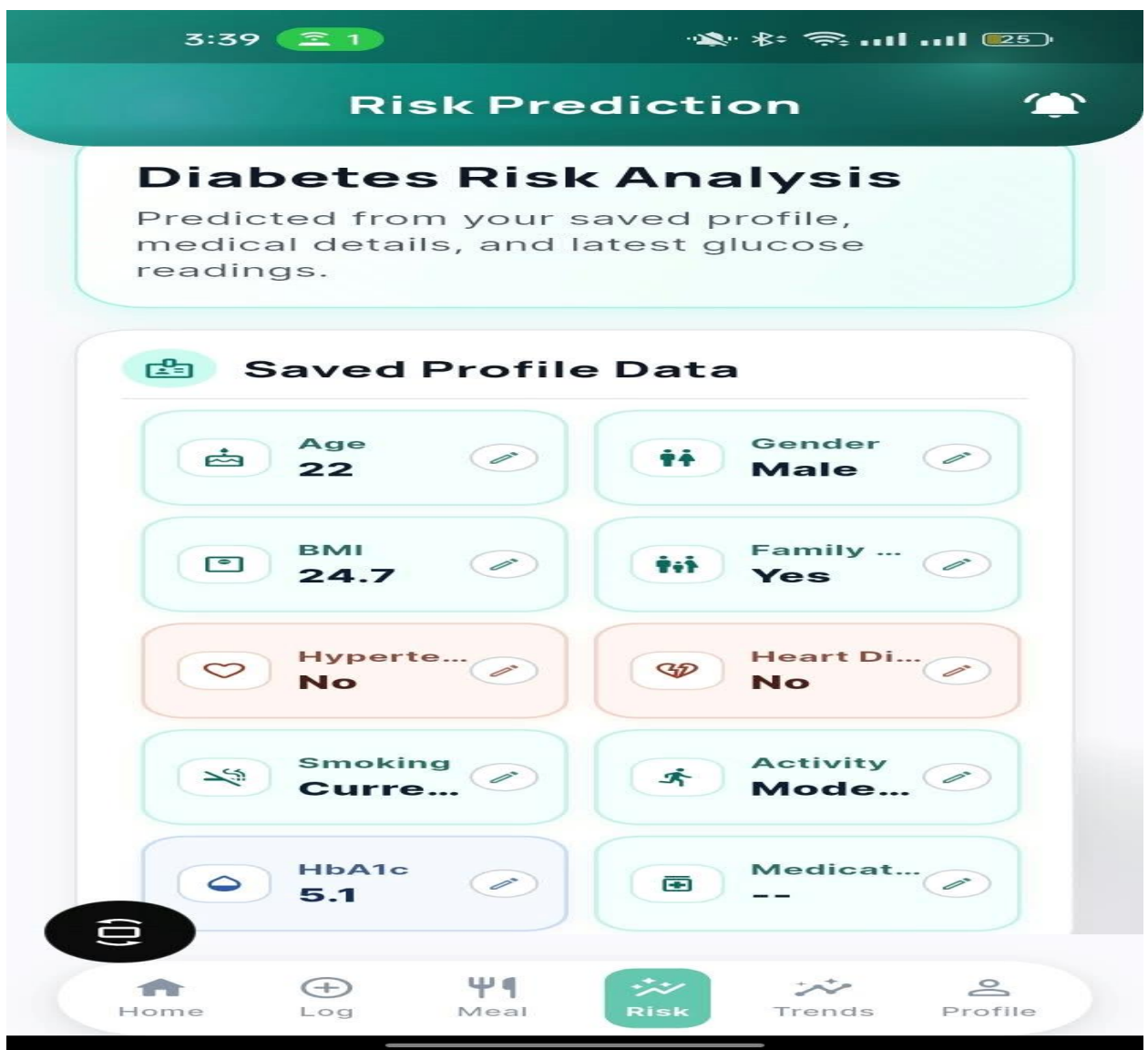


Figure 6.13: Diabetes Risk Prediction Screen

6.13 Risk Prediction Result Screen

The Prediction Result Screen presents the final diabetes-risk classification generated by the machine-learning model.

Instead of displaying only a numerical result, the system provides a clear categorical output so that the user can easily understand the result.

For example, the interface can display:

Risk Category: Prediabetic

along with the corresponding confidence and additional guidance returned by the prediction API.

The prediction information is intended to provide awareness and decision support rather than replace professional medical diagnosis.

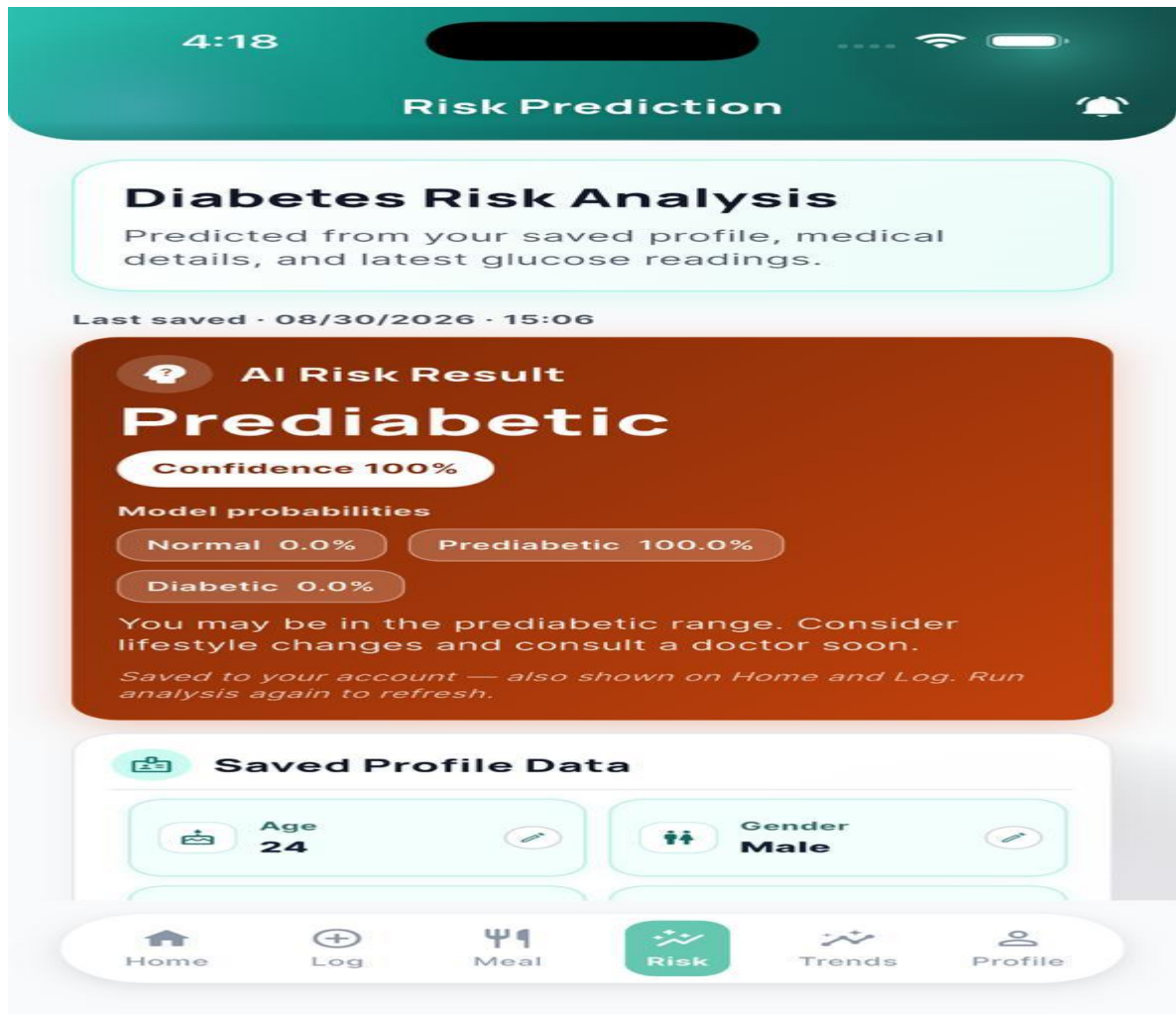


Figure 6.14: Diabetes Risk Prediction Result

6.14 Trends Screen

The Trends Screen provides graphical analysis of historical glucose information.

The user can analyse glucose data over periods such as:

- 7 days
- 30 days
- 90 days

The application can present measurements such as:

- Average glucose
- Time in Range
- Variance
- Glucose range
- Time-of-day patterns

These visualizations provide users with a more understandable view of their glucose patterns compared with reviewing individual measurements separately.

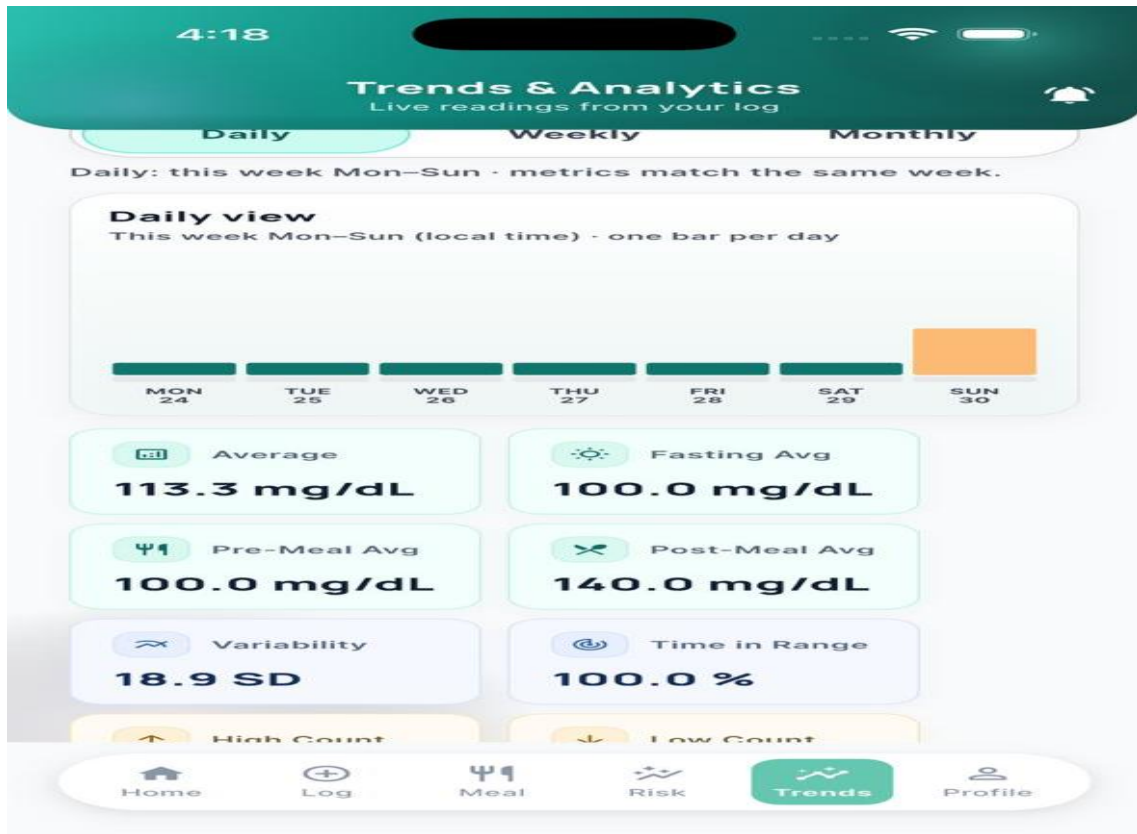


Figure 6.15: Glucose Trends Screen

6.15 Reports Screen

The Reports Screen allows users to select a particular date range and review summarized information from their recorded glucose readings and analysed meals.

The report feature is intended to provide a structured overview of user activity over a specified period.

The application documentation describes reports as shareable summaries containing readings and meals for a selected date range.

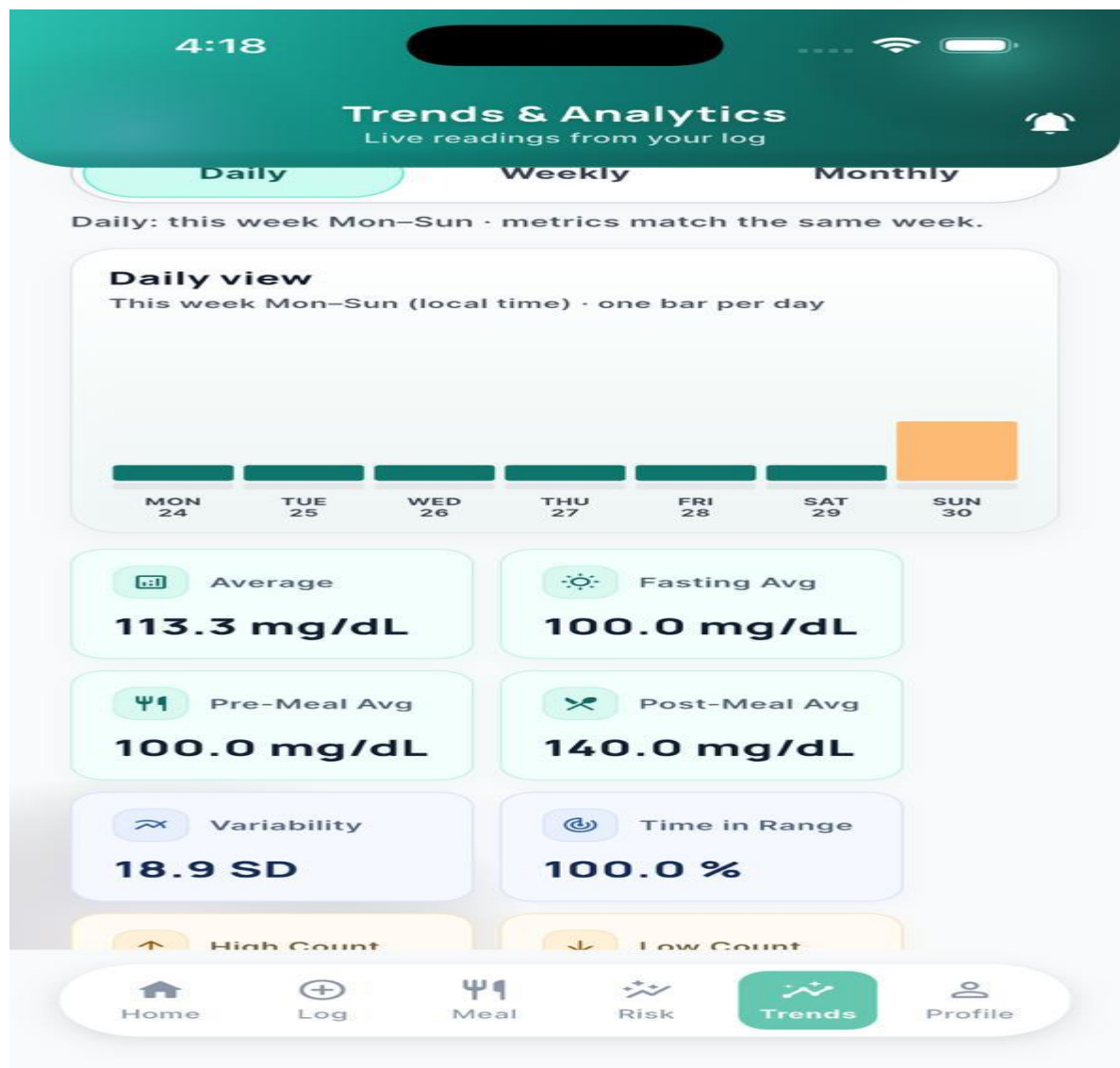


Figure 6.16: Reports Screen

6.16 Reminder Settings Screen

The Reminder Settings Screen enables users to configure monitoring reminders according to their daily routine.

The system supports reminders for activities such as:

- Fasting glucose monitoring
- Breakfast-related readings
- Lunch-related readings
- Dinner-related readings
- Snack-related measurements

Users can configure reminder times and active days.

The reminder system uses local operating-system scheduling rather than requiring the application to remain continuously open.

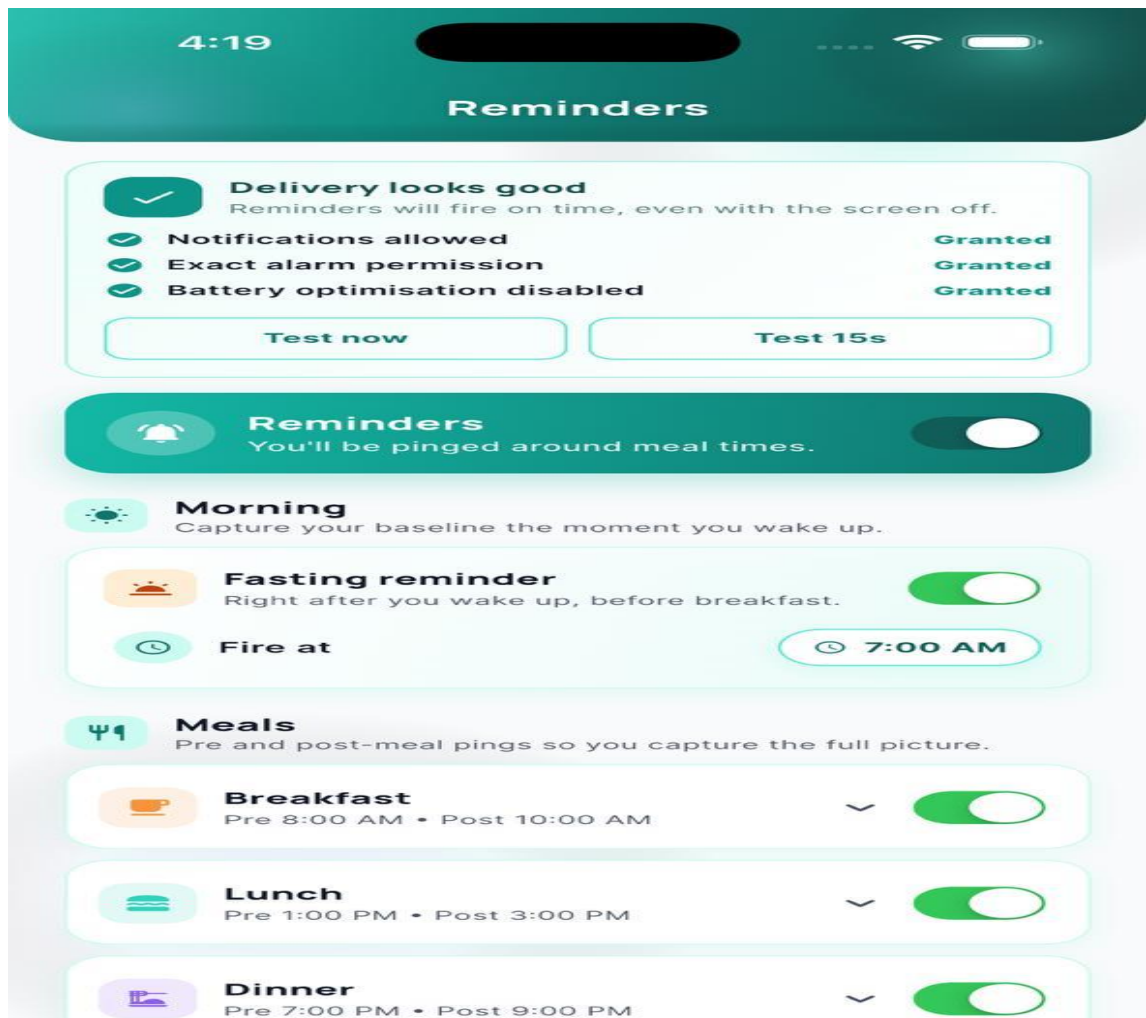


Figure 6.17: Reminder Settings Screen

6.17 Notification Delivery and Smart Reminder Interface

When a scheduled reminder becomes due, the mobile operating system displays the notification to the user.

If the user taps the notification, the system can open the glucose logging interface and automatically provide the relevant reading type or meal context.

The reminder implementation also provides smart suppression. If a matching glucose reading has already been logged within the configured interval, the corresponding reminder can be skipped during the next scheduling cycle.

This functionality prevents unnecessary notifications and makes reminders more context-aware.

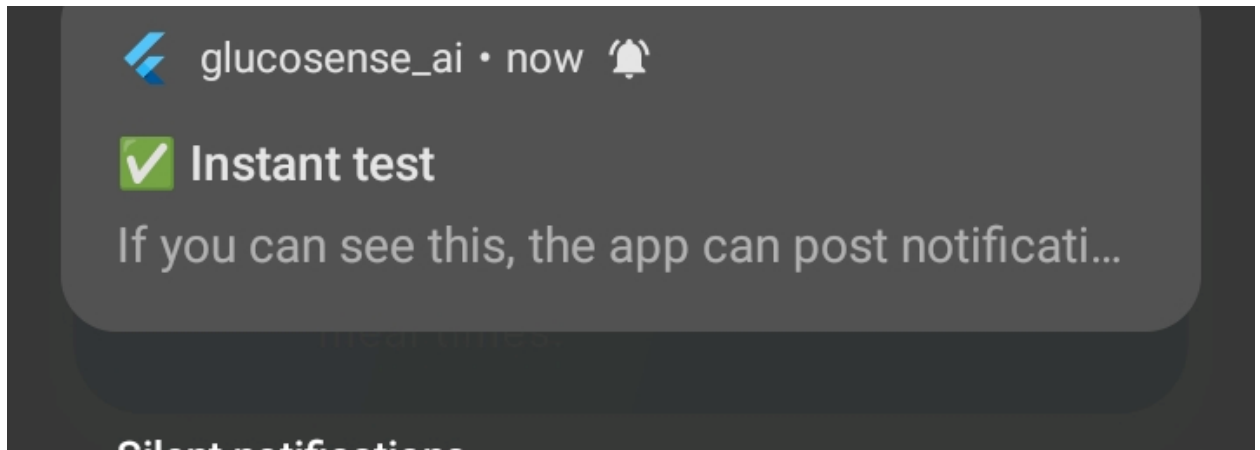


Figure 6.18: Smart Reminder Notification

6.18 Profile Screen

The Profile Screen provides access to the user's personal and medical information.

Users can review and modify information stored in their profile when required. The Profile Screen also acts as an access point for additional application functions such as:

- Reports
- Trends
- Reminders
- Notifications

The documented application structure identifies the Profile tab as the main entry point to these additional features.

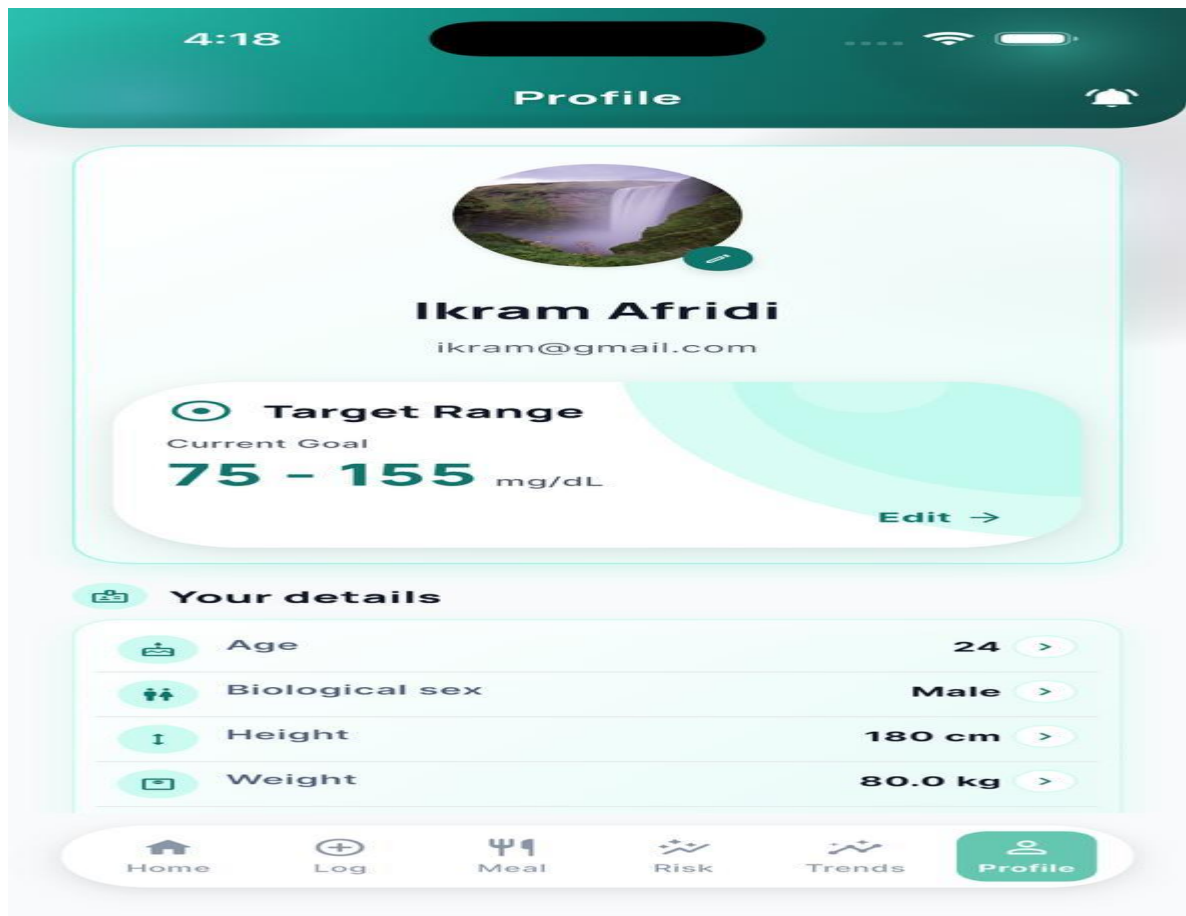


Figure 6.20: User Profile Screen

6.19 Edit Profile Screen

The Edit Profile Screen allows users to modify stored personal and medical information.

This functionality is important because health-related information may change over time. Updated profile information can subsequently be used by relevant application features, including diabetes-risk prediction.

The system therefore enables the user's stored profile to remain aligned with the most recent available information.

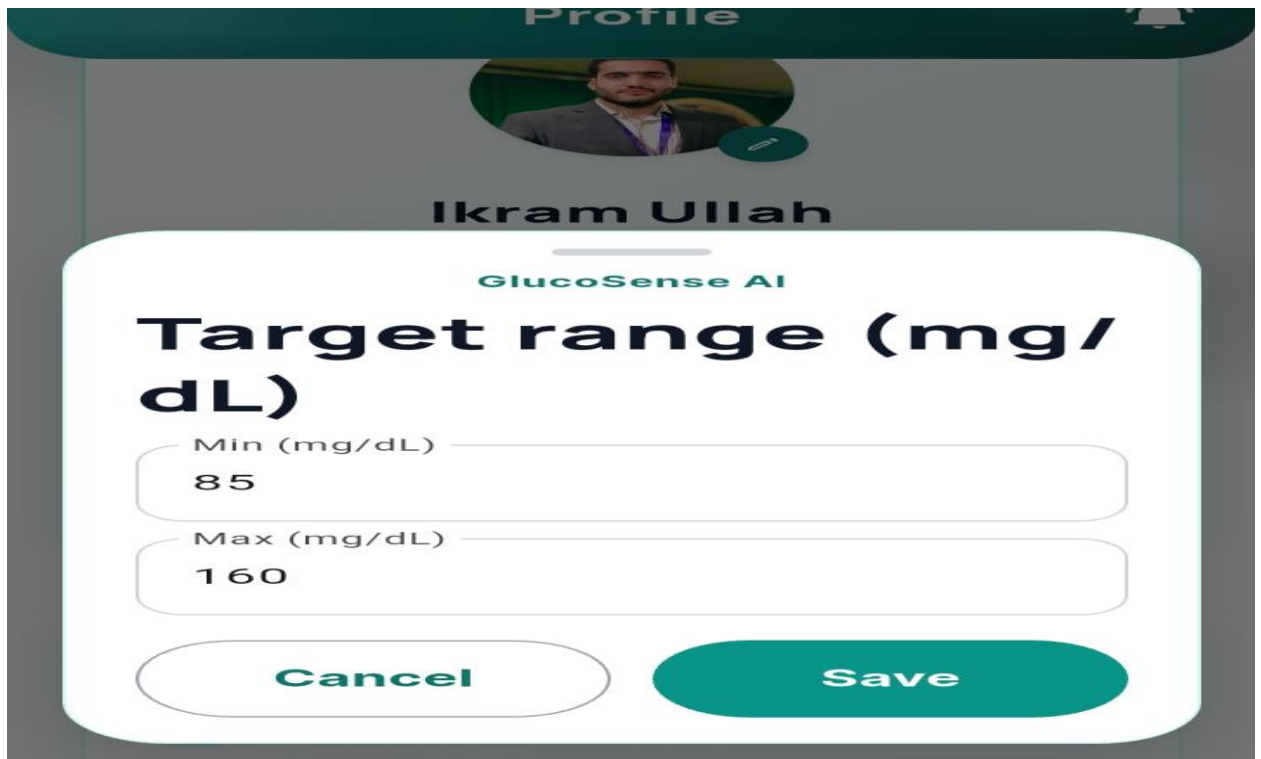


Figure 6.21: Edit Profile Screen

6.20 Application Navigation Flow

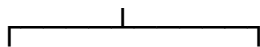
The application starts from the splash screen.

The basic navigation flow is:

Splash Screen



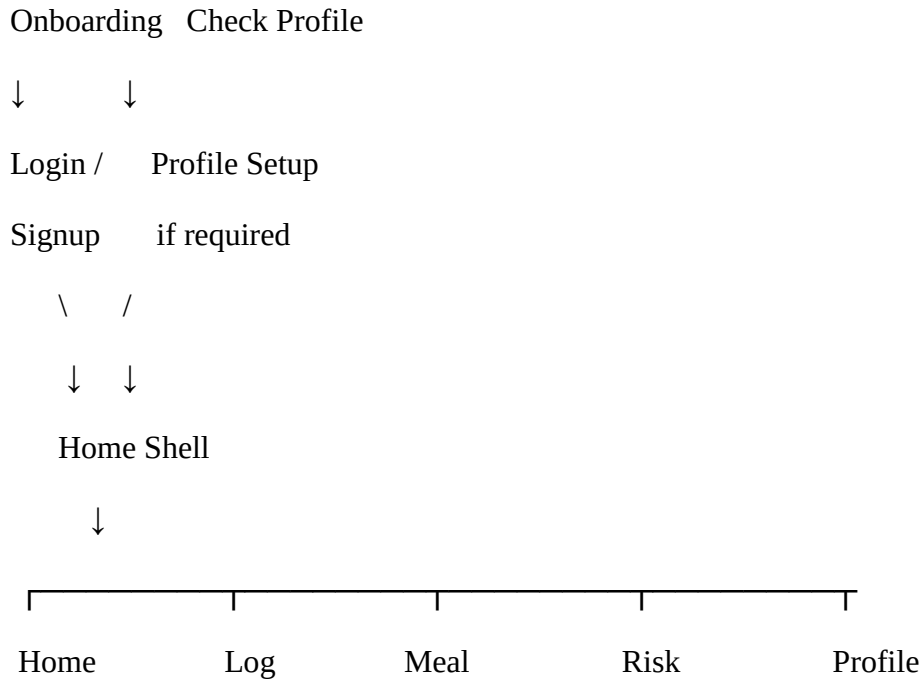
Check Authentication



No

Yes





The five main tabs of the home shell are:

1. Home
2. Log
3. Meal
4. Risk
5. Profile

The source project explicitly uses this five-tab application structure, with Profile also acting as the entry point for reports, trends, reminders, and notification features.

6.21 Overall Application Navigation

The user flow of glucosense_ai begins from the Splash Screen.

For a new user, the general navigation flow is:

Splash Screen → Onboarding → Login/Signup → Profile Setup → Home

For an existing authenticated user whose profile is already complete, the system can proceed directly from the Splash Screen to the main Home interface.

The main application then provides five principal navigation tabs:

Home → Log → Meal → Risk → Profile

Additional screens such as Trends, Reports, Reminders, Notifications, and Saved Meals can be accessed from their corresponding application features.

This navigation structure is designed to separate major tasks while maintaining easy access to frequently used functions.

6.22 User Experience Considerations

The user experience of glucosense_ai is designed around simplicity, clear navigation, and reduced interaction complexity.

The application uses BLoC/Cubit-based state management so that interfaces can respond explicitly to different system conditions such as:

- Loading
- Successful operation
- Empty data
- Error conditions

The architecture also separates user-interface components from Firebase, Gemini, and other external data sources, reducing direct coupling between the interface and backend implementation.

Another important user-experience feature is the reminder deeplink mechanism. Instead of simply notifying the user that a glucose measurement is required, tapping the notification can directly open the correct logging interface and pre-fill the relevant reading context.

The combination of structured navigation, AI-assisted meal analysis, visual glucose trends, smart reminders, and simplified risk output provides a unified experience in which users can perform the major diabetes-monitoring activities from a single application.

6.23 Conclusion

glucosense_ai provides an integrated mobile interface for glucose monitoring, meal analysis, diabetes-risk prediction, trend visualization, reporting, reminders, and profile management. The application's interface is organized around a five-tab navigation structure containing **Home, Log Reading, Meal Predictor, Risk Prediction, and Profile**, while additional health-management functions remain accessible through dedicated screens.

The combination of Flutter-based mobile interfaces, Firebase-backed authentication and storage, Google Gemini meal analysis, and Flask-based XGBoost prediction allows the user to interact with multiple intelligent services through a single application.

The proposed interface also emphasizes practical user experience through features such as saved meals, historical glucose information, graphical trends, scheduled reminders, notification history, and direct navigation from reminders to the glucose-logging screen. Overall, the developed user interface provides a structured and accessible way for users to monitor their glucose-related information and obtain AI-assisted health insights. Together with the machine-learning prediction system described in the previous chapters, the interface completes the proposed glucosense_ai solution by connecting the underlying technical components with the end user.

CHAPTER 07

CONCLUSION AND FUTURE WORK

7.0 Conclusion

The current glucosense_ai system provides a strong foundation for intelligent diabetes-risk monitoring by integrating glucose logging, AI-based meal analysis, machine-learning prediction, glucose trends, reports, and smart reminders into a single mobile application.

Future development can transform the system from a personal monitoring application into a more comprehensive digital-health platform. Improvements such as larger clinically validated datasets, additional medical features, advanced prediction algorithms, CGM and wearable integration, personalized recommendations, smarter alerts, healthcare-professional access, enhanced reports, and stronger security can significantly increase the usefulness of the system.

Particular attention should be given to **clinical validation and model reliability** before the prediction component is considered for higher-stakes healthcare use. Furthermore, improved model explainability, API validation, model versioning, and data privacy would make the system more transparent, maintainable, and reliable.

By gradually introducing these improvements, glucosense_ai can become a more personalized, scalable, and intelligent diabetes-management platform that supports users in understanding their glucose patterns, identifying potential risk factors, and maintaining healthier monitoring habits.

7.1 Current Technical Limitations

Although the implemented architecture provides a strong foundation, some limitations remain.

The first limitation is related to reproducibility of the machine-learning results. The current repository snapshot does not contain the training dataset, and the notebook does not preserve all final numeric outputs. Therefore, exact final model metrics need to be regenerated from a clean training run.

The second limitation is prediction-input validation. The Flask API verifies whether required fields are missing, but strict validation of types and medically reasonable ranges should be strengthened.

The third limitation concerns the derived three-class target. The original dataset contains a binary diabetes label, while the Prediabetic class is generated using glucose and HbA1c threshold rules. This should be considered when interpreting the resulting classifier.

Another technical concern is that some of the same clinical variables used to derive the target are also included among the predictive inputs. This can make classification easier for the model and may lead to performance that reflects the target-construction rules rather than fully independent risk discovery. This does not prevent the system from functioning as implemented, but it is important when interpreting the model academically.

Finally, meal-image analysis is based on AI-generated estimation and should not be treated as an exact laboratory or nutritional measurement.

7.2 Future Work

The current version of **glucosense_ai** provides an integrated mobile platform for glucose logging, meal analysis, diabetes-risk prediction, trend monitoring, reporting, reminders, and medical-profile management. The system combines a Flutter mobile application with Firebase and Supabase services, Google Gemini for meal analysis, and a Flask-based machine-learning prediction API.

Although the implemented system provides the major functions required for intelligent glucose monitoring and diabetes-risk assessment, several improvements can be introduced

in future versions. These enhancements can improve prediction reliability, personalization, clinical usefulness, security, scalability, and overall user experience.

The major areas proposed for future development are discussed below.

7.3 Enhanced Diabetes Risk Prediction

The current machine-learning component classifies users into three categories:

- **Normal**
- **Prediabetic**
- **Diabetic**

The prediction is generated using demographic, behavioural, medical, and glucose-related features. XGBoost is used as the final classification model after comparison with Logistic Regression and Random Forest.

Future work can further improve the prediction component in the following ways.

7.3.1 Larger and More Diverse Dataset

Future versions should train and validate the model using a larger and more diverse dataset containing patients from different age groups, geographic locations, lifestyles, and medical backgrounds.

A more representative dataset may improve the ability of the prediction model to generalize to users who were not represented adequately in the original training data.

The dataset should ideally contain clinically verified labels for:

- Normal individuals
- Prediabetic individuals
- Diabetic individuals

Using directly verified three-class labels would provide stronger clinical validity than relying only on derived risk categories.

7.3.2 Additional Clinical Features

Future versions can include additional health indicators such as:

- Blood pressure
- Cholesterol levels
- Triglyceride levels
- Waist circumference
- Insulin level
- Medication history
- Physical activity duration
- Sleep patterns
- Dietary habits

These additional variables may help the system build a more complete representation of the user's diabetes-risk profile.

7.3.3 Advanced Machine-Learning Models

Other machine-learning and deep-learning algorithms can also be evaluated in future versions.

Possible approaches include:

- LightGBM
- CatBoost
- Neural Networks
- Ensemble learning
- Stacking classifiers

These algorithms can be compared with the current XGBoost model using consistent evaluation metrics to determine whether prediction performance can be improved.

7.4 Clinical Validation of the Prediction System

A major future improvement would be the validation of `glucosense_ai` in a real clinical environment.

The current prediction system provides machine-learning-based risk information, but future work should include evaluation with healthcare professionals and real patient data.

Clinical validation could involve collaboration with:

- Doctors
- Diabetologists
- Nutritionists
- Medical researchers
- Hospitals or healthcare institutions

Predicted risk categories could be compared with professional medical assessment and laboratory results.

This would help determine whether the system performs consistently enough to support diabetes-risk screening in practical healthcare environments.

The system should continue to be presented as a **decision-support and awareness tool rather than a replacement for professional medical diagnosis**.

7.5 Improved Model Explainability

The current machine-learning workflow already incorporates **SHAP** and **LIME** for model interpretation.

Future versions can integrate these explanations more directly into the mobile application so that users can understand which health factors contributed most strongly to their predicted diabetes-risk category.

For example, instead of displaying only:

Prediabetic

the application could provide an explanation such as:

Your elevated HbA1c level, BMI, fasting glucose, and family history contributed significantly to the predicted risk.

Visual explanations could include:

- Feature contribution charts
- Risk-factor ranking
- Positive and negative risk influences
- Personalized explanation cards

This would make the prediction process more transparent and may increase user confidence in the system.

7.6 Personalized Health Recommendations

The current prediction API provides short guidance along with the predicted risk category.

Future versions could provide more personalized recommendations according to the user's:

- Diabetes-risk category
- Glucose history
- Meal patterns
- BMI
- Activity level

- Family history
- Recent glucose trends

Instead of displaying generic advice, the system could generate personalized recommendations such as:

- Increasing physical activity
- Reducing high-glycaemic meals
- Improving meal timing
- Monitoring fasting glucose more frequently
- Scheduling professional medical consultation when repeated high-risk patterns are identified

However, future recommendations should be designed carefully and should avoid presenting automatically generated suggestions as medical prescriptions.

7.7 Enhanced Meal Analysis

The current Meal Predictor allows users to capture or select a meal image and obtain AI-generated information related to nutritional content, glycaemic impact, and diabetes-friendly alternatives using Google Gemini.

Future versions can expand this module considerably.

7.7.1 Improved Food Recognition

The application could support more accurate recognition of:

- Multiple foods in one image
- Portion sizes
- Ingredients
- Sauces and toppings
- Mixed dishes

Improved food recognition could provide more reliable nutritional estimates.

7.7.2 Meal Risk Scoring

Each analysed meal could be assigned a simple diabetes-oriented score such as:

- Low glycaemic impact
- Moderate glycaemic impact
- High glycaemic impact

The score could be generated using estimated carbohydrate content, glycaemic load, portion size, and the user's recent glucose patterns.

7.7.3 Meal and Glucose Correlation

A valuable future feature would be to compare meals with the glucose readings recorded after those meals.

For example, the system could identify patterns such as:

Meals containing high refined-carbohydrate content are repeatedly associated with elevated post-meal glucose readings.

This would provide personalized dietary insights rather than analysing meals independently.

7.8 Integration with Continuous Glucose Monitoring Devices

The current application mainly depends on manually entered glucose readings.

Future development can integrate **Continuous Glucose Monitoring (CGM)** systems and compatible glucose meters.

With appropriate device APIs, glucose measurements could automatically synchronize with `glucosense_ai`.

Potential advantages include:

- Reduced manual data entry
- More frequent glucose measurements
- Improved glucose trend analysis
- Better Time in Range calculation
- Identification of rapid glucose changes
- More accurate personalized risk analysis

Automatic glucose synchronization could significantly improve the usefulness of the application for users who regularly monitor their glucose levels.

7.9 Wearable Device Integration

Future versions may also integrate wearable devices such as smartwatches and fitness trackers.

These devices can provide additional lifestyle information including:

- Daily step count
- Heart rate
- Exercise duration
- Calories burned
- Sleep duration
- Activity intensity

The activity information could be combined with glucose readings and medical-profile information to provide a more comprehensive assessment of health patterns.

Such integration could also improve the activity_level information currently used by the risk-prediction model.

7.10 Advanced Trends and Analytics

The current system provides glucose charts and aggregates for periods such as **7 days, 30 days, and 90 days**, including measurements such as averages, variance, Time in Range, and time-of-day patterns.

Future versions can introduce more advanced analytics.

Possible enhancements include:

- Weekly glucose comparison
- Monthly trend comparison
- Morning versus evening glucose analysis
- Meal-specific glucose trends
- Detection of repeated glucose spikes
- High-risk glucose periods
- Long-term HbA1c trend visualization
- Predicted future glucose patterns

The application could also identify unusual changes automatically and highlight them to the user.

For example:

Your average fasting glucose has increased during the last three weeks.

Such insights would provide more meaningful interpretation than displaying charts alone.

7.11 Smart Alerts and Early Warning System

The existing application provides configurable local reminders for fasting and meal-related glucose monitoring. It also includes notification scheduling and smart suppression when the related activity has already been completed.

Future development can expand reminders into an intelligent warning system.

Possible alerts include:

- Repeated high fasting glucose
- Repeated high post-meal glucose
- Multiple missed measurements
- Increasing diabetes-risk score
- Abnormal glucose patterns
- Long periods without monitoring

For example, if several consecutive glucose readings are significantly elevated, the system could display an alert recommending that the user review the pattern and consider professional medical consultation.

7.12 Improved Reports and Healthcare Sharing

The current system provides date-range summaries of glucose readings and analysed meals.

Future reports could contain more detailed information, including:

- Average glucose
- Minimum and maximum glucose
- Time in Range
- Fasting glucose patterns
- Pre-meal glucose patterns

- Post-meal glucose patterns
- Risk-prediction history
- Meal analysis
- Activity information
- Important detected trends

Reports could be exported in formats such as **PDF** so that users can share them with doctors or healthcare professionals.

A dedicated **doctor-sharing feature** could also be developed where users provide temporary or permission-based access to selected medical information.

7.13 Doctor or Healthcare Professional Module

A future version of glucosense_ai could introduce a separate healthcare-professional module.

With explicit user consent, doctors could access selected patient information such as:

- Glucose history
- Generated reports
- Diabetes-risk predictions
- Meal trends
- Abnormal glucose patterns

Healthcare professionals could review information remotely and provide appropriate feedback.

The system would require strict authorization and privacy controls so that health information could only be accessed by authorized users.

7.14 Improved API Input Validation

The current prediction API checks whether required fields are present, but future versions should implement stronger validation.

The API should verify:

- Data types
- Allowed categorical values
- Minimum and maximum values
- Missing values
- Invalid numeric inputs
- Impossible medical values

For example, validation rules could prevent clearly invalid values such as negative age, negative BMI, or impossible glucose measurements from reaching the prediction model.

A structured schema-validation mechanism would improve the reliability and security of the prediction service.

7.15 Model Versioning and Monitoring

Future versions should maintain formal metadata for each deployed machine-learning model.

Information may include:

- Model version
- Training date
- Dataset version
- Dataset hash
- Features used
- Evaluation metrics

- Python environment
- Model parameters

For example:

Model Version: 2.1

Algorithm: XGBoost

Training Date: 2026-08-20

Number of Features: 12

Target Classes: Normal / Prediabetic / Diabetic

Maintaining this information would improve reproducibility and make it easier to determine which version of the model produced a particular prediction.

The deployed model should also be monitored for performance degradation or **model drift** when newer data becomes available.

7.16 Offline Functionality and Synchronization

The application already incorporates local technologies such as Hive, SharedPreferences, and secure storage.

Future versions could provide stronger offline functionality.

Users could be allowed to:

- Record glucose readings offline
- View previously stored readings
- Access saved meals
- View cached reports
- Configure reminders

Once internet connectivity becomes available, locally stored records could automatically synchronize with the cloud database.

This would make the application more useful in areas with unstable or limited internet connectivity.

7.17 Security and Privacy Enhancements

Because glucosense_ai manages health-related and personal information, security should continue to be improved as the system evolves.

Future security improvements may include:

- Multi-factor authentication
- Biometric application lock
- Stronger encryption
- Improved Firestore security rules
- API authentication tokens
- Rate limiting
- Audit logging
- Suspicious-login detection
- Secure model API access
- Data-retention controls

Users could also be provided with options to download or permanently delete their stored information.

Regular security audits and vulnerability assessments should be performed before deployment in a larger healthcare environment.

7.18 Scalability and Performance Optimization

The application follows feature-first clean architecture with separate data, domain, and presentation layers, which provides a suitable foundation for future expansion.

As the number of users increases, future versions should perform:

- API load testing
- Database performance testing
- Query optimization
- Cloud-resource monitoring
- Caching
- Prediction API optimization
- Image compression before meal analysis
- Reduced network requests

The Flask prediction service may eventually be deployed using a production-grade application server and scalable cloud infrastructure rather than relying only on a development-style server.

This would enable multiple users to request predictions simultaneously while maintaining acceptable response times.

7.19 Cross-Platform and Accessibility Improvements

The Flutter-based implementation supports development for Android and iOS.

Future versions should be fully tested across different:

- Android devices
- iOS devices
- Screen sizes
- Operating-system versions

Accessibility improvements should also be introduced for users with visual or physical limitations.

Possible improvements include:

- Larger text options
- High-contrast interface
- Screen-reader compatibility
- Simplified navigation
- Voice input
- Multilingual support

These features would make glucosense_ai accessible to a wider population.

7.20 Future Integration with Healthcare Systems

A long-term enhancement could involve integration with electronic healthcare systems.

With appropriate standards, permissions, and institutional agreements, the application could potentially exchange selected health information with:

- Electronic Health Record systems
- Hospital systems
- Diagnostic laboratories
- Diabetes clinics

Such integration could reduce manual information entry and provide healthcare professionals with a more complete view of the user's glucose-monitoring history.

However, such integration would require strict compliance with relevant healthcare privacy, security, and data-governance regulations.